The pathwidth complex

Alexandre Linhares
Getulio Vargas Foundation (on leave) and Argo Laboratory
alex@argolab.org ORCID 0000-0002-4227-6879

Horacio Hideki Yanasse
Institute of Science and Technology, Federal University of São Paulo (UNIFESP), São José dos Campos, SP, Brazil
horacio.yanasse@unifesp.br ORCID 0000-0002-6946-9670

19 October 2026

Abstract

In 2001 we listed twelve problems from operations research, VLSI design and graph theory, the minimization of open stacks problem (MOSP) among them, as equal "plus or minus one", and gave no proofs (Linhares, 2001; Linhares and Yanasse, 2002). We now correct and extend the table, proving formally in Lean that most of them are, mathematically, one quantity, *graph pathwidth*, studied independently by communities that rarely cite one another. Eight rows are exact, counting pathwidth itself: each equals pathwidth or pathwidth plus one. Two lie in *bands*: they stay within one or two of pathwidth, but not at a fixed offset. Two are false as originally stated. We add three members the table missed: (i) the maximum wavefront of sparse matrix elimination, which brings a fourth discipline, numerical linear algebra, into the family; (ii) progressive pebbling; and (iii) the vertex search of Bienstock, Robertson, Seymour and Thomas. We call this cluster the *pathwidth complex*, as the literature uses the name *pathwidth* more than three times as often as the other eleven names combined. Since 2005, the next most active name is MOSP, and the open-stacks community produced, by way of that year's Constraint Modelling Challenge, the best exact algorithm for MOSP: the *customer search* of Chu and Stuckey (2009), which branches on the order in which customers' stacks close (also a pathwidth algorithm). We prove in Lean that two of its published dominance theorems, rules that discard a branch as no better than another, are *not true*, and that on a 34-customer counter-example instance their algorithm, with its better move off, reports an optimum of 7 where the optimum is 6. Our corrected algorithm (and a SAT solver with a checked DRAT proof) confirms the 6. Our corrected definite move, the first of the two, was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1) in the pathwidth literature; we prove our corrected algorithm sound, formally, in Lean. We re-prove all 115 benchmark optima that rested on Chu and Stuckey's algorithm, changing none. Their search is sequential; we make the refutation parallel by a split of the search tree into independent subtrees that inherit the right refuted moves from their siblings, proved sound in Lean, and with it certify the two hardest MOSP optima on record, one after 17,001 subtrees, 7.4 trillion search nodes and 2,231 core-hours on 25 cores. Ported to graphs, our corrected algorithm proves the pathwidth of all 11,534 Rome graphs, a standard graph-drawing benchmark of 10 to 110 vertices (97.1% of them within 10 minutes each), and, on every benchmark set with published exact counts, as many graphs or more than the published branch and bound of Coudert, Mazauric and Nisse (2016). In the other direction, *from pathwidth to MOSP*, a lower bound from graph minors, proved in Lean, settles in seconds instances whose refutation took core-days, and closes

the Mycielski graph M_7. All of our proofs and solvers are open source. We also release a dataset of 17,714 graphs, distinct up to isomorphism, from twenty benchmark collections for pathwidth and the problems equal to it, the MOSP instances and the Rome graphs among them, 16,634 of them with a certified pathwidth, with independent checkers. We believe ours is currently the strongest exact pathwidth solver openly available.

Contents

1 Introduction **5**

2 The equivalences, proved **10**
2.1 The problems . . . 10
2.2 What is true . . . 11

3 Why *pathwidth*: the names and the communities **18**
3.1 Pathwidth has absorbed the family . . . 19
3.2 Besides pathwidth, open stacks . . . 19
3.3 From an open-stacks challenge to a pathwidth solver . . . 19

4 An exact open-stacks search, read as a pathwidth algorithm **20**
4.1 The search decides pathwidth . . . 21
4.2 The algorithm . . . 22
4.3 The definite move is false as published, and its correction is a matching test . . . 23
4.4 The other rules . . . 26
4.5 Our corrected algorithm is sound . . . 27
4.6 The rules in pathwidth language . . . 28
4.7 A lower bound from minors . . . 31

5 Computational results and a dataset **32**
5.1 Experimental setup . . . 32
5.2 The cost of our correction, and the benchmark values . . . 33
5.3 Our corrected algorithm as a pathwidth solver . . . 36
5.4 Head to head with SageMath . . . 38
5.5 A checkable proof of a refutation . . . 40
5.6 A dataset with certified pathwidth . . . 42

6 Conclusion **43**

Data, code and proofs **45**

Acknowledgements **46**

Appendix A Formal names **54**

Appendix B Supporting material **64**
B.1 Formal definitions of eight problems . . . 64
B.2 Edge cases and a gap in a published proof . . . 65
B.3 Members outside the table . . . 65
B.4 Why the published proof of the definite move fails . . . 66
B.5 The search for smaller counterexamples . . . 66
B.6 The search for prior reports of the error . . . 66
B.7 The customer search, as published . . . 67
B.8 Implementation checks . . . 67
B.9 The filter and the runs are sound . . . 67
B.10 The split re-certification . . . 68

Appendix C The bibliometric study **69**

Appendix D A SAT formulation of MOSP **72**

Appendix E The pathwidth of sparse random graphs **74**
E.1 The picture . . . 74
E.2 The statement . . . 74
E.3 The proof, in four steps . . . 75
E.4 The benchmark generator . . . 77
E.5 The excess, and a phase diagram . . . 78
E.6 What is checked, and what is known . . . 83

Appendix F The dual of pathwidth **85**
F.1 The problem of the dual . . . 85
F.2 Blockages . . . 85
F.3 Why it is the dual . . . 86
F.4 Three theorems on maximising the dual . . . 87
F.5 What is open: the size of a blockage . . . 89

Appendix G A tabu search over minors, and its certificates **91**

1 Introduction

A factory makes a set of patterns, one at a time, and each customer's order needs some of them. A customer's stack opens when the first pattern the customer needs is made, and it closes when the last one is made. The *minimization of open stacks problem* (MOSP) asks for the order of the patterns that keeps the fewest stacks open at any one time. In the cutting industry, where the problem arose, the patterns are cutting patterns and the customers are piece types (Yanasse, 1997b). Figure 1.1(a) shows a small instance. Six customers, a to f, need four patterns: $P_1 = \{a, b\}$, $P_2 = \{b, c, d\}$, $P_3 = \{c, d, e\}$ and $P_4 = \{e, f\}$. Made in order A, $P_1P_2P_3P_4$, the patterns keep at most 3 stacks open. Made in order B, $P_1P_3P_4P_2$, they keep 5 open, because customer b is served first but waits for P_2, which comes last. Of the 24 orders, 2 reach 3 (order A and its reverse), 16 give 4 and 6 give 5.

The same matrix is a problem in VLSI design. In a *gate matrix layout* the patterns are gates, drawn as vertical lines in some order, and the customers are nets. Each net is a horizontal wire from the first gate it connects to the last, and wires that overlap must lie on different horizontal tracks. The task is to order the gates so that the fewest tracks suffice. Figure 1.1(b) and (c) draw the two orders. The wires are the same; only the gate order changes which wires overlap. Order A packs into 3 tracks and order B needs 5. The number of tracks a gate order needs is its *density*, the most wires crossing any one gate, and that is exactly the number of open stacks there. So the two problems compute the same number from the same matrix (Linhares and Yanasse, 2002, Proposition 2).

The same matrix is also a graph. Make each customer a vertex, and join two customers when some pattern needs both, so that each pattern becomes a clique (Figure 1.1(d)). Write down, for order A, the set of open stacks at each position: $\{a, b\}$, $\{b, c, d\}$, $\{c, d, e\}$, $\{e, f\}$. These sets cover every vertex and every edge, and each customer appears in a run of consecutive sets. In graph theory such a sequence is a *path decomposition*, its sets are *bags*, and its width is the size of its largest bag minus one. The least width over all path decompositions is the *pathwidth* of the graph, here 2. The construction works in both directions, so the fewest open stacks is always the pathwidth plus one (Theorem 2.3).

The same graph is, finally, a sparse symmetric matrix of numerical linear algebra, with a nonzero wherever two customers share a pattern. The frontal method eliminates its rows one at a time and must keep in memory the *wavefront*, the rows already touched but not yet eliminated. Eliminating the customers in the order $a, b, \ldots, f$ keeps at most 3 rows active at once, and no order does better: the least maximum wavefront is again the pathwidth plus one (Theorem 2.4). This fourth reading is not in our 2002 table, and we add it below.

We collected twelve such problems (Linhares and Yanasse, 2002), from operations research, VLSI design and graph theory, in the table that Table 1.1 reproduces. Each computes, on corresponding inputs, "a function $f(\Pi)$ that is either equal to the number of open stacks or closely related to it (plus or minus one)" (Linhares and Yanasse, 2002, p. 1764). The table gives a reference for each problem, but no proofs and no graph of which problem reduces to which. "Plus or minus one" has two readings. Under the *literal* reading, f is within one of the number of open stacks on every input. Under the *fixed-offset* reading, f equals the number of open stacks plus one constant, the same on every input. Only the second makes solving one problem the same as solving another. We call the problems that are, on every input, pathwidth or pathwidth plus one the *pathwidth complex*: most of the table, and three problems we originally missed. Pathwidth is

Table 1.1: Twelve problems from three fields, published as one family: eight rows are exact, counting the pathwidth row itself; two are bands; two are false as the table states them. Our own table of 2002, reproduced from Linhares and Yanasse (2002, Table 1, p. 1764) with its row order, wording and references as printed, and its caption: "Display of a family of equivalent problems studied independently on the literature". The shaded column corrects it: it gives the relation to pathwidth (pw) that is true, proved in Section 2 (Table 2.3). The last three rows, below the rule, are new: members of the family that the table missed, which bring a fourth field, numerical linear algebra.

Problem	Discipline	References	This paper
MOSP	Operations research	Yanasse (1997b); Fink and Voß (1999)	exact: pw $+1$
Gate matrix layout	VLSI design	Möhring (1990); Wing et al. (1985)	exact: pw $+1$
One-dimensional logic	VLSI design	Ohtsuki et al. (1979)	exact: pw $+1$
PLA folding	VLSI design	Möhring (1990)	false[a]
Interval thickness	Graph theory	Kashiwabara and Fujisawa (1979)	exact: pw $+1$
Node search game	Graph theory	Kirousis and Papadimitriou (1985)	exact: pw $+1$
Edge search game	Graph theory	Kirousis and Papadimitriou (1986)	band: pw to pw $+2$
Narrowness	Graph theory	Kornai and Tuza (1992)	exact: pw $+1$
Split bandwidth	Graph theory	Fomin (1998)	band: pw to pw $+1$
Graph path-width	Graph theory	Kinnersley (1992)	exact: pw (reference)
Edge separation	Graph theory	Lengauer (1981)	false[b]
Vertex separation	Graph theory	Kinnersley (1992)	exact: pw
Progressive pebbling[c]	Graph theory	Kirousis and Papadimitriou (1986)	exact: pw $+1$
Vertex search[d]	Graph theory	Bienstock et al. (1991)	exact: pw $+1$
Maximum wavefront[e]	Numerical linear algebra	Kumfert and Pothen (1997)	exact: pw $+1$

New: not in Linhares and Yanasse (2002).

[a] Exact (pw $+1$) for multiple folding, with any number of nets per track. [b] As cutwidth; the vertex separator game of the same reference is exact (pw, for graphs with an edge). [c] Added by us. Its relation to pathwidth is proved in two papers the table already cites, for other rows: Lengauer (1981, Thms 2–3) and Kirousis and Papadimitriou (1986, Thm 3.1). We prove it in Lean (Appendix B.3). [d] Added by us. Searchers occupy vertices and an invisible fugitive moves along unguarded paths. Exact on every graph with a vertex, edgeless ones included (Bienstock et al., 1991, (5.1)), and equal to node search when the graph has an edge. We prove it in Lean, following their construction for the upper bound (Appendix B.3). [e] Added by us. The least, over symmetric orderings of a sparse symmetric matrix with nonzero diagonal, of the largest wavefront (Kumfert and Pothen, 1997, §2.1); it is narrowness in another form. The source defines it but does not state the equality, which we prove in Lean (Appendix B.3).

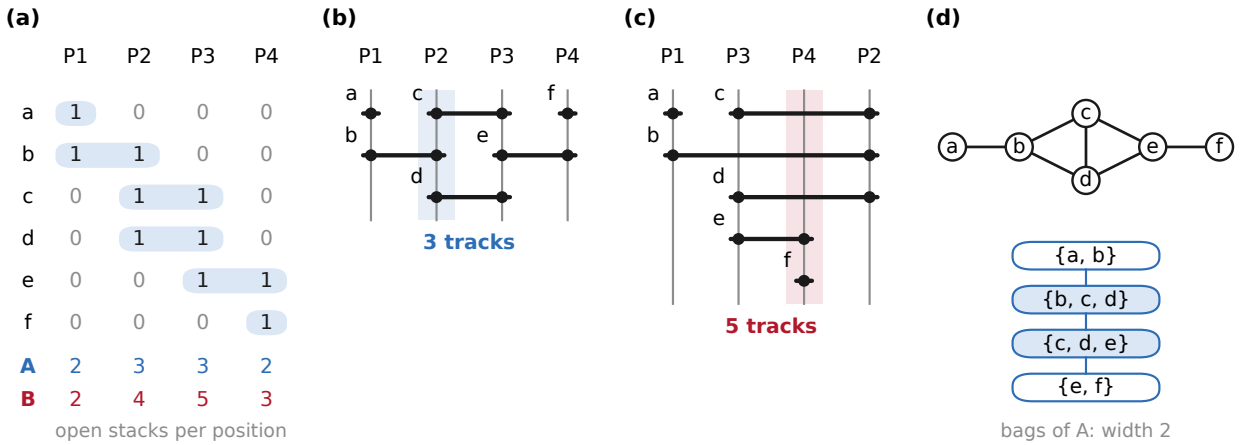


Figure 1.1: One instance, three readings, one number. (a) Six customers and four patterns, with each customer's open stack shaded in order A; order A keeps at most 3 stacks open and order B at most 5, and only 2 of the 24 orders reach 3. (b, c) The same matrix as a gate matrix layout: the same two orders need 3 and 5 tracks (the shaded gate is where the density peaks). (d) The same matrix as a *graph*, one clique per pattern: the open sets of order A are the bags of a path decomposition of width 2, so the optimum is the pathwidth plus one (Theorem 2.3).

the quantity through which every true relation in the table is proved (Section 2). It is also the name the literature uses, more than three times as often as the other eleven names combined (Section 3), so a reader who searches under "open stacks" or "gate matrix layout" misses most of what is known. The communities that could have checked the table rarely read each other: of 844 works that cite the table's papers, only six cite both an open-stacks paper and a graph-theory paper.

The table is ours, and its history explains why no one checked which of its equalities are true. It was first printed in 2001, in the first author's doctoral thesis (Linhares, 2001, Table 2.1, p. 13), supervised by the second author, which proved MOSP NP-hard and equal to gate matrix layout. Its rows were collected over the years of the doctorate, while reading the literature of each field. Each came with a reference and nothing else: no row's relation to open stacks was checked, and no published proof was restated. The table appeared again, row for row, as Table 1 of Linhares and Yanasse (2002), and it has been read since as a statement of equivalence. Garcia de la Banda and Stuckey (2007), for instance, credit the speed of their search in this journal to "the fact that the problem is fixed parameter tractable (Linhares and Yanasse 2002)". When MOSP was set as the first Constraint Modelling Challenge in 2005, the organisers' report pointed to the table for the problems equivalent to it, "including graph path-width", and credited the second author with the idea of permuting customers rather than products (Smith and Gent, 2005; Yanasse, 1997b), the idea on which Chu and Stuckey's customer search (Section 4) branches. So the table is already used to justify algorithms, and whether it is true matters. If it is true, *every result* about any one of the twelve problems transfers to *all* the others. Looking back, the table was the thesis's most important finding, and this article does what the thesis did not: it states each problem precisely, proves what is true and refutes what is not. This paper asks two questions:

1. *which of the equalities in the Linhares–Yanasse table are true?*

2. *and what follows when results, algorithms and benchmarks are moved between the twelve problems*

through pathwidth?

Three more members. The table was also incomplete. Progressive pebbling, a game in which pebbles are placed on the vertices of a graph and each vertex is pebbled once, needs exactly pathwidth plus one pebbles (Kirousis and Papadimitriou, 1986, Thm 3.1), and Lengauer's black-white pebbling of a directed acyclic graph equals the pathwidth of its undirected version plus one (Lengauer, 1981, Thm 2). Both theorems are in papers that the table already cites, for its edge-search and edge-separation rows, yet pebbling itself is not among its problems. Bienstock et al. (1991, (5.1)) search for a fugitive who hides in the vertices, not the edges, and need exactly pathwidth plus one searchers on every graph with a vertex, including the edgeless graphs where the table's node search row fails. From numerical linear algebra, the wavefront of the frontal method, met above (Kumfert and Pothen, 1997, §2.1), is the problem of the table: the least maximum wavefront is the pathwidth of the matrix's graph plus one, on every nonempty matrix. We add the three as the last rows of Table 1.1, marked as new, and prove all three relations in the Lean proof assistant (Appendix B.3). For pebbling and the vertex search, what is new is the membership: the equivalences were published, but the table did not include them. For the wavefront the equality itself is ours: it is elementary, as the wavefront is narrowness read in another form, but no source we have read states it.

What we find.

- *The table, settled 25 years later.* Eight of the twelve rows are exact: on every input, outside degenerate cases such as an empty graph, the problem equals pathwidth or pathwidth plus one (the pathwidth row itself is one of the eight). Two rows, split bandwidth and edge search, lie in *bands*: within one and two of pathwidth, but not at a fixed offset. Two rows, PLA folding and edge separation, are false as stated, with counterexamples. A variant of each false row is exact. Three problems outside the table are exact members too: progressive pebbling, a vertex search, and the maximum wavefront of sparse matrix elimination (Section 2).

- *One name.* The literature has settled on "pathwidth", which has more than three times as many relevant works as the other eleven names combined. Since 2005, after pathwidth, the most active name of the twelve is MOSP, and the best exact search for the whole family came from that community, by way of the 2005 Constraint Modelling Challenge (Section 3).

- *Two false theorems, and our correction.* The best exact algorithm for MOSP, the *customer search* of Chu and Stuckey (2009), branches on the order in which customers' stacks close, and it is also a pathwidth algorithm. We prove in Lean, however, that two of its published dominance theorems, the *definite move* and the *better move*, are false, as are their later restatements (Chu, 2011; Fink, 2012): each can discard the last remaining solution at a node of the search tree. We provide a counterexample instance: our implementation of their algorithm, with its better move off, obtains an optimum of 7, while our corrected version obtains 6, provable also by a SAT (Boolean satisfiability) solver. This is a theorem checked in Lean: the whole run of their algorithm on the instance, and its optimum of 6 (Theorem 4.9). Our corrected definite move was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1), in the exact pathwidth literature; we show that it can be tested by one bipartite matching per

candidate, and prove our corrected algorithm sound. Our correction costs at most 2% more search nodes on each group of benchmarks we measured.

- *A pathwidth solver.* Ported to graphs, our corrected algorithm is an exact pathwidth solver, which we release. Within 600 seconds per graph it proves the pathwidth of 11,194 of the 11,534 Rome graphs, a graph-drawing benchmark of 10 to 110 vertices, and with the minor step below and no time limit, all of them; all 50 trees of VSPLIB, a benchmark library for vertex separation; and 39 of its 73 Harwell–Boeing graphs, the sparsity patterns of engineering matrices. On each set this is as many graphs or more than the published branch and bound of Coudert et al. (2016). A lower bound from graph minors, the pathwidth of a contracted copy of the graph (Theorem 4.11, proved in Lean), runs before each refutation: it proves the pathwidth of the Mycielski graph M_7 in a fraction of a second, 271 of the 340 Rome graphs left open at 600 s, and MOSP optima that took up to 1,128 core-hours to refute (Sections 4.7 and 5).

- *A parallel refutation, proved sound.* Chu and Stuckey's algorithm is sequential, and its two reaching ingredients, the nogood memo and the old moves a node inherits from the siblings searched before it, are what stop the textbook split of a depth-first search from being a proof. Our split (Section 5.2, Appendix B.10) expands the top of the tree exactly as the sequential search would, hands every child to its own worker with the old moves it would have inherited and an empty memo, re-splits any task that exceeds its node cap, and claims a refutation only when every task has answered. We prove in Lean that every such run lifts to a run of our corrected algorithm (Appendix A, A.78), so the parallel answer is a certificate and not a heuristic, and we measure its price: the memo hits the tasks cannot share and the nodes of tasks that were split again. It is what certified the two hardest MOSP optima on record, 21 for `Random-125-125-2-3_0` after 9,665 subtrees, 4.25×10^{12} search nodes and 1,128 core-hours, and 25 for `Random-125-125-2-2_0` after 17,001 subtrees, 7.38×10^{12} nodes, 7.4 trillion states visited, and 2,231 core-hours, on 25 cores; on one core each would have taken months, and neither had been certified before. The split scales with the cores it is given, up to those two losses, and the ledger of finished tasks survives a crash, so the next harder instance is a budget and not a new method.

- *Benchmarks re-certified, and a dataset.* Our corrected algorithm re-proved all 115 benchmark optima, on 113 distinct graphs, that rested on Chu and Stuckey's algorithm, and none changed. We release 17,714 graphs, distinct up to isomorphism, from twenty benchmark collections of MOSP, VLSI layout, vertex separation, graph drawing and treewidth; 16,634 have a certified pathwidth with a layout that attains it, and so a certified value for every problem of the pathwidth complex (Section 5).

How to read the paper. Section 2 states the twelve problems and proves what is true. Section 3 measures which names and communities the literature uses. Section 4 reads the exact open-stacks search as a pathwidth algorithm, finds the two false rules and proves our corrected algorithm sound. Section 5 reports what our corrected algorithm costs, how it performs as a pathwidth solver, and the dataset, and Section 6 returns to the two questions this paper asks. Every theorem we state is machine-checked in the Lean 4 proof assistant, unless the text says otherwise. The reader need not read Lean: each proof is given or cited in the text, and Appendix A maps

each result to its formal statement. Appendix B holds the supporting material, Appendix C the bibliometric method, and Appendix E a lower bound on the pathwidth of sparse random graphs, proved in plain language and checked in Lean. The Lean development, both solvers, the dataset and an independent checker for each are openly available, the code and proofs as open source under the MIT licence (page 45).

2 The equivalences, proved

This section answers the first of our two questions: which rows of the Linhares–Yanasse table are true, and in what sense. Section 1 separated two readings of "plus or minus one". We call a row *exact* when it meets the fixed-offset reading: its value is pathwidth, or pathwidth plus one, on every input. We call it a *band* when it meets only the literal reading: its value stays within a fixed distance of pathwidth, and both ends of that range occur. We call it *false* when it meets neither. The answer is that eight rows are exact, two are bands and two are false. Section 2.1 states the problems. Section 2.2 proves the answer in four steps: the hub theorem, which ties MOSP to pathwidth; the other exact rows; the bands; and the false rows. Table 2.3 and Figure 2.1 summarise it.

All theorems below are verified in the Lean proof assistant, unless the text says otherwise; Appendix A lists the formal statements. Where a result is published, we cite the published proof and do not repeat it.

Notation. Table 2.1 lists the symbols used in the rest of the paper. Graphs are finite and simple. A 0/1 matrix M has the customers as rows and the patterns as columns; in the VLSI problems the rows are nets and the columns gates. Row r is *active* at position j of a column order π if it has 1s in two columns x, y with $\pi(x) \le j \le \pi(y)$, possibly $x = y$: its stack is open there. "M has a 1" excludes only the all-zero matrix. One letter needs care. In this section k is the bound in each decision problem, so for pathwidth it bounds a width. In Section 4 k bounds the number of open stacks, which is one more than a width; we say so again there.

2.1 The problems

Each problem is a decision problem with an instance and an integer k; its value is the least k for which the answer is yes. Four of the twelve carry the rest of the paper, and we define them in full. Figure 1.1 shows each on one instance.

Pathwidth $\mathrm{pw}(G)$**.** Is there a sequence of bags $X_1, \dots, X_r \subseteq V$ covering V, with every edge inside some bag, $X_i \cap X_\ell \subseteq X_j$ whenever $i \le j \le \ell$, and $|X_i| \le k+1$? (Kinnersley, 1992, p. 346). The condition on $X_i \cap X_\ell$ is the *interval property*: the bags that contain a vertex are consecutive.

Vertex separation $\mathrm{vs}(G)$**.** Is there a layout L such that for every $i < n$ at most k vertices u with $L(u) \le i$ have a neighbour v with $L(v) > i$? (Kinnersley, 1992, p. 346).

MOSP $Z(M)$**.** Given a 0/1 matrix M of customers by patterns, is there an order of the patterns with at most k customers active at every position? (Linhares and Yanasse, 2002, eqs. (1)–(2)).

Table 2.1: Notation. Each symbol has one meaning throughout the paper; the symbols of the search are defined in Section 4.1.

Symbol	Meaning
$G = (V, E)$, $n = \|V\|$	a graph, its vertices and edges; in Section 4 the vertices are the customers, $V = C$
L	a *layout*, a bijection $V \to \{1, \ldots, n\}$
$X_1, \ldots, X_r$	the bags of a path decomposition
M; C; P	a 0/1 matrix; its rows (customers, nets); its columns (patterns, gates)
G_M	the *MOSP graph*: vertex set C, two customers adjacent when some pattern has a 1 in both rows; each pattern becomes a clique
H	the *net graph* of a circuit: two nets adjacent when they share a gate
π	an order of the columns
$Z(M)$, $t(M)$	the MOSP value (fewest open stacks) and the gate matrix layout value (fewest tracks)
pw, vs	pathwidth and vertex separation
θ, ns, mns, es, pes, ν, sb, cw, mcw, VSG, pla	the values of the other problems (Table 2.2)
k	the bound of a decision problem; in Section 4, a bound on open stacks, so width $k - 1$
w	a width, as in "pathwidth w"
$N[c]$	the closed neighbourhood of c: c and its neighbours
∂T, $b(T)$	the border of a vertex set T, the vertices outside T with a neighbour in T; $b(T) = \|\partial T\|$

Gate matrix layout $t(M)$**.** Given a net–gate matrix, is there an order of the gates and an assignment of nets to k tracks such that nets active at a common position get different tracks? (Möhring, 1990, p. 18); (Wing et al., 1985, Problem 1).

The other eight problems are needed only to read the table. Table 2.2 gives each in one line; Appendix B.1 gives the formal definitions, with the sources.

2.2 What is true

Table 2.3 gives the answer row by row, and Figure 2.1 draws it as a graph around pathwidth. The rest of this subsection proves it.

The hub. Two theorems tie the table to pathwidth. The first joins the two graph rows, vertex separation and pathwidth. The second joins MOSP to pathwidth, and through it all four matrix problems. The second rests on one lemma about path decompositions. For a vertex v, let $\text{first}(v)$ and $\text{last}(v)$ be the least and greatest indices of bags containing v; by the interval property, $v \in X_i$ exactly when $\text{first}(v) \le i \le \text{last}(v)$.

Lemma 2.1 (cliques sit in a bag)**.** *In a path decomposition of G, every clique of G lies inside one bag (Appendix A, A.1).*

Proof. The bags containing a vertex u of the clique form the interval $[\text{first}(u), \text{last}(u)]$. Two vertices of the clique are adjacent, so their edge lies in a bag and their intervals meet. Pairwise intersecting

Table 2.2: The other eight problems of the Linhares–Yanasse table look unrelated (two circuit layouts, two search games and four graph problems), yet each asks for the least k that is possible. One line each. Formal definitions are in Appendix B.1.

Problem	In one line
One-dimensional logic	Order the gates of a circuit so that the intervals spanned by the nets overlap at most k at any point. Its graph is the *net graph* H, which joins two nets that share a gate. A boundary variant pins two gates to the ends.
PLA folding $\mathrm{pla}(M)$	Gate matrix layout in at most k tracks, with at most two nets on a track. With no limit per track (*multiple folding*) it is gate matrix layout itself.
Interval thickness $\theta(G)$	Add edges to G to make an interval graph whose largest clique has at most k vertices.
Node search $\mathrm{ns}(G)$	Searchers are placed on and removed from vertices. An edge is cleared when both its ends hold a searcher, and becomes contaminated again if a searcher-free path joins it to a contaminated edge. Clear every edge with at most k searchers at once. $\mathrm{mns}(G)$ is the same with no recontamination allowed.
Edge search $\mathrm{es}(G)$	Node search with a third move: sliding a searcher along an edge clears it. $\mathrm{pes}(G)$ is the same over *progressive* strategies, which never recontaminate an edge.
Narrowness $\nu(G)$	Add the vertices one at a time in some order, and drop each as soon as all its neighbours have been added. Never hold more than k.
Split bandwidth $\mathrm{sb}(G)$	Repeatedly split a vertex into an edge, sharing its neighbours between the two ends, so that the result has a layout in which adjacent vertices are at most k apart (its *bandwidth*).
Edge separation	The table's reference (Lengauer, 1981) supports three readings: cutwidth $\mathrm{cw}(G)$, the most edges crossing a gap of a layout; modified cutwidth $\mathrm{mcw}(G)$, the most edges passing strictly over a vertex; and Lengauer's vertex separator game $\mathrm{VSG}(G)$, a pebbling game that counts the unpebbled vertices adjacent to pebbled ones.

Table 2.3: The original Linhares–Yanasse table, settled 25 years later. Eight rows are exact, two are bands and two are false, and every true relation is a relation to pathwidth. *Relation*: what is true and proved in Lean, on the graph the problem lives on (the MOSP graph or the net graph H for the matrix problems, the input graph otherwise). *Published source*: the published work the row rests on; the Lean proofs are listed in Appendix A. *Status*: *exact*, a fixed offset; *band*, an inequality whose ends both occur; *false*, the table's claim fails, with a counterexample family proved in Lean.

Problem	Published source	Relation	Status
MOSP	Yanasse (1997a); Fellows and Langston (1989); Linhares and Yanasse (2002)	$Z(M) = \mathrm{pw}(G_M) + 1$ (M has a 1)	exact
Gate matrix layout	Möhring (1990)	$t(M) = Z(M) = \mathrm{pw}(G_M) + 1$	exact
One-dim. logic	Ohtsuki et al. (1979)	tracks $= \theta(H) = \mathrm{pw}(H) + 1$; boundary variant $\geq \mathrm{pw}(H) + 1$	exact
PLA folding	Möhring (1990)	$\max(\mathrm{pw} + 1, \lceil \lvert C \rvert / 2 \rceil) \leq$ pla, gap unbounded; multiple folding $= \mathrm{pw} + 1$	false; variant exact
Interval thickness	Möhring (1990); Kirousis and Papadimitriou (1985)	$\theta = \mathrm{pw} + 1$ ($V \neq \emptyset$)	exact
Node search	Kirousis and Papadimitriou (1985, 1986); Bienstock and Seymour (1991)	$\mathrm{ns} = \mathrm{mns} = \mathrm{vs} + 1 = \mathrm{pw} + 1 = \theta$ ($E \neq \emptyset$)	exact
Edge search	Ellis et al. (1994); Kirousis and Papadimitriou (1986)	$\mathrm{vs} \leq \mathrm{es} \leq \mathrm{vs} + 2$, also for pes	band
Narrowness	Kornai and Tuza (1992)	$\nu = \mathrm{pw} + 1$ ($V \neq \emptyset$)	exact
Split bandwidth	Fomin (1998)	$\mathrm{pw} \leq \mathrm{sb} \leq \mathrm{pw} + 1$	band
Pathwidth	Kinnersley (1992)	the reference	exact
Edge separation[a]	Lengauer (1981)	cw, mcw unbounded against pw (stars); $\mathrm{VSG} = \max(1, \mathrm{vs})$	false; VSG exact
Vertex separation	Kinnersley (1992)	$\mathrm{vs} = \mathrm{pw}$	exact
Not in the Linhares–Yanasse table			
Progressive pebbling	Lengauer (1981); Kirousis and Papadimitriou (1986)	$\mathrm{mpb} = \mathrm{pw} + 1$ ($V \neq \emptyset$), $= \mathrm{ns}$ ($E \neq \emptyset$)	exact
Vertex search	Bienstock et al. (1991)	$= \mathrm{pw} + 1$ ($V \neq \emptyset$), $= \mathrm{ns}$ ($E \neq \emptyset$)	exact
Maximum wavefront	Kumfert and Pothen (1997)	$\min \mathrm{wf} = \mathrm{pw} + 1 = \nu$ (matrix nonempty)	exact

[a] Also misattributed: what Lengauer (1981) proves exactly is the vertex separator game, which is the table's vertex separation row.

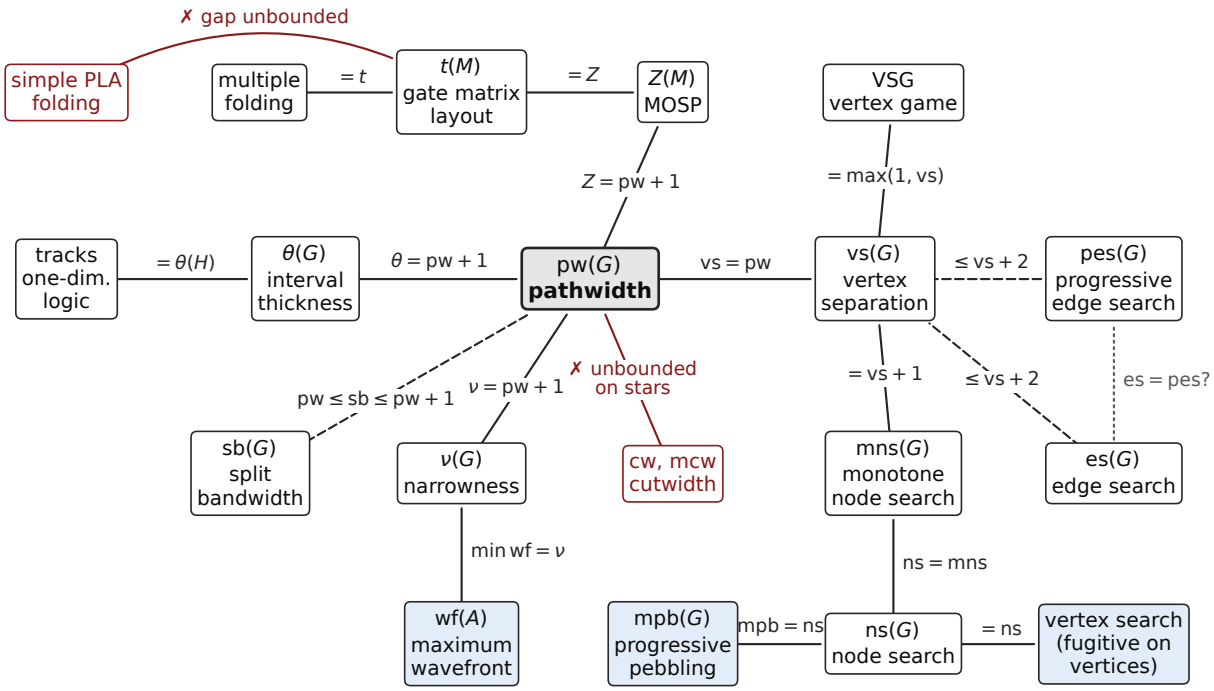


Figure 2.1: Pathwidth is the hub of the table: every exact relation is one or two steps from it, and the only failures are the two false claims, in dark red. Solid edges are exact relations and dashed edges bands, both proved in Lean; the dotted edge is LaPaugh's es $=$ pes, the one relation not proved, which no row needs. Each false claim is refuted by a counterexample family proved in Lean. The four matrix problems (top left) live on the MOSP graph G_M or the net graph H. Hypotheses are omitted: M has a 1, $V \neq \emptyset$, or G has an edge, as in Table 2.3. Two edges that follow from the others, mns $= \theta$ and ns $=$ vs $+ 1$, are not drawn. The shaded nodes on the bottom row are members the table missed. The vertex search edge holds whenever G has an edge, and the vertex search is pw $+ 1$ on every graph with a vertex (Bienstock et al., 1991, (5.1)) (Appendix A, A.84).

intervals of a line have a common point (the Helly property): let a be the largest left end; every right end is at least every left end, so at least a, and a lies in all of them. So all the intervals share an index i, and the clique lies in X_i. □

The first theorem is known. It says the two graph rows of the table are the same quantity.

Theorem 2.2 (vertex separation is pathwidth; known)**.** *For every graph,* $\mathrm{vs}(G) = \mathrm{pw}(G)$ *(Appendix A, A.2).*

This is Theorem 3.1 of Kinnersley (1992). Both directions are constructions. From a layout of separation k, each bag is the set counted at a cut together with the next vertex. From a decomposition, ordering the vertices by first gives a layout of the same width.

The second theorem is the one the rest of the paper uses. In Figure 1.1(d), the sets of customers active at each step of the good order are the bags of a path decomposition of the MOSP graph. The theorem says that this always works, in both directions.

Theorem 2.3 (MOSP is pathwidth plus one)**.** *If M has a 1, then* $Z(M) = \mathrm{pw}(G_M) + 1$ *(Appendix A, A.3).*

Proof. ($\leq$) Take a decomposition of G_M of width k. The rows of each column p form a clique, so by Lemma 2.1 they lie in a bag $X_{\beta(p)}$. Order the columns by $\beta(p)$. If row r is active at the position of column p, it has columns x, y with $\beta(x) \leq \beta(p) \leq \beta(y)$, and $r \in X_{\beta(x)} \cap X_{\beta(y)}$, so the interval property gives $r \in X_{\beta(p)}$. At most $k+1$ rows are active there.

($\geq$) Take an order with at most $Z = Z(M)$ active rows at every position. The active sets, with a singleton bag for each row with no 1, form a path decomposition: rows sharing a column are both active at its position, and a row is active on an interval of positions. Its width is $Z - 1$, using $Z \geq 1$. □

An independent, graph-theoretic route to the same equality is Theorem 2 of Suchan and Todinca (2007): every minimal interval completion of a graph with an edge clique cover is obtained by ordering the cover's cliques, and a MOSP matrix's patterns are such a cover of G_M. For the table, this settles the MOSP row as exact, with offset one, on the MOSP graph. Read as a cutting problem, Theorem 2.3 is Proposition 5 of Yanasse (1997a), which turns each pattern into a clique on its piece types. Read as a layout problem, it is Theorem 7 of Fellows and Langston (1989), with the column-expansion Lemma 4.1 of Fellows and Langston (1987). Linhares and Yanasse (2002, Prop. 2) identify MOSP with gate matrix layout. The proof above needs no assumption that the instance is reduced.

The graph matters. The MOSP literature also uses a second graph, with the patterns as vertices, joined when they share a customer, to split an instance into independent parts (Yanasse and Senne, 2010). It is natural to expect its pathwidth to measure the open stacks as well; it does not, in either direction. One pattern shared by four customers who each have a private pattern has $Z = 4$, while its pattern graph is the star $K_{1,4}$, of pathwidth 1 (Appendix A, A.4). A single customer with many patterns errs the other way.

The exact core. The other exact rows attach to Theorems 2.2 and 2.3 by published arguments. We do not reprove them on paper; each item below cites the published proof, and the Lean proofs are listed in Appendix A. We call the problems that equal pathwidth or pathwidth plus one on every input the *exact core*, the members of the pathwidth complex of Section 1: the eight exact

rows, with multiple folding and the vertex separator game, and the three members outside the table (Appendix B.3), thirteen problems, or twelve if multiple folding is counted with gate matrix layout, which it equals.

Theorem 2.4 (the exact core)**.**

1. *Gate matrix layout is MOSP on the same matrix: if M has a 1, $t(M) = Z(M) = \mathrm{pw}(G_M) + 1$. For a fixed gate order the fewest tracks equals the most nets active at one position, by the left-edge algorithm (Möhring, 1990, p. 31) (Appendix A, A.5).*

2. *Interval thickness: if $V \neq \emptyset$, $\theta(G) = \mathrm{pw}(G) + 1$ (Möhring, 1990, Prop. 3.5) (Appendix A, A.6).*

3. *One-dimensional logic: if every net meets at least one gate, the fewest tracks is $\theta(H)$; if at least one net meets a gate (the matrix has a 1), it is $\mathrm{pw}(H) + 1$ (Ohtsuki et al., 1979, Thm 3) (Appendix A, A.7).*

4. *Narrowness: if $V \neq \emptyset$, $\nu(G) = \mathrm{pw}(G) + 1$, and for each order σ, $\nu(\sigma) = \mathrm{vs}(\sigma^{\mathrm{rev}}) + 1$ (Kornai and Tuza, 1992, Prop. 3.1) (Appendix A, A.8).*

5. *Node search: for every graph $\mathrm{ns}(G) = \mathrm{mns}(G)$, and if $E \neq \emptyset$, $\mathrm{ns}(G) = \mathrm{vs}(G) + 1 = \mathrm{pw}(G) + 1 = \theta(G)$ (Kirousis and Papadimitriou, 1985, Thm); (Kirousis and Papadimitriou, 1986, Thms 2.3, 4.1) (Appendix A, A.9).*

6. *Multiple folding: the fewest tracks is $t(M) = \mathrm{pw}(G_M) + 1$ (Möhring, 1990, Thm 3.14) (Appendix A, A.10).*

7. *Lengauer's vertex separator game: $\mathrm{VSG}(G) = \max(1, \mathrm{vs}(G))$ (Lengauer, 1981, Thm 4) (Appendix A, A.11).*

8. *Progressive pebbling: if $V \neq \emptyset$, $\mathrm{mpb}(G) = \mathrm{pw}(G) + 1$ (Kirousis and Papadimitriou, 1986, Thm 3.1) (Appendix A, A.83).*

9. *Maximum wavefront: for a symmetric matrix with nonzero diagonal and graph G, $V \neq \emptyset$, the least maximum wavefront is $\mathrm{pw}(G) + 1 = \nu(G)$, and for each order the maximum wavefront is vs of that order plus one (Appendix A, A.85).*

10. *Vertex search: if $V \neq \emptyset$, the least n admitting a successful search of Bienstock et al. (1991, §5) with every $|X_i| \leq n$ is $\mathrm{pw}(G) + 1$, whether or not the search is monotone, and it equals $\mathrm{ns}(G)$ when G has an edge (Bienstock et al., 1991, (5.1)) (Appendix A, A.84). The formal proof follows theirs for the upper bound; for the lower bound, where they use blockages, it reduces to node search.*

For the table, this makes the gate matrix layout, one-dimensional logic, interval thickness, node search and narrowness rows exact, and gives an exact variant of each false row. A single certified pathwidth therefore answers every problem in the exact core; Section 5.6 uses this.

Bands. Two rows are not fixed offsets of pathwidth, but stay within a fixed distance of it.

Theorem 2.5 (bands)**.** *For every graph, $\mathrm{pw}(G) \leq \mathrm{sb}(G) \leq \mathrm{pw}(G) + 1$ (Fomin, 1998, Thm 8), and $\mathrm{vs}(G) \leq \mathrm{es}(G) \leq \mathrm{vs}(G) + 2$, and the same for* pes *(Ellis et al., 1994, Thm 2.1) (Appendix A, A.12).*

Every end of both bands occurs. K_2 has sb $=$ pw $= 1$ and es $=$ vs $= 1$; the star $K_{1,3}$ has pw $= 1$ and sb $= 2$, and vs $= 1$ with es $= 2$; and $K_{3,3}$ has vs $= 3$ with es $= 5$. These values are computed by brute force from each quantity's definition, not proved in Lean.[1] Fomin (1998) assumes a connected graph with at least two vertices; the Lean proof needs neither. LaPaugh's equality es $=$ pes (LaPaugh, 1993) is the one relation we state and do not prove, and no row needs it.

For the table, this means the following. Relative to the number of open stacks $Z = \mathrm{pw} + 1$, split bandwidth lies in $[Z-1, Z]$ and edge search in $[Z-1, Z+1]$. Both stay within one of Z, so they meet the literal reading of "plus or minus one". Since both ends occur, neither is a fixed offset, so neither meets the fixed-offset reading.

False rows. Two rows fail even the literal reading: no constant bounds their distance from pathwidth.

Theorem 2.6 (two rows are false)**.**

1. *PLA folding: for a matrix with a 1 and net set C,* $\max(\mathrm{pw}(G_M)+1, \lceil |C|/2 \rceil) \le \mathrm{pla}(M)$*, and the gap to* $\mathrm{pw}+1$ *is unbounded. The* 5×5 *identity needs 3 tracks while* $\mathrm{pw}(G_M)+1 = 1$*, and the path P_n as a matrix, one gate per edge, has $t = 2$ and* $\mathrm{pla} \ge \lceil n/2 \rceil$*, so the gap is unbounded on connected instances too (Möhring, 1990, Prop. 3.15) (Appendix A, A.13).*

2. *Edge separation read as cutwidth:* $\mathrm{pw} \le \mathrm{cw}$ *always, but* $\mathrm{cw}(K_{1,n}) \ge n/2$ *and* $\mathrm{mcw}(K_{1,n}) \ge n/2 - 1$ *while* $\mathrm{pw}(K_{1,n}) = 1$*; so neither cutwidth reading is within a constant of pathwidth (Appendix A, A.14).*

For the table, these rows are false as stated. Each has an exact variant (Theorem 2.4): the PLA claim holds for multiple folding, and the edge-separation reference proves exactly the vertex separator game.

Several published equalities need a nondegeneracy hypothesis, and one published proof has a gap that we correct; Appendix B.2 gives the details. Appendix B.3 adds three problems outside the table to the exact core.

Summary. Eight of the twelve rows of the Linhares–Yanasse table are exact, counting the pathwidth row itself, which is the reference. The other seven are pathwidth, or pathwidth plus one, exactly: MOSP, gate matrix layout, one-dimensional logic, interval thickness, node search, narrowness and vertex separation. So are two variants of the false rows, multiple folding and Lengauer's vertex separator game. Split bandwidth and edge search are bands of width one and two, within one of the number of open stacks but not at a fixed offset. Simple PLA folding and edge separation are false as stated. Three problems the table does not list are exact as well: progressive pebbling, the vertex search of Bienstock et al. (1991), and the maximum wavefront of Kumfert and Pothen (1997). One theorem is stated and not proved, LaPaugh's es $=$ pes, and no row needs it. One of the table's references, Kashiwabara and Fujisawa (1979), is one we could not obtain; its row is proved from Möhring (1990) and Kirousis and Papadimitriou (1985).

[1] $\mathrm{sb}(K_{1,3}) \ge 2$ is also argued by hand: splits of a tree are trees and keep at least three leaves.

3 Why *pathwidth*: the names and the communities

Section 2 showed that eight of the twelve rows of the Linhares–Yanasse table are one quantity. A result proved under any of their names therefore holds under all of them. But a reader who searches under one name will not find a result published under another. This section asks two questions about the literature. Which name does it use for the quantity? And, besides that name, which community has been most active? The answers are pathwidth, by far (Figure 3.1), and open stacks (Table 3.1). The second answer explains where the best exact algorithm for the family came from, and leads to Section 4.

How we counted. We searched OpenAlex (Priem et al., 2022) on 29 September 2026 for each problem's name and its common variants, in titles and abstracts, within Computer Science, Mathematics, Engineering and Decision Sciences. A search hit need not use the name in the table's sense: "narrowness" matches term rewriting, and "path-width" matches roads and tool paths. So we checked the 2,271 hits and kept the 1,593 that are about one of the twelve problems. The 700 hits for the eleven smaller names were judged one by one, by one annotator with model assistance; the 1,571 pathwidth hits were filtered by a stated rule, checked on samples. A mechanical phrase rule agrees with the judgements on 78% of the 700 and leads to the same conclusions. The counts are lower bounds, since a work can study a problem without naming it in its abstract. Appendix C gives the method in full, the robustness check and the counts per name. The census covers the twelve names of the table. The three members we add (Section 1) are not counted: progressive pebbling and the vertex search are graph-theory names, and the maximum wavefront belongs to a fourth discipline, numerical linear algebra.

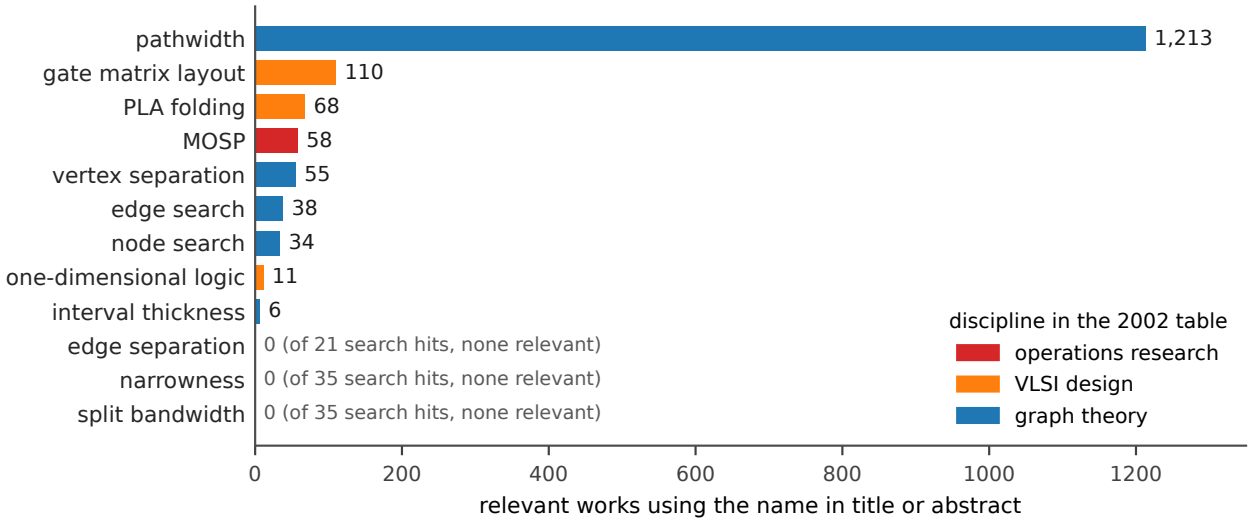


Figure 3.1: Pathwidth has more relevant works than the other eleven names combined, three times over, and three names have none. Works using each problem's name, in title or abstract, in the sense of the Linhares–Yanasse table; all years to September 2026. Colour gives the discipline that the table assigns the problem.

Table 3.1: Besides pathwidth, the most active name over 2005–2024 is MOSP, although vertex separation led in 2015–2019. Relevant works per five-year period, for the three names with the most works over 2005–2024; the next are edge search (28) and node search (20).

Name	2005–09	2010–14	2015–19	2020–24	2005–24
Pathwidth	114	179	267	311	871
MOSP	17	17	5	12	51
Vertex separation	7	10	15	0	32

Table 3.2: The open-stacks papers are read apart from graph theory. Works citing the papers that the Linhares–Yanasse table cites, by the discipline of the papers they cite. Of 135 works citing an operations-research paper, 6 also cite a graph-theory paper (4%); of 285 citing a VLSI paper, 65 do (23%). The 2002 paper itself counts as operations research. OpenAlex, 29 September 2026; 844 citing works in all.

		… of which also cite		
Works citing	All	operations research	VLSI	graph theory
operations research	135	–	14	6
VLSI	285	14	–	65
graph theory	507	6	65	–

3.1 Pathwidth has absorbed the family

Figure 3.1 gives the counts. Pathwidth has 1,213 relevant works, 21 times as many as MOSP and more than three times as many as the other eleven names combined (380). Three names have no relevant work at all: narrowness, split bandwidth and edge separation. Two more are rare: interval thickness has 6 works and one-dimensional logic 11. Pathwidth is also the only name still growing: its 311 works in 2020–2024 are more than the whole history of any other name (Appendix C). So anything one would want to know about algorithms for this class of problems was most likely published under "pathwidth".

3.2 Besides pathwidth, open stacks

The VLSI names belong to the 1980s: since 2005 gate matrix layout has 9 relevant works and PLA folding 6. Table 3.1 shows the three names with the most works over 2005–2024. MOSP is second, with 51, ahead of vertex separation (32) and edge search (28). Vertex separation led only in 2015–2019. The open-stacks community, in operations research, is thus the one most active besides pathwidth.

It has also worked largely apart from graph theory. Table 3.2 counts the 844 works that cite the papers of the table, by the disciplines of the papers they cite. Only six works cite both an open-stacks paper and a graph-theory paper. Two are by the table's authors, one is open-stacks work on graph properties, and three are pathwidth or vertex-separation work from computer science. The VLSI papers are read with graph theory about five times as often.

3.3 From an open-stacks challenge to a pathwidth solver

It is not a coincidence that the best exact algorithm for MOSP came from the open-stacks community and is also, on our measurements (Section 5.3), among the strongest exact algorithms for

pathwidth. The account is in Chu and Stuckey (2009, §1). MOSP was the subject of the first Constraint Modelling Challenge (Smith and Gent, 2005), posed in May 2005, which drew 13 entries. The fastest entry, by García de la Banda and Stuckey, combined dynamic programming with branch and bound and was an order of magnitude faster than any other (Garcia de la Banda and Stuckey, 2007). Chu and Stuckey (2009) built on it. They branched on which customer's stack closes next rather than on which pattern is made next, recorded the states already refuted, added dominance rules, and became five to six orders of magnitude faster on hard instances. Their solver closed every open instance of the challenge within ten seconds. MOSP's busiest decade in Table 3.1, 2005–2014, is the decade of this work.

The customer search reads an instance only through its MOSP graph (Chu and Stuckey, 2009, p. 4), so it decides pathwidth (Section 4). Ported to graphs, our corrected version of it proves more instances than reported on the benchmarks where published exact pathwidth results exist, those of Coudert et al. (2016). It proves the pathwidth of 11,194 of the 11,534 Rome graphs (97.1%), against their 95.6%, both within 600 seconds per graph, and with the minor step of Section 4.7 and no time limit, all 11,534. It proves all 50 VSPLIB trees, against their 30, and 39 of the 73 Harwell–Boeing graphs, against their 26. These are comparisons with published numbers obtained on other hardware; the head-to-head run against their solver on one machine, which proves everything it proves and 218 graphs more, is Section 5.4, and Table 5.3 in Section 5 gives the full count.

Two consequences shape the rest of the paper. First, a result on any of the twelve problems should be stated, and searched for, as a result on pathwidth. Section 4 does this for the customer search. It finds that two of the search's published dominance theorems are false, and that the first, the definite move, is in its corrected form a lemma already known in the pathwidth literature. Second, a benchmark for any problem in the exact core is a pathwidth benchmark, which is how Section 5.6 builds its dataset.

4 An exact open-stacks search, read as a pathwidth algorithm

If MOSP is pathwidth (Theorem 2.3), then the best exact MOSP algorithm is a pathwidth algorithm, and its rules can be checked against what the pathwidth literature knows. This section does that for the *customer search* of Chu and Stuckey (2009), the search that Section 3 traced from the open-stacks challenge. It searches over the orders in which the customers' stacks close, not over pattern sequences, and it prunes with dominance rules. Three of the rules are stated as theorems (their Theorems 1, 2 and 3) and a fourth, the subset rule, in their §2. Chu and Stuckey (2009) place MOSP among "graph path-width and gate matrix layout", citing Linhares and Yanasse (2002), but they do not use pathwidth in the search. Any graph is the MOSP graph of the instance with one customer per vertex and one pattern per edge, so the search decides pathwidth for every graph, and ported to graphs, our corrected algorithm is an exact pathwidth solver, which we release (Sections 3 and 5).

The stakes are these. Two of the published dominance theorems, the *definite move* and the *better move*, are false, and so are their restatements in Chu's thesis (Chu, 2011) and, with a stronger premise, in Fink (2012). The error rarely shows: on every benchmark Chu and Stuckey's algorithm, the search as Chu and Stuckey (2009) publish it, answers correctly, and the only whole instance we found on which our implementation of it reports a wrong optimum, with the better move off, has 34 customers. There the *old move*, a rule that lets a failed sibling prune later

branches, passes a false refutation on (Figure 4.2). And when our corrected algorithm re-proved every benchmark optimum that rested on Chu and Stuckey's algorithm, no value changed (Section 5). Our corrected definite move was also found by Tamaki, as the commitment lemma (Kitsunai et al., 2016, Lemma 1), which says when an exact pathwidth search may commit to one extension of a vertex set and try no other (Section 4.6). We show that it can be tested by one small bipartite matching per candidate. The main result, Theorem 4.8, says informally: *with both rules corrected, the search is sound; whenever it answers that k stacks do not suffice, they do not, and the graph has pathwidth at least k.* Our correction costs at most 2% more search nodes on each benchmark set (Section 5).

Section 4.1 defines the search and shows that it decides pathwidth, in both vocabularies. Section 4.2 gives the algorithm. Sections 4.3 and 4.4 take the rules one at a time, Section 4.5 proves our corrected algorithm sound, and Section 4.6 places each rule in the pathwidth literature.

4.1 The search decides pathwidth

Chu and Stuckey (2009, p. 4) observe that the patterns matter only through the MOSP graph. So fix a graph $G = (C, E)$ whose vertices are the customers, and an integer $k \geq 0$. In this section k bounds the number of open stacks, which is one more than a width: k stacks suffice exactly when the pathwidth is at most $k - 1$ (Theorem 4.2). For a customer c write $N[c]$ for its closed neighbourhood, the stacks that must be open when c's stack closes. For a set $T \subseteq C$ of vertices, its *border* ∂T is the set of vertices outside T with a neighbour in T. For a set T of closed customers write

$O(T) = \bigcup_{c \in T} N[c]$	the stacks opened once T has closed,
$\mathrm{fin}(X) = \{c : N[c] \subseteq X\}$	the customers free to close when X is open,
$\mathrm{cl}(T) = \mathrm{fin}(O(T))$	the *free closure* of T,
$b(T) = \lvert O(T) \setminus T \rvert = \lvert \partial T \rvert$	the stacks open after T has closed,
$o(c, T) = N[c] \setminus O(T)$, $\quad \mathrm{open}(c, T) = \lvert o(c, T) \rvert$	the stacks closing c would newly open.

The open stacks after T has closed are exactly the border of T, and this one identity carries the whole translation to pathwidth.

A *move* at T closes one customer $c \notin T$, at cost $\mathrm{cost}(T, c) = \lvert O(T \cup \{c\}) \setminus T \rvert = b(T \cup \{c\}) + 1$, the stacks open at that moment counting c. It is *playable* when $\mathrm{cost}(T, c) \leq k$. Write $\mathrm{Ext}_k(T)$ ("T extends") when some order of $C \setminus T$ has every step playable. On entering a node the search closes every free customer, so it stands only at *states*, sets with $S = \mathrm{cl}(S)$. The *child* of S by c is $S \cdot c = \mathrm{cl}(S \cup \{c\})$. The search's predicate $\mathrm{Sol}_k(S)$ holds when $S = C$, or when some playable c has $\mathrm{Sol}_k(S \cdot c)$ (Appendix A, A.15). Every node the search visits satisfies the invariant $b(S) \leq k$.

An example. Take the graph of Figure 1.1(d), with $k = 3$. At $T = \{a\}$ we have $O(T) = \{a, b\}$, so one stack, b, is open: $b(T) = 1$ and $\partial T = \{b\}$. No other customer is free, since $N[b]$ contains c and d, so T is a state. Closing b newly opens $o(b, T) = \{c, d\}$ and costs $\mathrm{cost}(T, b) = 3$, the stacks b, c and d; the move is playable. At $\{a, b, c\}$ every neighbour of d is open, so d is free, and the child $\{a, b\} \cdot c$ is $\{a, b, c, d\}$. The closing order a, b, c, e (with d and f closed free) costs $2, 3, 3, 2$, the profile of order A in Figure 1.1(a).

Table 4.1: The customer search is, ingredient by ingredient, an exact pathwidth algorithm: each notion of the search beside its name in the pathwidth literature. The memo and the definite move are described in Section 4.2.

Customer search	Layout
closing order from $\emptyset$	vertex sequence, read forwards
open stacks $b(T)$	border size $\lvert\partial T\rvert$
cost(T, c)	$\lvert\partial(T \cup \{c\})\rvert + 1$
$\mathrm{Sol}_k(\emptyset)$	$\mathrm{vs}(G) \leq k - 1$
free closure $\mathrm{cl}(T)$	full set of T (Suchan and Villanger, 2009, p. 328)
memo	failure table
definite move, corrected	commitment to the child

Lemma 4.1 (free moves never hurt)**.** *If* $b(T) \leq k$*, then* $\mathrm{Ext}_k(T) \iff \mathrm{Sol}_k(\mathrm{cl}(T))$ *(Appendix A, A.16).*

Closing a customer whose neighbourhood is already open opens nothing, never makes a later step dearer, and leaves one fewer stack open. The hypothesis is needed: closing the free customers one by one costs $b(T)$ at each step.

Theorem 4.2 (the search decides pathwidth)**.** *For nonempty* C*,* $\mathrm{Sol}_k(\emptyset) \iff \mathrm{pw}(G) + 1 \leq k$*. On the MOSP graph of an instance* M *that has a 1,* $\mathrm{Sol}_k(\emptyset) \iff Z(M) \leq k$ *(Appendix A, A.17).*

Proof. For a closing order $c_1, \ldots, c_n$ with prefixes S_i, the i-th step costs $1 + \lvert\partial S_i\rvert$. Reverse the order to get a layout. Its first $n - i$ vertices are $C \setminus S_i$, and those with a neighbour among the remaining ones are exactly ∂S_i. So the cost of the closing order is the vertex separation of the reversed layout plus one, and Lemma 4.1 at the root and Theorem 2.2 finish the proof. □

So a refutation, $\neg\mathrm{Sol}_k(\emptyset)$, may only mean $\mathrm{pw}(G) > k - 1$. The rest of the section asks whether the pruning rules preserve that meaning.

In pathwidth language. The exact pathwidth literature writes $N(T)$ for the border and $d(T)$ for its size (Kobayashi et al., 2014, p. 390); we keep ∂T and $b(T)$. A vertex sequence is *w-feasible* when every prefix has border at most w, and $\mathrm{vs}(G)$ is the least w for which some permutation is w-feasible. Table 4.1 translates each notion of the search into this vocabulary. Read it row by row: every ingredient of the customer search is a known ingredient of an exact pathwidth algorithm, and the last two rows are the rules of Section 4.2.

4.2 The algorithm

Algorithm B.1, in Appendix B.7, gives Chu and Stuckey's customer search. It is a depth-first search over states. At each node it closes the free customers, consults a table of states already refuted, keeps the playable candidates, filters them with the dominance rules and recurses on the survivors.

Line 2 is safe by Lemma 4.1, and the cost cut of line 5 is exact, since an unplayable move is not a move of Sol_k. The rest is pruning, and each piece has a one-sentence intuition.

- *The memo* (lines 4 and 12): whether a state can be finished does not depend on how it was reached, so a state refuted once is refused when met again.

- *Old moves* (lines 7–11): a sibling whose subtree failed stays failed further down, as long as closing it first would have kept each later step within the bound.
- *The definite move*: if closing q now leaves no more stacks open than before, close q at once and try nothing else (Section 4.3 shows why this looks safe and is not).
- *The subset rule*: if d newly opens a subset of what r newly opens, then d is at least as good a move as r.
- *The better move*: if q would be a definite move right after r, then closing q before r is at least as good as closing r first.

Each of the three filtering rules discards a move r by citing a *covering*, or *link*, from r to a kept move c with $\mathrm{Sol}_k(S \cdot r) \Rightarrow \mathrm{Sol}_k(S \cdot c)$. The filter runs the definite move, then the subset rule, then the better move, each once. If the definite move fires, it keeps its customer alone.

This loop is the decision procedure of Kobayashi et al. (2014, Algorithm 1, p. 393), which asks whether a vertex set extends to a w-feasible permutation. It has four steps: replace the set by a committed extension, answer from a failure table or at the full vertex set, recurse on each feasible one-vertex extension, and record failures. The customer search is that loop with $w = k - 1$. The free closure and the definite move play the part of the commitment, the memo is the failure table, and the cost cut is the feasibility test. Coudert et al. (2014, §3.2) has the same three parts in a branch and bound. Neither line cites the other in the papers we have read: Chu and Stuckey (2009) predates the commitments and both 2014 papers, and none of the pathwidth papers cites it. This is the isolation measured in Table 3.2, at the level of one algorithm.

4.3 The definite move is false as published, and its correction is a matching test

The definite move is the strongest of the rules, since when it fires it discards every other candidate. Its premise looks like a safe accounting: closing q opens some new stacks and lets some customers close, and if it lets at least as many close as it opens, nothing seems lost by closing it first. Chu and Stuckey (2009, §3.1, p. 6) state:

> **Theorem 1.** Suppose S ++ [q] is playable and close(q, S) $\geq$ open(q, S), then if U′ = S ++ R is a solution, there exists a solution U = S ++ [q] ++ R′.

Here ++ is concatenation of closing sequences, R and R' are closing sequences, a *solution* is a complete closing order with every step playable, and $\mathrm{close}(q, S)$ counts the customers d with $o(d, S) \subseteq o(q, S)$ (p. 4). The rule keeps q alone. We read d as ranging over customers not yet closed, as our implementation does; read literally, closed customers count too, which weakens the premise, and both readings are refuted below. Chu (2011, Thm 6.3.6, p. 142) restates the theorem with the same premise and proof, reworded for k stacks. Write $X = S \cdot q$. The first lemma confirms that the premise says what the intuition says.

Lemma 4.3 (what the premise says)**.** $\mathrm{close}(q, S) = |X \setminus S|$. *Hence the premise holds iff* $b(X) \leq b(S)$*: the child has no more open stacks than the parent (Appendix A, A.24).*

The premise compares only the two ends, S and X. The rule fails because a set between them can be cheaper still. Figure 4.1(a) shows the graph on which it fails, and panel (b) the cheaper set between.

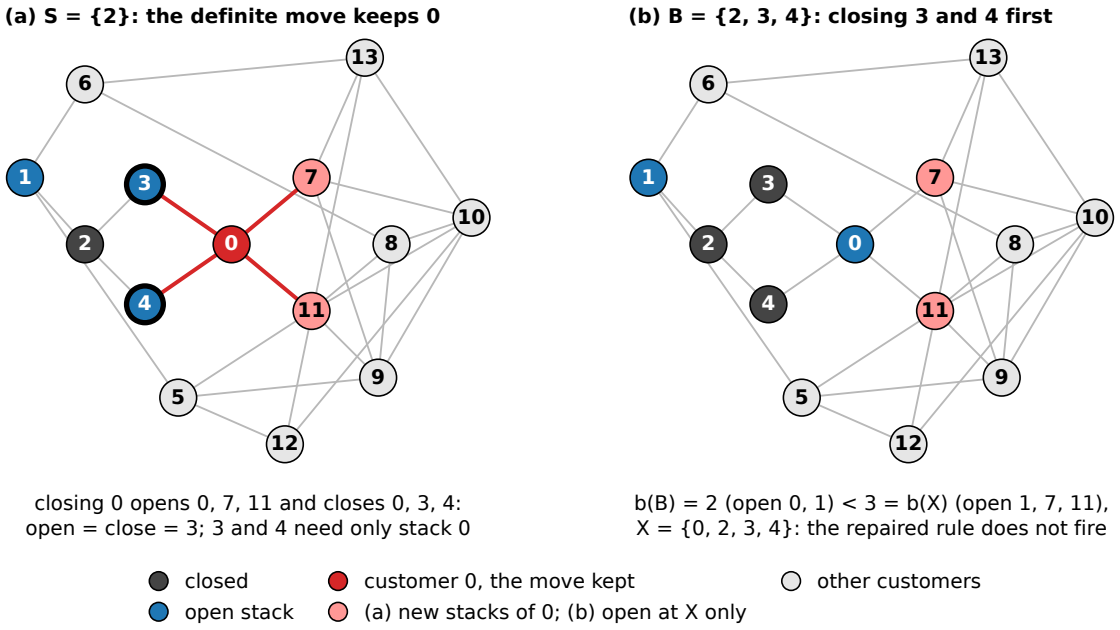


Figure 4.1: Chu and Stuckey's definite move fails because a set between parent and child is cheaper than both. (a) The graph of Counterexample 4.4 at the state where the rule fires. Customer 2 is closed; 1, 3 and 4 are open. Moving 0 opens the new stacks 0, 7 and 11 and closes 0, 3 and 4, so open = close = 3 and the rule keeps 0 alone. But 3 and 4 have the single new stack 0 between them: closing them first gains two stacks for the price of one, which a solution starting with 0 cannot match. (b) The intermediate set $B = \{2,3,4\}$ leaves two stacks open, fewer than the three of the child $X = \{0,2,3,4\}$, so the corrected rule of this section does not fire.

Counterexample 4.4. Let G be the graph of Figure 4.1(a), on customers $0,\ldots,13$ with 26 edges, and let $S = \{2\}$, $k = 6$ and $q = 0$. Then $O(S) = \{1,2,3,4\}$, $o(0,S) = \{0,7,11\}$ and $\text{cost}(S,0) = 6$, so 0 is playable. The customers with $o(d,S) \subseteq \{0,7,11\}$ are 0, 3 and 4, so $\text{close}(0,S) = 3 \geq \text{open}(0,S)$, and 0 comes first in index order, so the rule keeps only 0. A solution exists from S: the order

$$1,3,4,6,12,13,0,5,7,8,9,10,11$$

costs at most 6. None exists from the child $X = \{0,2,3,4\}$: only seven sets are reachable from X by moves of cost at most 6, and none is C (Appendix A, A.25).

The published implication, as a universal statement, is refuted in Lean under both readings of close, and so is the thesis's Theorem 6.3.6 (Appendix A, A.26). Fink (2012, pp. 27–28) restates the rule with a stronger premise, counting only dominated customers of smaller index than q. The graph above refutes it under no labelling, but a third twin of 3 and 4 gives a 15-customer graph that does (Appendix A, A.27). The published proof moves q to the front of a solution and counts, after each prefix, the stacks q opens against the customers it lets close. But customers that the solution has already closed no longer count as closed by q, while the stacks q opens may still be new. Appendix B.4 gives the full analysis, and Appendix B.5 the search for smaller counterexamples.

The correction. We require the comparison at every set between the two ends, not only at S. Call q *hereditarily definite* at S when $b(X) \le b(B)$ for every B with $S \subseteq B \subseteq X$ and $q \notin B$. By Lemma 4.3, the published premise is the case $B = S$. In the counterexample the hereditary condition fails at $B = \{2,3,4\}$ (Figure 4.1(b)), where $b(B) = 2 < 3 = b(X)$ (Appendix A, A.28). The proof that the corrected rule is sound needs one inequality.

Lemma 4.5 (open stacks are submodular)**.** *For all $A, B \subseteq C$, $b(A \cup B) + b(A \cap B) \le b(A) + b(B)$ (Appendix A, A.29).*

Proof. $b(T) = |O(T)| - |T|$, $O(A \cup B) = O(A) \cup O(B)$ and $O(A \cap B) \subseteq O(A) \cap O(B)$; the customers themselves balance exactly. □

Theorem 4.6 (the corrected definite move is sound)**.** *If $q \notin S$ is hereditarily definite at S, then $\mathrm{Ext}_k(S) \Rightarrow \mathrm{Ext}_k(X)$. At a state with the invariant, $\mathrm{Sol}_k(S) \Rightarrow \mathrm{Sol}_k(S \cdot q)$ (Appendix A, A.30).*

Proof. Take an order from S of cost at most k with prefixes T_i, in which q is the j-th step, and follow it from X, skipping customers already in X. A step $i < j$ closes c_i at $T_{i-1} \cup X$. We have $S \subseteq T_i \cap X \subseteq X$ and $q \notin T_i$, so the hypothesis gives $b(X) \le b(T_i \cap X)$, and Lemma 4.5 gives $b(T_i \cup X) \le b(T_i)$: the step costs no more than the original. After step $j-1$ the closed set contains T_j with the same opened set, and the rest of the order applies. The search's form follows by Lemma 4.1. □

The corrected definite move was also found by Tamaki as a lemma: it is the commitment lemma of Kitsunai et al. (2016, Lemma 1, p. 142), stated there for directed pathwidth, read on the symmetric digraph with the committed extension fixed to the child $X = \mathrm{cl}(S \cup \{q\})$. The hereditary premise is their commitment condition for that extension (Lemma 4.10, Section 4.6) and the proof is theirs. The one difference of statement is that their lemma also assumes that the paths to S and to X stay within budget, which holds at every node the search visits. Both statements are transcribed in Lean and each is derived from the other under that assumption; a five-vertex graph shows the assumption is not redundant as a statement (Appendix A, A.32).

The matching test. The corrected rule seems to require checking exponentially many intermediate sets. It does not, and this cheap test is our algorithmic contribution in this section; we did not find it in the papers we hold. In words: the rule is safe exactly when all but one of the new stacks can be paired with a different freed customer who needed it. In the counterexample, closing 0 opens three new stacks and frees 3 and 4, both of whom needed only stack 0: one pairing where two are required.

Theorem 4.7 (the corrected premise is a matching condition)**.** *Let $Y = X \setminus (S \cup \{q\})$, and join each $d \in Y$ to the stacks of $o(d,S) \subseteq o(q,S)$. Then q is hereditarily definite at S iff this bipartite graph has a matching with at least $\mathrm{open}(q,S) - 1$ edges (Appendix A, A.31).*

Proof. The sets of the hereditary condition are $B = S \cup D$, $D \subseteq Y$, and $b(X) \le b(S \cup D)$ iff $|D| - |o(D)| \le \delta$ with $\delta = |Y| + 1 - \mathrm{open}(q,S)$, where $o(D)$ is the union of the $o(d,S)$. By the deficiency form of Hall's theorem the largest matching has size $|Y| - \max_D(|D| - |o(D)|)$, which is at least $|Y| - \delta = \mathrm{open}(q,S) - 1$ exactly when the hereditary condition holds. □

The bipartite graph has at most close -1 customers on one side and open stacks on the other, and the matching can stop at open -1 edges. Every q with $\mathrm{open}(q,S) \le 1$ passes at once, so the definite move of Chu and Stuckey's algorithm is right there (Appendix A, A.33).

4.4 The other rules

Each of the remaining rules is quoted, proved sound in the form used, and shown to need the test or tie-break it carries.

The subset rule. "If $o(c_i, S) \subseteq o(c_j, S)$ and $i < j \ldots$ move j can be removed" (Chu and Stuckey, 2009, §2, p. 5). As implemented, d dominates r when $o(d, S) \subseteq o(r, S)$ and the inclusion is strict or $d < r$. Every dominated playable r is dropped, with a fallback that keeps A if all would go. The covering is sound for every set S: if $o(d, S) \subseteq o(r, S)$ and r is playable, then d is playable and $\mathrm{Sol}_k(S \cdot r) \Rightarrow \mathrm{Sol}_k(S \cdot d)$, because d is free after r and closing d then r reaches the same state at no greater cost. With the index tie-break, domination is a strict order, so an undominated move with a solution survives, and the fallback never fires when a solution exists (Appendix A, A.34). The tie-break is necessary: without it, two customers with equal new stacks dominate each other and both go, which loses a solution on a 7-customer graph (Appendix A, A.35).

The better move. Chu and Stuckey (2009, §3.2, p. 6): "Suppose S ++ [q] and S ++ [r, q] are playable and close(q, S ∪ {r}) ≥ open(q, S ∪ {r}) then if U′ = S ++ [r] ++ R is a solution there exists a solution U = S ++ [q] ++ R′." Its premise is the definite move's premise for q at the child $\mathrm{cl}(S \cup \{r\})$, together with playability of r then q, and its conclusion swaps q and r (Appendix A, A.36). The swap is sound, because both orders end at the same state and neither step overflows. The appeal to the definite move is not.

Counterexample. On the graph of Counterexample 4.4 at the root with $k = 6$, the definite move does not fire, customers 0 and 2 both survive the subset rule, and the premise holds for $r = 2$, $q = 0$, so the rule drops 2 citing 0. But $\{2\}$ has a solution and $\{0\}$ has none (Appendix A, A.37). At that node another kept customer still has a solution, so this is a false link and not a lost node; no node where the better move loses the last solution has been found. Chu (2011, Thm 6.3.8, p. 143) states the better move with a different premise, $\mathrm{close}(q, S) \geq \mathrm{open}(q, S \cup \{r\})$, and proves it through his Theorem 6.3.6. The same graph refutes it at $S = \{2\}$, $r = 3$, $q = 0$ (Appendix A, A.38).

Correction and test. Cite q for r only if q is hereditarily definite at $\mathrm{cl}(S \cup \{r\})$, which is one matching at the child (Theorem 4.7); then the swap argument goes through (Appendix A, A.39).

Two requirements. The close count in the premise must not include the customers that r finishes on its own; counting them makes the rule's own conclusion false on a 12-customer graph. And the rules must run in the order definite, subset, better, each citing only customers still standing. If the better move runs first, the subset rule can cite a customer the better move has already dropped. An 8-customer graph then loses a node through a cycle of links that are each true (Appendix A, A.40). The second is an instance of the general observation of Beck et al. (2025, Prop. 3, Example 6) that mutually dominating transitions must be tie-broken.

The old move. Chu and Stuckey (2009, Thm 3, p. 7) let a child inherit a sibling q whose subtree failed, provided that inserting q before each later step keeps that step playable (line 9 of Algorithm B.1). The rule is sound as implemented: if $\mathrm{cost}(S \cup \{q\}, c) \leq k$, then $\mathrm{Sol}_k(\mathrm{cl}(\{q\} \cup S \cdot c)) \Rightarrow \mathrm{Sol}_k(S \cdot q)$, so a refuted sibling stays refuted along any path whose steps pass the test (Appendix A, A.41). The test is necessary; the smallest graph that shows it is the path on five vertices (Appendix A, A.42).

The memo. Whether a state with $b(S) \leq k$ has a solution depends on the state alone, not on the path that reached it (Appendix A, A.43). One concern is that the memo and the old move might not combine: a refutation obtained with inherited old moves seems to depend on the ancestors, and our Python reference implementation of the search declines to run both for that reason. A node taken on its own, with an old move that is not in fact refuted, can indeed record a solvable state (Appendix A, A.44). Theorem B.2 (Appendix B.9) shows that this never happens in a real run, because every old move is a refutation completed earlier in the same run.

4.5 Our corrected algorithm is sound

What a rule must satisfy. At a state S with old moves Q, the cost cut keeps the playable candidates A and the filter keeps a sublist $L \subseteq A$. The filter is *node-sound* at (S, Q) if, whenever every $q \in Q$ has $\neg\mathrm{Sol}_k(S \cdot q)$, $\mathrm{Sol}_k(S)$ implies $\mathrm{Sol}_k(S \cdot c)$ for some $c \in L$ (Appendix A, A.45). In terms of coverings, a node is sound when two things hold. First, every link a rule cites is a true covering. Second, every chain of links from a discarded candidate ends in L or in Q. The second condition can fail through a cycle even when every link is true, which is why the order of the rules matters. Proposition B.1 shows that the corrected filter is node-sound, and Theorem B.2 that node soundness, the memo and the old move together give sound runs; both are in Appendix B.9. Together they give the main theorem.

Theorem 4.8 (our corrected algorithm is sound)**.** *Run the customer search with free moves, the memo, old moves and the filter definite → subset → better. Let the definite move fire only on a hereditarily definite customer, and the better move cite only under the corrected premise. If the search refutes k from the root, then no closing order costs at most k,* $\mathrm{pw}(G) > k - 1$*, and on the MOSP graph of an instance M that has a 1, $Z(M) > k$ (Appendix A, A.46).*

In words: each rule throws away only moves that a kept move is proved to be at least as good as, and the memo and the old moves remember only failures that really happened earlier in the same search. So when the search says "no", no closing order fits in k stacks.

What is proved and what is tested. The theorem is about the search as specified; that our C and Python implementations run it is checked, not proved. Appendix B.8 lists the checks that link them.

Chu and Stuckey's algorithm. Under it, the filter is not node-sound (Appendix A, A.47). In a check of 570,206 runs with memo and old move, it lost the last solution at a node in 58 runs, all on graphs built around Counterexample 4.4 and all at k equal to the optimum. Every one found a solution through another branch and answered correctly.

A wrong optimum. A targeted search did produce a whole instance on which Chu and Stuckey's algorithm gives a wrong answer. It started from Counterexample 4.4, climbed toward graphs on which Chu and Stuckey's algorithm refutes more and more of the root's solvable branches, and joined two copies of a 17-customer near-miss by one edge. On the resulting 34-customer instance (Figure 4.2), we ran our implementation of Chu and Stuckey's algorithm with its definite move, the subset rule and the old move, and with the better move off. It answers that 6 stacks do not suffice and that 7 do, so it reports an optimum of 7, with a refutation as its proof (Figure 4.2(a)).

Yet the order of Figure 4.2(b) keeps at most 6 stacks open (6 tracks, in the figure's gate matrix reading); our corrected algorithm finds it and refutes 5, so the optimum is 6. This is a theorem, checked in Lean.

Theorem 4.9 (Chu and Stuckey's algorithm reports a wrong optimum)**.** *Let G_{34} be the MOSP graph of the 34-customer instance of Figure 4.2. Run with its definite move as published, the subset rule, the old move and the memo, and with the better move off, Chu and Stuckey's algorithm refutes $k = 6$ on G_{34} and finds a solution at $k = 7$, so it reports an optimum of 7. The optimum is 6, and the pathwidth of G_{34} is 5 (Appendix A, A.48).*

The proof is a computation checked by the Lean kernel itself, with no unverified evaluation. A transcription of Chu and Stuckey's algorithm is proved, for every graph and every k, to produce runs of the search of Section 4.1 with the published filter. On G_{34} it refutes 6 in 526 nodes with the memo and in 839 without it, the counts of our C and Python implementations, and without the memo it makes exactly the calls of our Python implementation, in the same order and at the same states. The optimum 6 is proved in Lean as well, by a sound filter. So the node-level failure of the published filter (Counterexample 4.4) is not only a possibility: on this instance it reaches the root, and the answer is wrong. What the theorem does not prove is that the transcription matches our code line for line; that rests on reading it and on tests, which every node count, the full trace and eight mutated traces passed. A SAT solver on our encoding of MOSP (Appendix D), which shares no code with either search, agrees: it finds an order with 6 open stacks and proves that 5 do not suffice, with a proof in DRAT, the standard proof format of SAT solvers, which the independent checker drat-trim verifies. The mechanism is the old move: the definite move falsely refutes a subtree, and the old move passes that refutation on to the later branches, which then skip the moves their solutions need. Without the old move, or with the better move switched on, Chu and Stuckey's algorithm answers this instance correctly; whether it can also fail in that configuration is open. The failing configuration separates two rules that Chu and Stuckey run as one: "Our implementation of 'better move' subsumes 'definite move' so we will simply consider them as one improvement" (Chu and Stuckey, 2009, p. 6). Run in our implementation, their published configuration, with the better move on, answers this instance correctly. The theorems are false whatever the configuration, and the instance shows that a false dominance theorem can produce a wrong optimum. The instance, the order each search returns and a checker that needs nothing but Python are in the repository (`paper1/data/false_refutation/`, page 45).

4.6 The rules in pathwidth language

The definite move has an exact counterpart in the pathwidth literature, and this subsection makes the correspondence precise. Kitsunai et al. (2016, p. 142), stating Tamaki's notion, call an extension τ of a vertex sequence σ *committable* when $|\partial X'| \geq |\partial V(\tau)|$ for every X' with $V(\sigma) \subseteq X' \subseteq V(\tau)$. Their Lemma 1 says that if σ extends to a feasible permutation, so does τ.

Lemma 4.10. *For $q \notin S$ and $X = \mathrm{cl}(S \cup \{q\})$: the published premise holds iff $b(X) \leq b(S)$, the commitment condition at the single set $X' = S$; and the corrected premise holds iff X is a commitment from S, the condition at every X' between them (Appendix A, A.72).*

So Theorem 4.6 is the commitment lemma at the child, and the published definite move is a commitment whose interior condition was dropped. In Counterexample 4.4 the endpoint test

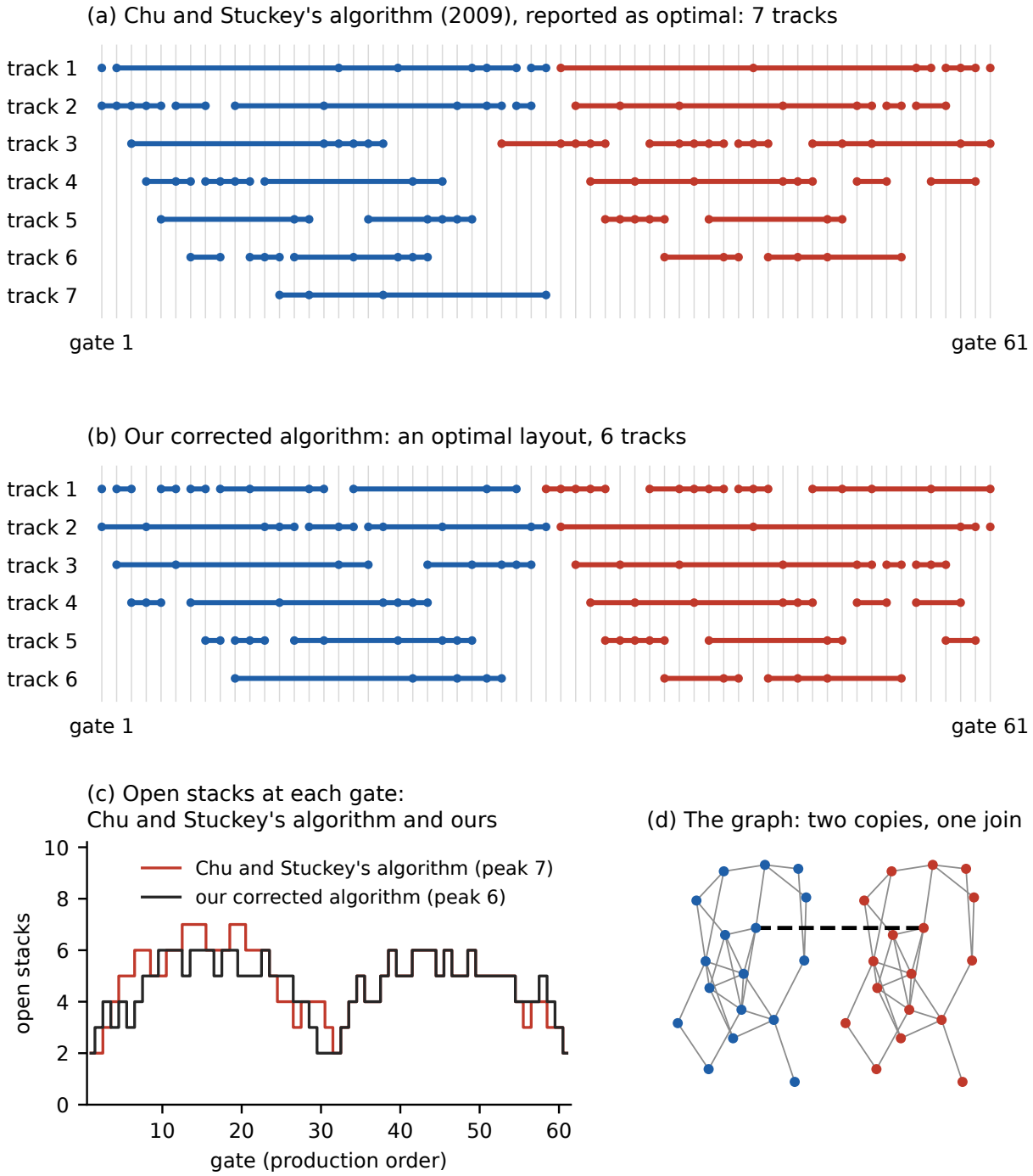


Figure 4.2: Chu and Stuckey's algorithm reports a wrong optimum. A 34-customer instance with 61 patterns, drawn as a gate matrix layout of 34 nets on 61 gates. (a) The layout Chu and Stuckey's algorithm (better move off) returns and certifies as optimal: 7 tracks. (b) A layout with 6 tracks, found by our corrected algorithm, which also proves that 5 do not suffice, as do a SAT solver with a DRAT proof checked by drat-trim and Lean (Theorem 4.9). (c) The open stacks at each gate of the two orders: Chu and Stuckey's algorithm's order reaches 7 in the first copy, where the optimal order never exceeds 6. (d) The instance's graph, two copies of a near-miss joined by one edge.

Table 4.2: Two rules of Chu and Stuckey's algorithm are false; the free move, the memo and the corrected definite move are known pathwidth results; the subset rule, the corrected better move and the old move, like the matching test, are proved here. Each rule of the customer search against the exact-pathwidth literature.

Rule	Published counterpart	Status
definite move, Chu and Stuckey's	endpoint case of a commitment (Lemma 4.10)	false (Cex. 4.4)
better move, Chu and Stuckey's	none found	false
free move	full sets (Suchan and Villanger, 2009); Kitsunai et al. (2016, Prop. 3)	known
definite move, open ≤ 1	depth-one commitments; greedy step; push rule	known
definite move, corrected	commitment lemma (Kitsunai et al., 2016, Lemma 1)	known; matching test (Thm 4.7) not found
memo	subset DP (Bodlaender et al., 2012); prefix and failure tables	known; with the old move, proved here (Thm B.2)
subset rule	none found	proved here
better move, corrected	none found	proved here
old move	none found	proved here

passes with $b(X) = 3 = b(S)$, while the interior set $\{2, 3, 4\}$ has border 2 (Appendix A, A.73). The published premise is not wasted: by Kitsunai et al. (2016, Lemma 10, Cor. 2, pp. 148–149) it guarantees a commitment to the least-border set between S and X, which need not contain q (Appendix A, A.74). In the counterexample that set is $\{2, 3, 4\}$ (Appendix A, A.75). Committing there when the matching fails is a second correction, which gives up no pruning; we have not built it.

A commitment of depth one, S to $S \cup \{q\}$, holds iff $\text{open}(q, S) \leq 1$ (Appendix A, A.76). The greedy step of Coudert et al. (2014, Lemma 3, p. 49) is exactly that condition (Appendix A, A.77), and the push rule of Suchan and Villanger (2009, Rule 1, p. 330) is a case of it. So every published pathwidth rule of this kind lies in the region open ≤ 1, where the published definite move is right; Kobayashi et al. (2014, p. 388, Table 2) report depth-one commitments as "extremely effective". Chu and Stuckey's definite move reaches into open ≥ 2, and that is where it fails. For the cost of the test, Kitsunai et al. (2016, Cor. 2) find the least border between the ends of a given extension by a minimum s–t separator; Theorem 4.7 is that computation for the single target X, where it becomes a bipartite matching.

Table 4.2 sums up, and its rows are grouped by what they say. The published definite and better moves are false. The free move, the memo and the corrected definite move are known pathwidth results. The subset rule, the corrected better move and the old move have no counterpart we found, and are proved here, as is the matching test. The comparison is limited to the pathwidth papers we have read (Suchan and Villanger, 2009; Bodlaender et al., 2012; Coudert et al., 2014, 2016; Kobayashi et al., 2014; Kitsunai et al., 2016); "none found" is not a claim of priority.

Prior reports. We found no earlier report of the error in the 66 works we read in full of the 141 that cite the customer search's papers (Appendix B.6). The pathwidth literature has the correct form of the rule, as a commitment, and none of the pathwidth papers we have read mentions MOSP.

4.7 A lower bound from minors

The search spends nearly all its effort on one question: whether the value below the best known one can be refuted. Its floor, contraction degeneracy, is like every bound built from degrees and cliques a lower bound on treewidth plus one, so where pathwidth exceeds treewidth no such floor can meet the optimum: on the Mycielski graph `myciel6` (95 vertices, pathwidth 38) it is 25. The pathwidth side of the complex offers a bound with no such ceiling.

Theorem 4.11 (minor monotonicity)**.** *Let G be a graph and uv an edge of G, and let G/uv be the graph obtained by merging v into u: u becomes adjacent to every neighbour of u or v other than u and v, and v is removed. Then* $\mathrm{pw}(G/uv) \leq \mathrm{pw}(G)$*. Deleting a vertex or an edge never increases pathwidth either. Hence if a sequence of edge contractions and deletions turns G into H, then* $\mathrm{pw}(H) \leq \mathrm{pw}(G)$ *(Appendix A, A.49).*

Proof. Take a path decomposition $X_1, \ldots, X_r$ of G of width $\mathrm{pw}(G)$, and replace v by u in every bag. No bag grows. Every edge of G/uv either is an edge of G not touching v, which keeps its bag, or joins u to a former neighbour x of v, whose bag held v and x and now holds u and x. The bags holding u were an interval of the path, and so were the bags holding v; since uv is an edge, some bag held both, so the two intervals meet and their union, the bags now holding u, is an interval. So the new bags form a path decomposition of G/uv of width at most $\mathrm{pw}(G)$. Deleting a vertex or an edge keeps the decomposition valid with every bag the same or smaller, and the last claim follows step by step. □

The theorem is the minor-closure of bounded pathwidth, stated for gate matrix layout by Fellows and Langston (1987, Lemma 4.2). It gives a certificate of a lower bound in two parts: a sequence of contractions turning G into a smaller graph H, which anyone can replay, and a refutation of $\mathrm{pw}(H) \leq w$, which then proves $\mathrm{pw}(G) \geq w + 1$; on the MOSP graph of an instance, by Theorem 2.3, it proves that every pattern order opens at least $w + 2$ stacks (Appendix A, A.49).

The minor step. Before our corrected algorithm refutes the value below the best known one, w, on the whole graph, it tries to refute w on minors. It contracts the graph, merging a vertex of least degree into its neighbour of least degree (or, as a second rule, into the neighbour it shares fewest neighbours with), to sizes from half the vertices to 85% of them, smallest first, and on each minor it runs the same search under a node budget that triples from one size to the next. A refutation on any minor ends the descent with the value proved; otherwise the largest width refuted on a minor raises the lower bound, and the search on the whole graph proceeds as before. The cost of a refutation grows about tenfold for every five vertices, so the failed attempts on small minors cost a fraction of the first success, and climbing from the smallest size costs about as much as knowing the right size in advance. The idea of solving a contracted graph exactly for a lower bound is used for treewidth (Bodlaender et al., 2006; Bodlaender and Koster, 2011); for pathwidth it is not capped by treewidth, which is where the rest of the floor stops. Section 5.3 reports what it settles.

5 Computational results and a dataset

Section 4 proved our corrected algorithm sound. This section asks five practical questions about it and answers each with a measurement. What does our correction cost? At most 2% more search nodes on each group of benchmarks we measured. Do the published benchmark optima survive? All 115 that rested on Chu and Stuckey's algorithm (Chu and Stuckey, 2009) were proved again, and none changed. How good a pathwidth solver is it? On every benchmark set with published exact counts it proves as many or more graphs than the published branch and bound of Coudert et al. (2016), and it is faster on each of the fourteen graphs whose times they publish. Can a reader check a refutation without trusting our code? Yes, up to about 75 customers, through a certificate and an independent checker. Finally, what does it make possible? A dataset of 17,714 graphs from twenty benchmark collections. For 16,634 of them the pathwidth is certified, and that gives the exact value of every problem in the exact core.

5.1 Experimental setup

Machine and code. Every run in this section used one machine, an AMD Ryzen 9 9950X (16 cores, 32 hardware threads) with 128 GB of memory under Ubuntu Linux. Both searches, the open-stacks customer search and its port to graphs, are written in C, compiled with gcc 15.2 using the flags `-O3` and `-march=native`, and driven from Python 3.12. A Python implementation of the same rules serves as a reference for cross-checks and emits the certificates of Section 5.5. Our corrected tests are the default in every implementation; the tests of Chu and Stuckey's algorithm remain available as an option. The C implementation handles components of at most 1,024 vertices; 154 classes of the dataset of Section 5.6 are larger.

Runs. The basic job asks one question: is the optimum of this instance at most k? A *worker* is one single-threaded process that answers such questions, one at a time. A *node* is one state visited by the search, the unit in which we report effort. Up to 32 workers ran at once on the machine, so wall-clock times carry load noise. We therefore compare node counts and report time only in *core-hours*, the hours of one core summed over workers (32 workers for one hour make 32 core-hours). A *pair* runs the same question twice in the same worker, once under Chu and Stuckey's algorithm and once under our corrected rules, alternating which goes first. A *descent* solves a whole graph: it lowers k from an upper bound until a refutation proves the last value optimal. Each table states its time or node budget.

Configurations. The open-stacks runs use two configurations of the rules of Section 4, which differ only in the better move. The *base* configuration leaves it off. The configuration *with better move* switches it on for instances whose customers need at most five patterns on average. This is the setting under which our implementation of Chu and Stuckey's algorithm certified the benchmark values it proved.

Instances. The *MOSP corpus* is the 6,376 instance files of four public open-stacks collections, one of which is distributed twice. They are the 2005 Constraint Modelling Challenge (Smith and Gent, 2005), with a second copy of 46 of its files under other names, the random instances of Chu and Stuckey (2009) (30 to 125 customers), those of Faggioli and Bentivoglio (1998), and

Table 5.1: Our correction never changed an answer and costs at most 2% more nodes in total. Each pair runs the same question under Chu and Stuckey's algorithm and under the corrected rules in one worker. Totals are over the pairs that finished under both rules; a pair stopped by its budget says nothing about cost. *Vertices*: customers in the open-stacks rows (the vertices of G_M). *Largest pair ratio*: the largest ratio of corrected to previous nodes over single pairs. Open-stacks pairs refute the optimum minus one, at 600 s per call at 50–100 customers and 2×10^8 nodes at 125. The graph pairs run whole descents on VSPLIB, the colouring and named graphs, and a sample of 500 Rome graphs.

Instances	Vertices	Pairs finished	Nodes, previous	Nodes, corrected	Total ratio	Largest pair ratio
Open stacks: one refutation of the optimum minus one per pair						
MOSP corpus, base	9–40	6,135 of 6,135	228,147	228,154	1.00003	1.009
MOSP corpus, with better move	9–40	6,135 of 6,135	202,962	202,971	1.00004	1.040
Chu & Stuckey, with better move	50–100	118 of 125	1,358,096,671	1,362,205,210	1.00303	1.013
Chu & Stuckey, cap 2×10^8 nodes	125	11 of 23	90,969,508	90,961,419	0.99991	1.000
Graph pathwidth: one whole descent per pair, 120 s each						
VSPLIB, colouring, named, 500 Rome	4–2,916	731 of 880	2,977,849,807	3,038,670,717	1.020	–

the 24 industrial instances of the SCOOP project, real instances from two sawmilling companies (Frinhani et al., 2018). They are the first rows of Table 5.6. A Chu–Stuckey name such as `Random-125-125-2-4_0` gives the number of customers, the number of patterns, the generator's density class (roughly, the number of customers per pattern) and the instance number. The graph benchmarks are those of Table 5.6.

Provenance. Every stored value comes with a witness, an order that attains it, and one of three provenances:

- *certified by search*: a search proved that no order does better, by refuting the value minus one;
- *certified by bound*: a lower bound equals the witness's value, so no search was needed;
- *upper bound only*: the witness is all there is, and optimality is not proved.

All 6,376 corpus values are certified; the last two were certified by the split search of Section 5.2.

5.2 The cost of our correction, and the benchmark values

Our corrected algorithm adds one matching test to the definite move and to the better move (Section 4.3). A candidate that fails the matching is passed over, so the corrected tree can only differ where the matching fails. Table 5.1 shows how rarely that matters.

In every row of Table 5.1 our corrected algorithm and Chu and Stuckey's algorithm gave the same answer, and the total node counts differ by at most 2%. Single open-stacks pairs differ by at most 4%. The time per node also rises, by about 2% in the C implementation, measured on six hard instances at 75 to 125 customers. The published test passes and the matching then fails on only 0.07–0.27% of definite-move candidates, a rate that rises with size.

The benchmark values survive. None of the 115 values changed under our corrected rules. Of the 6,374 values certified before this work, 6,259 do not depend on the customer search at all. Each was proved in one of four ways:

- by an exhaustive computation over all subsets of customers (up to 15 customers);
- by a refutation from a SAT (Boolean satisfiability) solver whose proof an independent checker accepted, in DRAT, the standard proof format for SAT solvers;
- by an earlier SAT search;
- or by a lower bound equal to the value.

The other 115 rested on the customer search of Chu and Stuckey's algorithm. Our corrected algorithm refuted the value minus one on all 115, so all 115 values stand. They are 115 benchmark entries on 113 distinct graphs: SP3 and SP4 are each stored twice.

The seven hardest are 125-customer Chu–Stuckey instances at density classes 2 and 4. They are too large for one worker, so their search was split: the top levels of the tree are expanded into independent subtrees, each refuted on its own worker, and the value is refuted when every subtree is. This is not the textbook split of a depth-first search, and it is worth saying exactly where Chu and Stuckey's algorithm resists it. Its pruning is a property of the path, not of the state: under the old move, their Theorem 3, a candidate q is dropped at a node because an ancestor or an earlier sibling already searched q and found nothing, so the candidate set depends on the order in which the tree was walked and on the answers of subtrees that finished before it. A right-hand subtree handed to a worker before the left-hand one has finished would rest on an answer nobody has. Their nogood memo assumes the opposite: a recorded failure at a closed set is reused by whatever path reaches it, which is sound only if the search below that set does not depend on the path, and under the old move it does; their algorithm runs both at once. The driver therefore reproduces the sequential tree's order and inheritance exactly, so that every task carries the old moves the sequential run would have given it; gives each task an empty, private memo, so no recorded failure crosses paths with different histories; re-splits a task that exceeds its node cap; and claims nothing until every task has answered *unsat*, since an inherited old move is justified only by a sibling that refuted. The split is proved sound (Appendix A, A.78): a larger memo changes nothing a run needs, so a task may start from none, and every such run lifts to a run of our corrected algorithm. The lifting is stated for the corrected rules; with the published definite and better moves there is no sound sequential run to lift onto.

Why our version is parallel. Because a task is a complete run of the search from a state that is fixed when the task is created and never changes afterwards. The state is three things: the closed set, the inherited old moves, and an empty memo. The first two come from the driver's expansion of the top of the tree, which is the sequential search's own expansion, so they are exactly what the sequential run would have carried at that node, and nothing a sibling does later can alter them. The memo is private, so nothing a sibling records can reach the task either. A task therefore reads nothing and writes nothing that another task reads: it takes its state, searches, and answers *unsat*, *sat* or *over cap*. Tasks can run in any order, on any number of workers, on any machines, and a ledger of finished tasks is the whole state of the run, which is why a crash loses only the tasks in flight. The one global step is the last: a refutation is the conjunction of every task's *unsat*, and that conjunction is what the split theorem (Appendix A, A.78) lifts to a sequential run. Chu and Stuckey's algorithm cannot be run this way as published, because its tasks would not be self-contained: their candidate sets would depend on siblings still running, and their shared nogoods would cross paths with different histories. Making the state explicit,

and making the rules true, is what makes the search parallel. Its price is measured and reported beside each count: the memo hits the tasks cannot share, which make the split visit more nodes than one worker would, and the nodes of tasks that hit their cap and were split again, which we count as waste. Chu and Stuckey's algorithm has no parallel form in its authors' papers. All seven were refuted at 18 to 369 core-hours each, 1,112.6 in total (Table B.1, Appendix B.10).

Measured on graphs, and what the memo was hiding. The split runs on graphs through the one-pattern-per-edge instance (Section 5.3), and Table 5.2 measures it on four graphs of proved pathwidth whose refutation takes minutes to an hour on one core: two Rome graphs, a PACE Gaifman graph and the Higman–Sims graph. Each is refuted three ways by the same engine under the same rules: on one core with the memo at its default limit of four million states, on one core with the limit raised to 64 million, and split at the root into about ninety tasks on sixteen workers, each task with a default memo of its own. The decomposition does not depend on the number of workers, so the split's node count is the same on one worker as on sixteen (checked on the first graph at 1, 2, 4, 8 and 16: identical nodes, wall time 58, 30, 15, 13 and 14 seconds), and only the scheduling changes.

The table says two things, and the second qualifies the first. On the dense Higman–Sims graph the split is parallelism and nothing else: the memo size does not change the node count, and changes the time only through the larger table's cache misses, 519 seconds with the default against 716 with the large one, while sixteen workers finish in 64 seconds, with ten of them busy on average. On the PACE graph the gain is a factor of 2.5, bounded not by the workers but by one root task that ran to its three-billion-node cap before it was split; the granularity of the split, not the core count, sets the speed-up there. On the two sparse Rome graphs, however, most of what the split gains is memory, not parallelism. With the default memo the sequential search revisits states it has already refuted: on `grafo8793.100` it visits $8.4 \cdot 10^8$ nodes, on $4.9 \cdot 10^7$ distinct states, and raising the memo to 64 million states, which costs 1.1 GB, brings the one-core time from 254 to 33 seconds, against the split's 14; on `grafo11458.100` the default memo costs one core eleven times the nodes and the time, and one core with the large memo, at 60 seconds, is faster than sixteen workers with default memos, at 69. The memo table is open addressing at 16 bytes a slot, capped at 2^{26} slots in the C, so 64 million is also its ceiling; the knee of the curve lies between 16 and 64 million on these graphs. Every hard count in this paper was run under such a limit: the MOSP refutations of Table B.1 with the default of four million states, and the graph program of Section 5.3 with a table capped at 512 MB, which at its load rule holds 23.5 million states on graphs of up to 128 vertices and fewer above, whatever the limit says. The recorded node counts are therefore upper bounds on what the same search does with an adequate memo; the widths and proofs are unaffected, since the memo changes the cost of a run and never its answer (Appendix A, A.78). On this measurement the default is now 64 million states in every entry point of the solver and the graph program's table cap is 2 GB. The shipped program then refutes `grafo8793.100` in 40 seconds and 53 million nodes with its default rules, where its recorded run took 2,678 seconds and $8.9 \cdot 10^9$ nodes, and the better move, on or off, makes no difference to it there; the 40 percent on dense graphs is the known price. What the table establishes about the split is therefore the honest form of the claim: it divides the wall-clock time of a refutation by the number of workers up to the granularity of its tasks, on top of, not instead of, an adequately sized memo, and on graphs where the memo is irrelevant the two coincide.

Table 5.2: The root split on graphs. The refutation of width minus one by one core with the engine's default memo limit (four million states), by one core with the limit raised to 64 million states (1.1 GB), and by the split on sixteen workers with a default memo per task: nodes and wall seconds, the split's summed task CPU seconds in brackets, and the two one-core times divided by the split's. The split's node count is identical at every worker count. Same engine, same rules, nice 19, other work on the machine.

graph	$\|V\|$	pw	one core, memo $4 \cdot 10^6$		one core, memo $6.4 \cdot 10^7$		16 workers		speed-up
			nodes	s	nodes	s	nodes	s (CPU)	default / large
`grafo8793.100` (Rome)	100	10	$8.42 \cdot 10^8$	254	$4.93 \cdot 10^7$	33	$1.19 \cdot 10^8$	14 (66)	19 / 2.4
`grafo11458.100` (Rome)	100	10	$2.00 \cdot 10^9$	652	$7.91 \cdot 10^7$	60	$3.72 \cdot 10^8$	69 (178)	9 / 0.9
`atco_enc2` Gaifman (PACE)	104	35	$1.09 \cdot 10^{10}$	4,906	$9.68 \cdot 10^9$	4,758	$9.73 \cdot 10^9$	1,902 (5,087)	3 / 2.5
Higman–Sims	100	74	$8.03 \cdot 10^8$	519	$8.03 \cdot 10^8$	716	$8.03 \cdot 10^8$	64 (658)	8 / 11.2

Two values never certified before. The corpus also held two values that no method had certified: 25 for `Random-125-125-2-2_0` and 21 for `Random-125-125-2-3_0`. The same split refutes the value minus one on each. It refuted 20 for `Random-125-125-2-3_0` after 9,665 subtrees, 4.25×10^{12} nodes and 1,128 core-hours, which certifies 21 as its optimum. It refuted 24 for `Random-125-125-2-2_0`, after 17,001 subtrees, 7.38×10^{12} nodes and 2,231 core-hours (Table B.1), which certifies 25; it is the largest refutation in this paper. With it, every value of the MOSP corpus is certified. The minor step of Section 4.7 reaches two of these values in seconds: a 69-customer minor of `Random-125-125-2-3_0` refutes 20 in 666,153 nodes, under a second of search, against 1,128 core-hours on the full instance, and an 87-customer minor of `Random-125-125-2-1_0` refutes 23 in 30 s with a larger budget, against 366.5 core-hours (Table B.1). It does not reach `Random-125-125-2-2_0`: every minor of up to 95 customers that we tried admits 24. The minor bound also equals the optimum on `Random-125-125-6-1_0` (81) and SP4 (53), which the search refutes on the full instance within a minute.

5.3 Our corrected algorithm as a pathwidth solver

Ported to graphs, our corrected algorithm is an exact pathwidth solver. Table 5.3 gives what it proves on the 11,915 benchmark graphs it was run on, counted as files, beside the run of Chu and Stuckey's algorithm under the same caps and the published counts of Coudert et al. (2016). Where both runs prove a graph, the widths agree on every one. The 24 Rome graphs proved only by the corrected run, and the 13 proved only by the previous one, sit near the 600 s cap. The comparison with Coudert et al. (2016) in this table is with published numbers obtained on other hardware; the head-to-head run on one machine is Section 5.4. On the fourteen graph-colouring benchmarks of TreewidthLIB whose times Coudert et al. (2016, Table 4) report, our search proves the same pathwidth and is faster on every one: by 150 to 1,800 times on `david`, `miles750` and `mulsol.i.5`, by 70 to 80 times on `miles1000` and `zeroin.i.1`, and by 1.4 to 10 times on the other nine, where a hardware difference could account for the margin. It also proves `queen11_11` (pathwidth 87) in 37 s and `queen12_12` (103) in 391 s, beyond their table, and `fpsol2.i.1` (496 vertices, pathwidth 67) in 5.7 s, against 323 s for Kobayashi et al. (2014). Without the minor step, one graph goes the other way: the Mycielski graph M_7 (`myciel6`, 95 vertices, pathwidth 38), which their tuned branch and bound proves in about ten minutes, while the full refutation of 37 by our search does not finish in hours, its floor being 25. Mycielski graphs are built to have no triangles and yet to need many colours, so their largest cliques are single edges and their degrees are uneven: the degree and clique bounds that make up our floor see

almost nothing. And their treewidth, 35 here, lies strictly below their pathwidth, so no bound through treewidth can reach 38 either.

With the minor step. The minor step of Section 4.7 changes this. A 58-vertex minor of `myciel6` already has pathwidth 38, and the step proves $\mathrm{pw} = 38$ in 0.2 s. On `myciel7` (191 vertices), which Coudert et al. (2016) leave open with $\mathrm{pw} \leq 72$, a 110-vertex minor proves $\mathrm{pw} \geq 71$ in seconds, and a 132-vertex minor refutes 71, in 5.2×10^9 nodes and 1.6 hours, so $\mathrm{pw} = 72$; the best published lower bound we found is the treewidth bound 52 of Bodlaender et al. (2006). The pathwidths of the Mycielski graphs $M_2, \ldots, M_8$ are therefore $1, 2, 5, 10, 20, 38, 72$, a sequence in no entry of the OEIS, and no closed form we tried survives a hold-out test (the Fibonacci convolution that fits the first six terms predicts 71). For every graph G its Mycielskian satisfies $\mathrm{pw}(G) + 1 \leq \mathrm{pw}(\mu(G)) \leq 2\,\mathrm{pw}(G) + 1$ (Appendix A, A.52), the upper bound by interleaving each vertex with its shadow, so $73 \leq \mathrm{pw}(M_9) \leq 145$; a layout found by our search lowers this to $\mathrm{pw}(M_9) \leq 138$. Of the 340 Rome graphs left open by the 600 s run of Table 5.3, the step proves 271 within 60 s each (median 0.5 s), and 292 with a larger budget; we refuted the other 48 without a time limit, the longest in 25.4 hours, so the pathwidth of every one of the 11,534 Rome graphs is now proved; the Harwell–Boeing count rises from 39 to 43, and the colouring benchmarks gain `anna` and `myciel6`. The VSPLIB grids need no search: each is an $n \times n$ grid, whose pathwidth is n by Ellis and Warren (2008, Theorem 4.1), and a row-by-row layout of width n attains it, so all 50 are settled where search alone proved 9. We machine-checked the theorem (Appendix A, A.50), by a direct argument on vertex separation: at the first prefix of any layout that completes a row or a column, the boundary meets every column or every row. That each VSPLIB grid is an $n \times n$ grid is checked by an isomorphism test, not in Lean. On graphs proved both with and without the step, it lowers the total node count by 55 to 99% on Rome, the VSPLIB grids, Harwell–Boeing and the colouring graphs, adds 1% on the named graphs and 20% on the VSPLIB trees (about 1.5 s a tree), and costs 4% of the time on a sample of 120 MOSP corpus instances. The minor bounds for `myciel6`, `myciel7` (the bound 71) and the four MOSP instances below were also refuted by a separate exact search that uses no dominance rule and writes out every refuted set, which an independent checker verifies; the refutation of 71 on the 132-vertex minor of `myciel7` rests on our corrected algorithm alone, its contraction sequence replayed and checked. Their branch and bound is openly available in SageMath, and the head-to-head run on one machine is Section 5.4.

What a proof costs. Figure 5.1 plots the search nodes of every width our search has proved, on 8,510 graphs, against the number of vertices n and against n times the width w. On the Rome graphs, sparse graph-drawing instances, the cost follows one law,

$$\log_{10}(\text{nodes}) \approx 1.75 + 0.00705\, nw,$$

with $r^2 = 0.96$ and a typical graph within a factor of 2.3 of it: each further 100 in nw multiplies the work about fivefold. Size alone explains less ($r^2 = 0.85$), and so does width alone (0.89); the states of the search are vertex sets whose boundary has at most w vertices, and their number grows with both. The 48 hardest Rome graphs, which we refuted without a time limit (the longest in 25.4 hours), lie above the line, so the law ranks graphs by hardness but understates the extreme tail. Graphs that the 600 s runs did not finish are absent, so the top of the curve is a lower envelope. The structured sets do not follow the Rome law: Harwell–Boeing graphs of up

Table 5.3: Our corrected algorithm as a pathwidth solver. Graphs whose pathwidth it proves, under our corrected rules and under Chu and Stuckey's algorithm with the same caps (Rome: 10, 120 and 600 s in stages), without the minor step of Section 4.7, and the published counts of Coudert et al. (2016) under a 10-minute cap. *Width differs*: graphs proved by both runs with different widths. [a] All 50 VSPLIB grids: each is an $n \times n$ grid, checked by isomorphism, so its pathwidth is n by Ellis and Warren (2008, Theorem 4.1); the 41 that search did not prove carry a witness layout of width n.

Set	Vertices	Graphs	Proved, corrected	Proved, previous	Width differs	Coudert et al. (2016)
Rome	10–110	11,534	11,194	11,183	0	95.6% (ours 97.1%)
VSPLIB trees	22–202	50	50	50	0	30
VSPLIB Harwell–Boeing	24–960	73	39	39	0	26
VSPLIB grids	25–2,916	50	9 (50[a])	9	0	9 (sides 5–13)
TreewidthLIB colouring	5–864	58	31	31	0	14 of 14 agree; ours faster on all 14
freetdi named	4–3,282	150	125	125	0	–
All		11,915	11,448	11,437	0	

to 960 vertices, the colouring and named graphs and the dataset's open classes are proved with orders of magnitude fewer nodes than it predicts, mostly because a lower bound, often from a minor, meets the width early. Time tracks nodes closely ($r^2 = 0.98$), which is why we report nodes.

5.4 Head to head with SageMath

The comparisons above are with published numbers. The branch and bound of Coudert et al. (2016) is openly available as SageMath's exact vertex separation (`vertex_separation` with `algorithm="BAB"`, SageMath 10.8), so the two solvers can be run on the same graphs on the same machine. The machine is the one of Section 5.1, and it was not idle: other work of ours ran beside the comparison throughout, with a one-minute load average between 8 and 84 at the moments the pairs were launched (median 18, on 16 physical cores). The protocol takes that into account rather than waiting for quiet. For every graph the two solvers start at the same moment, each pinned to its own physical core, both at the lowest scheduling priority, and each with the same cap of 600 seconds of *CPU time*, enforced by the operating system rather than by the clock, so that whatever the rest of the machine does it does to both, and what is recorded is the processor time each solver consumed, not the time of day. Six such pairs ran at once. SageMath's solver is exact when it returns, so a returned value counts as a proof; ours counts a proof when it reports a refutation, a lower bound meeting the width, or a minor, and reports a verified upper bound otherwise. The graphs are the sets of Table 5.3, with the Rome graphs represented by a random sample of 500: 881 graphs in all. The program is `pathwidth_solver/bench/sage_paired.py`; every row of `paper1/data/sage_paired_600s.csv` holds both widths, both statuses, both CPU and wall times, our node count and the load at launch, and Table 5.4 is generated from it.

Three things are in the table. First, there is no graph that SageMath proves and ours does not: the set of graphs it proves within the cap, 550 of the 881, is contained in the 768 ours proves, and on every one of the 550 the two widths are equal. The 113 graphs that neither proves are the large grids, half of the Harwell–Boeing matrices, the dense colouring graphs and 18 of the named graphs; on each, ours ends at the cap holding a verified layout and SageMath ends at the cap holding nothing. Second, where both prove the width, ours is faster on every graph on which SageMath needs at least half a second, by a median factor of 34 over the 83 graphs where

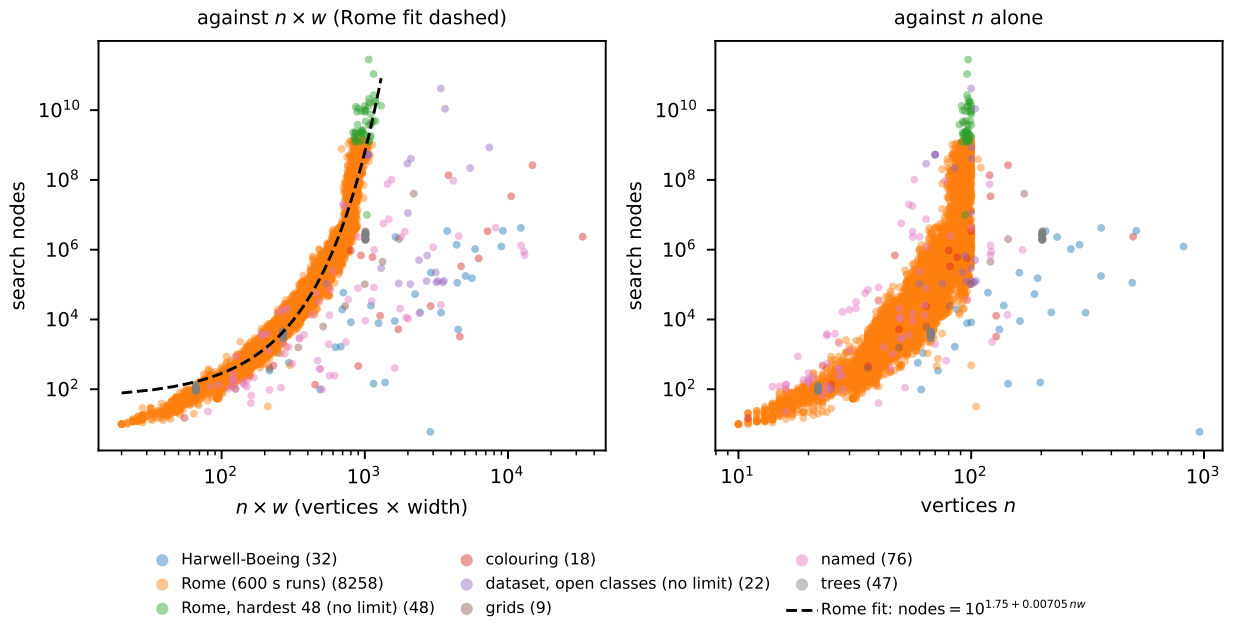


Figure 5.1: The cost of proving a pathwidth. Search nodes for every width our search has proved (8,510 graphs, both axes logarithmic), against n times the width (left) and against n alone (right). The dashed line is the fit on the Rome graphs, nodes $= 10^{1.75+0.00705\,nw}$. The structured benchmark sets fall far below it, and size alone gives no single law.

it needs at least a second and of 226 over the 29 where it needs five, the largest ratio being above 10^5; over the 550 graphs both prove, SageMath spends 4,182 CPU seconds and ours 20. Third, the margin is in the proofs, not only in the time: on the 500 Rome graphs ours proves all and SageMath 386; on the 50 VSPLIB trees, all against 15, with ours taking a median of 2.2 seconds on the 35 trees SageMath does not finish; on the colouring graphs 33 against 10. The minor step of Section 4.7 is a large part of this: it is the proof on 179 of our 768, including the 153 Rome graphs and the 13 Harwell–Boeing matrices on which the search alone would have run to the cap, and on `myciel6`, which Coudert et al. (2016) report proving in about ten minutes on their machine, SageMath did not finish within the cap here while the minor step proved pw $= 38$ in 0.2 seconds. What the comparison does not measure is our parallel refutation: both solvers ran on one core, which is the only fair unit, and the split is a separate claim, measured in Table 5.2 of Section 5.2.

Because the machine was loaded, the CPU-time cap is what makes the comparison fair, but it cannot make it exact: a process that is descheduled loses its cache, and the same graph can cost either solver more or fewer processor seconds on a busy machine than on a quiet one. To measure the variance this adds, a 50-graph sample drawn from the same sets (28 Rome, 7 grids, 5 named, 4 trees, 4 Harwell–Boeing, 2 colouring) was run a second time under the same protocol, at loads between 12 and 56 at launch. Every outcome was the same: each solver proved exactly the graphs it had proved the first time, with the same widths, and our node counts were identical on all 39 graphs we proved, since the search is deterministic and nodes do not see the load. The CPU times moved by factors of 0.6 to 1.6 for SageMath (median 0.84 on the 22 graphs where it ran half a second or more) and of 0.7 to 1.1 for ours on the three such graphs, which is the size

Table 5.4: Head to head with SageMath's branch and bound (Coudert et al., 2016) on one machine, both solvers started together on the same graph, each on its own physical core with a cap of 600 CPU seconds. *Ours* and *SageMath*: graphs whose pathwidth each proved within the cap; *both*, *ours only*, *SageMath only*, *neither*: the four cells of that pair of counts. The widths agree on every graph both proved. *Ratio*: the median, over the graphs both proved on which SageMath needed at least one second, of SageMath's CPU time divided by ours; on the trees SageMath proved, it needed less than a second.

set	graphs	ours	SageMath	both	ours only	SageMath only	neither	ratio
Rome (sample)	500	500	386	386	114	0	0	32
VSPLIB trees	50	50	15	15	35	0	0	–
Harwell–Boeing	73	44	21	21	23	0	29	107
VSPLIB grids	50	9	6	6	3	0	41	534
Colouring (TreewidthLIB)	58	33	10	10	23	0	25	5
Named graphs	150	132	112	112	20	0	18	25
all	881	768	550	550	218	0	113	34

of the noise in the ratio column of Table 5.4, against ratios of 34 and more. The rows of both runs are published with the data (Section 6).

5.5 A checkable proof of a refutation

A refutation by the search is a claim about a tree of up to 10^{12} nodes, and Theorem 4.8 proves the rules sound, not the code that runs them. So the search emits a *certificate*: the tree it explored, every pruning step with its rule and its witness, and for each definite and better step the matching of Theorem 4.7. An independent checker, which shares no code with the search, reads the instance matrix and the certificate. At every node it checks that

- the premise of every pruning step holds, recomputed from the matrix;
- every matching is valid: distinct customers, distinct stacks, each stack newly opened by its customer, and at least open $-$ 1 edges;
- every candidate within the bound is either a child or covered by a pruning step, and every chain of covers ends at a child or an old move, with no cycle;

and it replays the tree from the root. Each accepted step is a Lean theorem. That an acyclic covering by such steps is sound is a short paper argument, not yet a formal proof.

Table 5.5 gives the results. Every refutation emitted within its budget verifies, 6,275 of 6,286, and none is rejected. All 35.5 MB of certificates check in 147 s. The base configuration gives the same picture: 6,276 of 6,286 verify and none is rejected. The checker also catches the error of Section 4.3. On the graph of Counterexample 4.4, rooted at $\{2\}$, Chu and Stuckey's algorithm emits a complete tree that claims there is no solution. A checker of the published premises accepts it. Ours rejects it, even with a largest matching attached, because the matching has one edge where two are needed. At 125 customers the trees are too large to ship as certificates.

Searched lower bounds, with certificates. The witness half of every certified value is found by a tuned search over orderings. The lower bounds that start the descent, the clique, contraction degeneracy, Harper's vertex-isoperimetric bound at a cap (Harper, 1966) and the greedy minor

Table 5.5: Every certificate the search emitted verifies, none is rejected, and checking takes milliseconds. Certificates for our corrected rules, configuration with better move, refuting the optimum minus one on the certified MOSP corpus at 9–75 customers, with 60 s to emit each. *Not emitted*: the emitter did not finish in time. Size is the compressed certificate; check time is the independent checker's.

Customers	Instances	Verified	Rejected	Not emitted	Median size (bytes, gzip)	Median check (ms)
9–10	1,614	1,614	0	0	332	0.04
11–20	2,508	2,508	0	0	356	0.08
21–30	1,816	1,816	0	0	414	0.24
31–40	197	197	0	0	650	1.27
41–50	120	120	0	0	12,045	33.24
51–60	1	1	0	0	126,508	278.86
61–75	30	19	0	11	26,516	143.70

step of Section 4.7, were only ever computed, one formula or one greedy rule each. We ran a search over each family of obstruction on instances whose optimum is known, so that every answer is a measurement, and report the family's duality gap, the optimum minus the best value it certifies. Two kinds of family must be told apart. Harper's bound is universal, $\mathrm{pw} \geq \min_{|A|=t} |N[A]| - t$ for every t, so a set found by search proves only that the minimum is at most its value: it gives a *ceiling* on what the family could ever certify and no bound at all, and the floor needs the minimum proved, by branch and bound to a cap or by a SAT refutation with a DRAT proof. Cliques, contraction minors and separator branches are maximisations, and a found object is a certificate. Each certificate has a format and an independent checker sharing no code with the search: it regenerates the CNF of an expansion level and runs `drat-trim`; replays a contraction sequence with its own code, rebuilds the minor, re-runs our search on it and, to 80 vertices, a rule-free enumeration of refuted prefixes; checks a separator's branches by its own breadth-first search. That a verified certificate implies the bound is proved in Lean, and `decide` closes a 30-customer record instance from its own matrix (Appendix A, A.92).

On the 338 instances of the MOSP corpus where the solver's own bound misses the optimum by two or more (20 to 134 customers), the best floor on record was tight on 81. Searched contraction minors, each refuted on the minor by our corrected algorithm under a cap of 2 to 10 s per call, are tight on 327: at 20 to 50 customers on 251 of 251, beating the greedy ladder of Section 4.7 on 29, always by one; above 60 customers on 76 of 87. They reach 93 on `Random-125-125-8-2_0`, its optimum, on a 118-customer minor, and shrink the minor that certifies `Random-125-125-2-3_0` from about 70 customers to 51, refuting 20 in 145,355 nodes; none of the minors of up to 110 customers we tried certifies `Random-125-125-2-2_0`. Harper's ceiling is below the optimum on 320 of the 338, so no computation of that bound could close them; its exact floor, by SAT with a checked DRAT proof, settles every bracket at 30 to 60 customers and almost none at 75 and beyond, and closes 12. Separator branches close none. The 11 instances left are dense Chu and Stuckey instances at 100 to 125 customers with the best floor one to four stacks below the optimum. Of the dataset's 24 open classes (Section 5.6), two are closed by certificates filed with the code (the dataset's records are unchanged): the complete ternary tree of height 5 by the branch lemma, at the pathwidth that Ellis et al. (1994) compute for trees, and the PACE instance `NY_13` (283 vertices, width 10) by an 85-vertex contraction minor of pathwidth 10, refuted in 0.07 s where 236.7 core-hours on the whole graph had found nothing. Harper's ceiling is below the recorded width on all 24. Every certificate verifies: 158 expansion levels, 441 minors,

156 separators and 47 on the open classes, at about 35 core-hours in all. We found no prior report of a searched contraction-minor or searched vertex-isoperimetric lower bound on benchmark instances. The searches, the checker and the certificates are in the repository (Section 6).

5.6 A dataset with certified pathwidth

Section 2 showed that one certified pathwidth w answers every problem in the exact core. Pathwidth, vertex separation and the vertex separator game take the value w, and MOSP, gate matrix layout (which multiple folding equals), one-dimensional logic, interval thickness, narrowness, node search, progressive pebbling, the vertex search of Bienstock et al. (1991) and the maximum wavefront take $w + 1$: twelve problems in all, counting multiple folding with gate matrix layout. We therefore built one dataset from every benchmark collection of those problems that we could obtain, including the MOSP corpus.

The unit of the dataset is the *isomorphism class*: the set of instances whose graphs are the same up to renaming the vertices. For a matrix instance the graph is its MOSP graph. Two instances in one class have the same value under every problem, so the dataset holds one record per class, with its members and the renaming that maps each onto the record's graph. Classes were found with nauty, a standard tool that computes a canonical form for each graph. Each record carries the graph, the width w, its provenance, and a witness layout of exactly that width. A record certified by bound also carries a *minor certificate*: a list of vertex deletions and edge contractions after which every vertex has degree at least w. Any checker can replay it, and it proves the pathwidth is at least w. A record certified on a minor carries, when it is stored, the sequence of edge contractions that gives the minor on which our search refuted width $w - 1$, which proves $\mathrm{pw} \geq w$ by Theorem 4.11. A record certified by theorem names the theorem and its Lean proof; so far these are the VSPLIB grids, whose pathwidth is given by the grid theorem (Appendix A, A.50).

Table 5.6 gives the counts. The collections are the MOSP collections (Smith and Gent, 2005; Faggioli and Bentivoglio, 1998; Chu and Stuckey, 2009; de Carvalho and Soma, 2015; Frinhani et al., 2018), eleven VLSI gate matrix circuits (de Oliveira and Lorena, 2002), the VSPLIB sets (Duarte et al., 2012) and the Small graphs (Martí et al., 2008), the Rome graphs (Di Battista et al., 1997), and treewidth collections used as pathwidth benchmarks (van den Broek and Bodlaender, n.d.; Larisch and Salfelder, n.d.; Dell et al., 2017, 2018). Of the 17,714 classes, 16,634 carry a certified pathwidth with a witness layout and 337 a witnessed upper bound, and 743 have no witnessed value yet; one of those, a Rome class certified by refutation, has a certified pathwidth but no layout on file. The 610 large instances of Frinhani et al. (2018), with 400 to 1,000 customers, are included so that the dataset covers every published MOSP collection we have. None of them has a value yet, because an exact search at that size is out of reach.

An independent checker, sharing no code with the solvers, verifies the dataset. It re-reads all 20,984 member files with its own readers, maps each onto its record's graph by its renaming, recomputes the width of every witness layout, replays every minor certificate and every stored contraction sequence (424 of the 429 records certified on a minor), and recognises each grid certified by theorem from its edges alone. It reports no failure on the 16,971 records, the 16,634 certified classes and the 337 with an upper bound. A value certified by search cannot be checked from its record, and neither can the refutation on a minor; both rest on our corrected algorithm, and for instances within reach of the emitter the certificate of Section 5.5 is how a third party can check it.

Table 5.6: Twenty collections, 21,754 instance files, reduce to 17,714 isomorphism classes, and 16,634 of them have a certified pathwidth with a witness layout. Largely open: PACE, the VSPLIB Harwell–Boeing graphs, and the MOSP sets of de Carvalho and Soma and of Frinhani et al. *Classes*: isomorphism classes within the collection (78 graphs above 5,000 vertices are compared by file content only). *Counted here*: classes whose first member, in the order of the table, is in this collection, so the column sums to the dataset's class count. The next six columns split those classes by provenance: certified by search on the graph, by search on a contracted minor (Theorem 4.11), by a lower bound, or by a theorem; or an upper bound only. *No witnessed value*: no layout is stored yet; this includes one Rome class whose pathwidth is certified but which has no stored layout of that width. [a]The same 46 files distributed a second time under other names; every class is counted under its first copy.

				Certified					
Collection	Files	Classes	Counted here	by search	on a minor	by bound	by theorem	Upper bound only	No witnessed value
MOSP: Challenge 2005	5,806	3,169	3,169	3,168	0	1	0	0	0
MOSP: Challenge 2005 (second copy)[a]	46	46	0	0	0	0	0	0	0
MOSP: Chu & Stuckey	200	200	200	200	0	0	0	0	0
MOSP: Faggioli & Bentivoglio	300	287	274	274	0	0	0	0	0
MOSP: SCOOP	24	24	24	24	0	0	0	0	0
VLSI gate matrix circuits	11	11	11	5	0	5	0	1	0
VSPLIB trees	50	28	28	28	0	0	0	0	0
VSPLIB grids	50	50	50	9	0	0	41	0	0
VSPLIB Harwell–Boeing	73	72	72	32	5	7	0	28	0
Small (Marti et al. 2008)	84	84	84	32	0	52	0	0	0
Rome graphs	11,534	11,199	11,199	8,167	286	2,746	0	0	0
TreewidthLIB colouring	58	58	58	18	2	13	0	25	0
freetdi named graphs	150	146	143	103	1	22	0	16	1
freetdi control-flow graphs	1,817	1,070	1,064	398	2	661	0	2	1
MOSP: Frinhani et al. 2018, large	610	610	610	0	0	0	0	0	610
PACE 2016	291	288	112	38	14	18	0	36	6
PACE 2017 exact	200	199	193	61	66	7	0	49	10
PACE 2017 heuristic	200	197	173	11	5	3	0	40	114
PACE 2017 bonus	100	100	100	25	58	0	0	17	0
MOSP: Carvalho & Soma 2015	150	150	150	38	0	0	0	112	0
All	21,754		17,714	12,631	439	3,535	41	326	742

Agreement with published values. All 84 Small graphs agree with the pathwidths of Mallach (2018, Tables 4–5). Ten of the eleven VLSI circuits are certified at their published best-known track counts (de Oliveira and Lorena, 2002, Table I); the eleventh, W4 with 202 nets, is open at 28 against 27.

A caution about MOSP files. A MOSP file can be read with rows as customers or with rows as patterns, and the two readings give different graphs. The same public repository ships the de Carvalho and Soma (2015) instances as customers by patterns and the Chu and Stuckey (2009) instances beside them as patterns by customers. Only published densities and values tell them apart. Read the wrong way round, an instance certifies at a different value.

6 Conclusion

We asked which equalities of the Linhares–Yanasse table are true, and what follows when results, algorithms and benchmarks are moved between its twelve problems through pathwidth. Eight rows, counting the pathwidth row itself, hold at a fixed offset: each of those eight problems equals pathwidth, or pathwidth plus one, on every input, and these are the rows we call exact. Two rows, split bandwidth and edge search, are bands one and two wide, and two, PLA folding and edge separation, are false as stated, although a variant of each is exact (Table 2.3). This is the sense in which the exact problems form a *pathwidth complex*: they are *one* quantity, and the

literature names that quantity *pathwidth*. Moving between the problems paid in both directions: the complex gets an excellent algorithm from the MOSP side and an excellent lower bound from pathwidth. Read as a pathwidth algorithm, the best exact open-stacks search turned out to rest on two false dominance theorems; our corrected version of the definite move was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1), and with it and our corrected better move the search is sound. Read from pathwidth to MOSP, the minor lower bound, which rests on pathwidth never increasing under contraction, reaches values that no degree or clique bound can, and settles in seconds MOSP instances whose refutation took core-days (Section 4.7). And one certified pathwidth answers every problem of the pathwidth complex at once, which made a single dataset for all of them possible, including the three members the table did not list: the maximum wavefront of sparse matrix elimination, progressive pebbling and vertex search (Section 2.2 and Appendix B.3).

What transfers, and what does not. Within the exact core, anything proved or computed for one problem holds for the others with the offset of Table 2.3: an algorithm, a lower bound, a benchmark value. Across a band only an approximation transfers. Split bandwidth and edge search stay within one of the number of open stacks, as the table's literal reading says, but not at a fixed offset, so an exact value of one does not give an exact value of the other. Across a false row nothing transfers. Simple PLA folding can need arbitrarily more tracks than the pathwidth suggests, and cutwidth is not within a constant of pathwidth on stars. Section 3 showed why these distinctions went unchecked for two decades: the communities rarely read one another, and most of what is known about the family is published under "pathwidth".

What the search teaches about dominance rules. The two false theorems were published in 2009 and restated in later work, and the implementations that use them reported no failure. On every benchmark, and in every run of our checks where Chu and Stuckey's algorithm lost the last solution at a node, another branch found an optimum (Section 4.5). It took a search built for the purpose to find a whole instance with a wrong optimum, 34 customers and one stack too many (Figure 4.2), in our implementation of Chu and Stuckey's algorithm with the better move off; the optimum of 6 is confirmed by a SAT solver with a DRAT proof checked by drat-trim, and the whole run is a theorem checked in Lean (Theorem 4.9): a false dominance theorem, run as published, returns a wrong optimum. So the absence of wrong answers on benchmarks was never evidence that the rules were sound. Three findings of this paper bear on the soundness of dominance rules. Reading the rules in pathwidth language put the correct rule, the commitment lemma, beside the published one. Proving each rule on its own was not enough: the order in which the rules run matters, because a cycle of true links can still lose a node (Section 4.5). And a certificate whose checker recomputes every premise rejects the false refutation that a checker of the published premises accepts (Section 5.5). Our correction is cheap: one bipartite matching per candidate and at most 2% more nodes on each group of benchmarks we measured. Our corrected algorithm re-proved all 115 benchmark optima that rested on Chu and Stuckey's algorithm, and changed none.

What a reader can do differently. Cite our table as corrected here, Table 2.3, rather than the 2002 one. Look for results on MOSP and gate matrix layout under "pathwidth". Use our corrected algorithm as an exact pathwidth solver. On every benchmark set with published exact pathwidth

counts it proves as many or more graphs than the published branch and bound of Coudert et al. (2016) (Table 5.3). We believe it is currently the strongest exact pathwidth solver openly available, and it is open source. And in any implementation of the rules of Chu and Stuckey (2009), including the code behind the optima of Frinhani et al. (2018) that Martin et al. (2022) reuse (Appendix B.6), replace the definite and better moves of Chu and Stuckey's algorithm with the matching test. The values those implementations report are not known to be wrong, but their refutations are not proved.

Limits. Six limits remain.

- The Lean model of the search and the C and Python code that run it are linked by testing and by certificates, not by proof (Section 4.5).
- The certificate checker of Section 5.5 accepts a refutation when every move the search discarded is justified by a checked step and the justifications never go round in a circle. That such a certificate really proves the refutation is argued on paper there, and is not yet formalised in Lean; it could be, in the style of our other proofs about the search.
- LaPaugh's equality es = pes is stated and not formalised. No row needs it (Section 2.2).
- One reference of the table, Kashiwabara and Fujisawa (1979), is one we could not obtain. Its row, interval thickness, is proved from two other references of the table (Section 2.2).
- The head-to-head comparison of the graph solver (Section 5.4) is against one competitor, the branch and bound of Coudert et al. (2016) as shipped in SageMath, on one machine that was busy with other work, under a cap on processor time rather than on the clock; the comparisons with the other published numbers are with numbers obtained on other hardware.
- The census of names covers the twelve names of the table; the three members we add, and the wider treewidth and pebbling literatures, are not counted (Section 3).

Open problems. Every value of the MOSP corpus is now certified: the last two, `Random-125-125-2-3_0` and `Random-125-125-2-2_0`, fell to the split search (Section 5.2), at 1,128 and 2,231 core-hours. Of the 17,714 classes of the dataset, 1,079 have no certified pathwidth: 337 have a witnessed upper bound and 742 have no value. On the Rome graphs each step up in the time budget closed only about 40% of what it attempted, so the other graph sets will not close by budget alone. The 610 MOSP instances with 400 to 1,000 customers, and the 132 other open classes above the implementation's limit of 1,024 vertices per component (Section 5.1), are beyond an exact search. Beyond budgets, a class is certified without search only when a lower bound meets its witness or a theorem gives its value, as for the grids (Section 5.6). The open classes are therefore also a test set for stronger lower bounds on pathwidth.

Data, code and proofs

Our code, proofs and data are openly available at `https://github.com/Alex-Linhares/MOSP_trough_SAT`; the code and the proofs are open source under the MIT licence. Everything

this paper reports can be re-run from there, and most of it can be checked without re-running. The Lean proofs check in one build. Witnesses, minor certificates and the 34-customer instance have independent checkers, and refutations up to about 75 customers carry certificates (Section 5.5). The largest refutations demand enormous certificates and, hence, can only be re-run, at up to 2,231 core-hours each (Section 5.2). The repository holds:

- the Lean 4 development, with every theorem the paper names (Appendix A) and an axiom check that covers all of them;
- the source of both our solvers, the open-stacks customer search and the graph pathwidth solver, in C and in Python, with our corrected rules as the default and the rules of Chu and Stuckey's algorithm as an option;
- the graph pathwidth solver as a standalone package (`pathwidth_solver/`), with the scripts that produced Table 5.3 and the published values it is compared with, and the paired protocol and rows of the head-to-head run against SageMath of Table 5.4, the full run and the 50-graph variance sample;
- the dataset of Section 5.6, the certificates of Section 5.5 and the independent checkers of both; the benchmark graphs keep the terms of their original distributions;
- the 34-customer instance of Figure 4.2, with the optimum our implementation of Chu and Stuckey's algorithm reports with the better move off and its order, an optimal order, a checker that uses only the Python standard library, and the DRAT refutation of 5 stacks with the script that checks it (`paper1/data/false_refutation/`);
- the bibliometric data of Section 3: the cached OpenAlex responses, the relevance labels and the mechanical rule;
- a script for every table, figure and number in the paper, each of which regenerates it from the recorded data.

Acknowledgements

Use of AI tools. Much of this work was carried out with Claude Code (Anthropic), an AI coding assistant running Claude models. Under the authors' direction it wrote most of the C and Python software, the Lean proofs and drafts of the text, ran the computations, and also searched and summarised the literature. The authors chose the questions, directed the work, read the results and take full responsibility for the content of this paper. We did not take the AI's word for any result: every theorem the paper states is checked by the Lean kernel, except the two that the limits of Section 6 name as not formalised; every value in the dataset carries a witness that an independent checker verifies; and the refutations behind the central claims (the wrong optimum of Theorem 4.9, and the minor bounds for the Mycielski graphs and four MOSP instances) are re-checked by methods independent of our search: a SAT solver with a DRAT proof that drat-trim verifies, and a separate exact search with its own checker, written from scratch (`reports/minor_bound_verification.md`, `reports/scratch/minor_certificates/` and `tests/test_false_refutation_instance.py`). The other refutations rest on our corrected algorithm, whose rules are proved sound in Lean (Sections 4 and 5).

Software. The formal proofs use Lean 4 (de Moura and Ullrich, 2021), version 4.34, with the Mathlib library (The mathlib Community, 2020). The SAT checks use Kissat 4.0.4 (`https://github.com/arminbiere/kissat`) and CaDiCaL 1.9.5 (Biere et al., 2024) through PySAT (Ignatiev et al., 2018), and the proofs of unsatisfiability are checked by drat-trim (Wetzler et al., 2014). Isomorphism classes are computed with nauty (McKay and Piperno, 2014), through pynauty, and the graphs are handled with NetworkX. The bibliometric counts use OpenAlex (Priem et al., 2022). SageMath provides the openly available implementation of the branch and bound of Coudert et al. (2016).

References

Luisa Andreis, Wolfgang König, and Robert I. A. Patterson. A large-deviations principle for all the cluster sizes of a sparse Erdős–Rényi graph. *Random Structures & Algorithms*, 59(4):522–553, 2021. doi: 10.1002/rsa.21007.

J. Christopher Beck, Ryo Kuroiwa, Jimmy H. M. Lee, Peter J. Stuckey, and Allen Z. Zhong. Transition dominance in domain-independent dynamic programming. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 5:1–5:23, 2025. doi: 10.4230/LIPIcs.CP.2025.5.

Daniel Bienstock and Paul Seymour. Monotonicity in graph searching. *Journal of Algorithms*, 12 (2):239–245, 1991. doi: 10.1016/0196-6774(91)90003-H.

Daniel Bienstock, Neil Robertson, Paul Seymour, and Robin Thomas. Quickly excluding a forest. *Journal of Combinatorial Theory, Series B*, 52(2):274–283, 1991. doi: 10.1016/0095-8956(91)90068-U.

Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification (CAV 2024)*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152, 2024. doi: 10.1007/978-3-031-65627-9_7.

Hans L. Bodlaender and Arie M. C. A. Koster. Treewidth computations II. lower bounds. *Information and Computation*, 209(7):1103–1119, 2011. doi: 10.1016/j.ic.2011.04.003.

Hans L. Bodlaender, Thomas Wolle, and Arie M. C. A. Koster. Contraction and treewidth lower bounds. *Journal of Graph Algorithms and Applications*, 10(1):5–49, 2006.

Hans L. Bodlaender, Fedor V. Fomin, Arie M. C. A. Koster, Dieter Kratsch, and Dimitrios M. Thilikos. A note on exact algorithms for vertex ordering problems on graphs. *Theory of Computing Systems*, 50(3):420–432, 2012. doi: 10.1007/s00224-011-9312-0.

L. Sunil Chandran and T. Kavitha. The treewidth and pathwidth of hypercubes. *Discrete Mathematics*, 306(3):359–365, 2006. doi: 10.1016/j.disc.2005.12.011.

Kevin K. H. Cheung and Patrick Girardet. Improved approximation of layout problems on random graphs. *Open Journal of Discrete Mathematics*, 10(1):13–30, 2020. doi: 10.4236/ojdm.2020.101003.

Hsin-Hung Chou, Ming-Tat Ko, Chin-Wen Ho, and Gen-Huey Chen. Node-searching problem on block graphs. *Discrete Applied Mathematics*, 156(1):55–75, 2008. doi: 10.1016/j.dam.2007.08.007.

Geoffrey Chu and Peter J. Stuckey. Minimizing the maximum number of open stacks by customer search. In *Principles and Practice of Constraint Programming (CP 2009)*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257. Springer, 2009. doi: 10.1007/978-3-642-04244-7_21.

Geoffrey G. Chu. *Improving Combinatorial Optimization*. PhD thesis, Department of Computer Science and Software Engineering, The University of Melbourne, 2011. Minerva Access, http://hdl.handle.net/11343/36679.

David Coudert, Dorian Mazauric, and Nicolas Nisse. Experimental evaluation of a branch and bound algorithm for computing pathwidth. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 46–58. Springer, 2014. doi: 10.1007/978-3-319-07959-2_5.

David Coudert, Dorian Mazauric, and Nicolas Nisse. Experimental evaluation of a branch-and-bound algorithm for computing pathwidth and directed pathwidth. *ACM Journal of Experimental Algorithmics*, 21:1–23, 2016. doi: 10.1145/2851494.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006. doi: 10.1002/047174882X.

Marco Antonio Moreira de Carvalho and Nei Yoshihiro Soma. A breadth-first search applied to the minimization of the open stacks. *Journal of the Operational Research Society*, 66(6):936–946, 2015. doi: 10.1057/jors.2014.60.

Luigi De Giovanni, Gionata Massi, and Ferdinando Pezzella. Preliminary computational experiments with a genetic algorithm for the open stack problem. Technical report, Dipartimento di Matematica Pura ed Applicata, Università degli Studi di Padova, and Università Politecnica delle Marche, Padova and Ancona, 2010. Technical report of the SCOOP project.

Luigi De Giovanni, Gionata Massi, and Ferdinando Pezzella. An adaptive genetic algorithm for large-size open stack problems. *International Journal of Production Research*, 51(3):682–697, 2013a. doi: 10.1080/00207543.2012.657256.

Luigi De Giovanni, Gionata Massi, Ferdinando Pezzella, Marc E. Pfetsch, Giovanni Rinaldi, and Paolo Ventura. A heuristic and an exact method for the gate matrix connection cost minimization problem. *International Transactions in Operational Research*, 20(5):627–643, 2013b. doi: 10.1111/itor.12025.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37.

A. C. M. de Oliveira and L. A. N. Lorena. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(8): 969–974, 2002. doi: 10.1109/TCAD.2002.800454.

Holger Dell, Thore Husfeldt, Bart M. P. Jansen, Petteri Kaski, Christian Komusiewicz, and Frances A. Rosamond. The first parameterized algorithms and computational experiments challenge. In *11th International Symposium on Parameterized and Exact Computation (IPEC 2016)*, volume 63 of *LIPIcs*, pages 30:1–30:9, 2017. doi: 10.4230/LIPIcs.IPEC.2016.30.

Holger Dell, Christian Komusiewicz, Nimrod Talmon, and Mathias Weller. The PACE 2017 parameterized algorithms and computational experiments challenge: The second iteration. In *12th International Symposium on Parameterized and Exact Computation (IPEC 2017)*, volume 89 of *LIPIcs*, pages 30:1–30:12, 2018. doi: 10.4230/LIPIcs.IPEC.2017.30.

Giuseppe Di Battista, Ashim Garg, Giuseppe Liotta, Roberto Tamassia, Emanuele Tassinari, and Francesco Vargiu. An experimental comparison of four graph drawing algorithms. *Computational Geometry*, 7(5–6):303–325, 1997. doi: 10.1016/S0925-7721(96)00005-3.

Josep Díaz, Mathew D. Penrose, Jordi Petit, and Maria Serna. Approximating layout problems on random geometric graphs. *Journal of Algorithms*, 39(1):78–116, 2001a. doi: 10.1006/jagm.2000.1149.

Josep Díaz, Jordi Petit, Maria Serna, and Luca Trevisan. Approximating layout problems on random graphs. *Discrete Mathematics*, 235(1–3):245–253, 2001b. doi: 10.1016/S0012-365X(00)00278-8.

Reinhard Diestel and Sang-il Oum. Tangle-tree duality: in graphs, matroids and beyond. *Combinatorica*, 39:879–910, 2019. doi: 10.1007/s00493-019-3798-5. arXiv:1701.02651.

Tuan Anh Do, Joshua Erde, and Mihyun Kang. A note on the width of sparse random graphs. *Journal of Graph Theory*, 106(2):273–295, 2024. doi: 10.1002/jgt.23081.

Abraham Duarte, Laureano F. Escudero, Rafael Martí, Nenad Mladenovic, Juan José Pantrigo, and Jesús Sánchez-Oro. Variable neighborhood search for the vertex separation problem. *Computers & Operations Research*, 39(12):3247–3255, 2012. doi: 10.1016/j.cor.2012.04.017.

Keith Edwards and Graham Farr. Improved upper bounds for planarization and series-parallelization of degree-bounded graphs. *Electronic Journal of Combinatorics*, 19(2):P25, 2012. doi: 10.37236/2378.

Keith Edwards and Eric McDermid. A general reduction theorem with applications to pathwidth and the complexity of MAX 2-CSP. *Algorithmica*, 72(4):940–968, 2015. doi: 10.1007/s00453-014-9883-7.

J. A. Ellis, I. H. Sudborough, and J. S. Turner. The vertex separation and search number of a graph. *Information and Computation*, 113(1):50–79, 1994. doi: 10.1006/inco.1994.1064.

John Ellis and Robert Warren. Lower bounds on the pathwidth of some grid-like graphs. *Discrete Applied Mathematics*, 156(5):545–555, 2008. doi: 10.1016/j.dam.2007.02.006.

Joshua Erde. Directed path-decompositions, 2017. arXiv:1711.00718.

Enrico Faggioli and Carlo Alberto Bentivoglio. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research*, 110(3):564–575, 1998. doi: 10.1016/S0377-2217(97)00268-3.

Michael R. Fellows and Michael A. Langston. Nonconstructive advances in polynomial-time complexity. *Information Processing Letters*, 26(3):157–162, 1987. doi: 10.1016/0020-0190(87)90054-8.

Michael R. Fellows and Michael A. Langston. On search decision and the efficiency of polynomial-time algorithms. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing (STOC '89)*, pages 501–512, 1989. doi: 10.1145/73007.73055.

Andreas Fink and Stefan Voß. Applications of modern heuristic search methods to pattern sequencing problems. *Computers & Operations Research*, 26(1):17–34, 1999. doi: 10.1016/S0305-0548(98)80001-4.

Claudia Fink. *O problema de minimização de pilhas abertas – novas contribuições*. PhD thesis, Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2012.

Fedor V. Fomin. Helicopter search problems, bandwidth and pathwidth. *Discrete Applied Mathematics*, 85(1):59–70, 1998. doi: 10.1016/S0166-218X(97)00131-5.

Fedor V. Fomin and Kjartan Høie. Pathwidth of cubic graphs and exact algorithms. *Information Processing Letters*, 97(5):191–196, 2006. doi: 10.1016/j.ipl.2005.10.012.

Rafael de Magalhães Dias Frinhani, Marco Antonio Moreira de Carvalho, and Nei Yoshihiro Soma. A PageRank-based heuristic for the minimization of open stacks problem. *PLOS ONE*, 13(8):e0203076, 2018. doi: 10.1371/journal.pone.0203076.

Yong Gao. Treewidth of Erdős–Rényi random graphs, random intersection graphs, and scale-free random graphs. *Discrete Applied Mathematics*, 160(4–5):566–578, 2012. doi: 10.1016/j.dam.2011.10.013.

Maria Garcia de la Banda and Peter J. Stuckey. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, 19(4):607–617, 2007. doi: 10.1287/ijoc.1060.0205.

Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.

Carla Groenland, Gwenaël Joret, Wojciech Nadara, and Bartosz Walczak. Approximating pathwidth for graphs of small treewidth. *ACM Transactions on Algorithms*, 19(2):16:1–16:19, 2023. doi: 10.1145/3576044. SODA 2021; arXiv:2008.00779.

Martin Grohe and Dániel Marx. On tree width, bramble size, and expansion. *Journal of Combinatorial Theory, Series B*, 99(1):218–228, 2009. doi: 10.1016/j.jctb.2008.06.004.

L. H. Harper. Optimal numberings and isoperimetric problems on graphs. *Journal of Combinatorial Theory*, 1(3):385–393, 1966. doi: 10.1016/S0021-9800(66)80059-5.

Meike Hatzel, Paweł Komosa, Marcin Pilipczuk, and Manuel Sorge. Constant congestion brambles. *Discrete Mathematics & Theoretical Computer Science*, 24(1), 2022. arXiv:2008.02133.

Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing – SAT 2018*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437, 2018. doi: 10.1007/978-3-319-94144-8_26.

Toshinobu Kashiwabara and Toshio Fujisawa. NP-completeness of the problem of finding a minimum clique number interval graph containing a given graph as a subgraph. In *Proceedings of the 1979 IEEE International Symposium on Circuits and Systems*, pages 657–660, Tokyo, Japan, 1979.

Nancy G. Kinnersley. The vertex separation number of a graph equals its path-width. *Information Processing Letters*, 42(6):345–350, 1992. doi: 10.1016/0020-0190(92)90234-M.

Lefteris M. Kirousis and Christos H. Papadimitriou. Interval graphs and searching. *Discrete Mathematics*, 55(2):181–184, 1985. doi: 10.1016/0012-365X(85)90046-9.

Lefteris M. Kirousis and Christos H. Papadimitriou. Searching and pebbling. *Theoretical Computer Science*, 47:205–218, 1986. doi: 10.1016/0304-3975(86)90146-5.

Kenta Kitsunai, Yasuaki Kobayashi, Keita Komuro, Hisao Tamaki, and Toshihiro Tano. Computing directed pathwidth in $O(1.89^n)$ time. *Algorithmica*, 75(1):138–157, 2016. doi: 10.1007/s00453-015-0015-9.

Joachim Kneis, Daniel Mölle, Stefan Richter, and Peter Rossmanith. A bound on the pathwidth of sparse graphs with applications to exact algorithms. *SIAM Journal on Discrete Mathematics*, 23(1):407–427, 2009. doi: 10.1137/080715482.

Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability*. Addison-Wesley, Boston, 2015.

Yasuaki Kobayashi, Keita Komuro, and Hisao Tamaki. Search space reduction through commitments in pathwidth computation: An experimental study. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 388–399. Springer, 2014. doi: 10.1007/978-3-319-07959-2_33.

Brett Kolesnik and Nicholas Wormald. Lower bounds for the isoperimetric numbers of random regular graphs. *SIAM Journal on Discrete Mathematics*, 28(1):553–575, 2014. doi: 10.1137/120891265.

András Kornai and Zsolt Tuza. Narrowness, pathwidth, and their application in natural language processing. *Discrete Applied Mathematics*, 36(1):87–92, 1992. doi: 10.1016/0166-218X(92)90208-R.

Gary Kumfert and Alex Pothen. Two improved algorithms for envelope and wavefront reduction. *BIT Numerical Mathematics*, 37(3):559–590, 1997. doi: 10.1007/BF02510240.

Andrea S. LaPaugh. Recontamination does not help to search a graph. *Journal of the ACM*, 40(2): 224–245, 1993. doi: 10.1145/151261.151263.

L. Larisch and F. Salfelder. freetdi: named graphs and control-flow graphs. https://github.com/freetdi/named-graphs, https://github.com/freetdi/CFGs, n.d. Licence CC0 1.0.

Choongbum Lee, Joonkyung Lee, and Sang-il Oum. Rank-width of random graphs. *Journal of Graph Theory*, 70(3):339–347, 2012. doi: 10.1002/jgt.20620.

Thomas Lengauer. Black-white pebbles and graph separation. *Acta Informatica*, 16(4):465–475, 1981. doi: 10.1007/BF00264496.

Alexandre Linhares. *Industrial Pattern Sequencing Problems: Some Complexity Results and New Local Search Models*. PhD thesis, National Institute of Space Research (INPE), São José dos Campos, Brazil, 2001. Doctoral thesis in Applied Computing, advisor H. H. Yanasse.

Alexandre Linhares and Horacio Hideki Yanasse. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research*, 29(14):1759–1772, 2002. doi: 10.1016/S0305-0548(01)00054-5.

Malwina J. Luczak and Colin McDiarmid. Bisecting sparse random graphs. *Random Structures & Algorithms*, 18(1):31–38, 2001. doi: 10.1002/1098-2418(200101)18:1<31::AID-RSA3>3.0.CO;2-1.

Sven Mallach. Linear ordering based MIP formulations for the vertex separation or pathwidth problem. *Journal of Discrete Algorithms*, 52–53:156–167, 2018. doi: 10.1016/j.jda.2018.11.012.

Rafael Martí, Vicente Campos, and Estefanía Piñana. A branch and bound algorithm for the matrix bandwidth minimization. *European Journal of Operational Research*, 186(2):513–528, 2008. doi: 10.1016/j.ejor.2007.02.004.

Mateus Martin, Horacio Hideki Yanasse, and Maria José Pinto. Mathematical models for the minimization of open stacks problem. *International Transactions in Operational Research*, 29(5): 2944–2967, 2022. doi: 10.1111/itor.13053.

Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. doi: 10.1016/j.jsc.2013.09.003.

Rolf H. Möhring. Graph problems related to gate matrix layout and PLA folding. In *Computational Graph Theory*, volume 7 of *Computing Supplementum*, pages 17–51. Springer-Verlag, Wien, 1990. doi: 10.1007/978-3-7091-9076-0_2.

Burkhard Monien and Robert Preis. Upper bounds on the bisection width of 3- and 4-regular graphs. *Journal of Discrete Algorithms*, 4(3):475–498, 2006. doi: 10.1016/j.jda.2005.12.009.

Neil O'Connell. Some large deviation results for sparse random graphs. *Probability Theory and Related Fields*, 110(3):277–285, 1998. doi: 10.1007/s004400050149.

Tatsuo Ohtsuki, Hajimu Mori, Ernest S. Kuh, Toshinobu Kashiwabara, and Toshio Fujisawa. One-dimensional logic gate assignment and interval graphs. *IEEE Transactions on Circuits and Systems*, 26(9):675–684, 1979. doi: 10.1109/TCS.1979.1084695.

Guillem Perarnau and Oriol Serra. On the tree-depth of random graphs. *Discrete Applied Mathematics*, 168:119–126, 2014. doi: 10.1016/j.dam.2012.10.031.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833, 2022.

Anatolii A. Puhalskii. Stochastic processes in random graphs. *The Annals of Probability*, 33(1): 337–412, 2005. doi: 10.1214/009117904000000784.

Paul D. Seymour and Robin Thomas. Graph searching and a min-max theorem for tree-width. *Journal of Combinatorial Theory, Series B*, 58(1):22–33, 1993. doi: 10.1006/jctb.1993.1027.

Barbara M. Smith and Ian P. Gent. Constraint modelling challenge 2005. In *Fifth Workshop on Modelling and Solving Problems with Constraints, held at IJCAI 2005*, Edinburgh, 2005.

Karol Suchan and Ioan Todinca. Pathwidth of circular-arc graphs. In *Graph-Theoretic Concepts in Computer Science (WG 2007)*, volume 4769 of *Lecture Notes in Computer Science*, pages 258–269, 2007. doi: 10.1007/978-3-540-74839-7_25.

Karol Suchan and Yngve Villanger. Computing pathwidth faster than 2^n. In *Parameterized and Exact Computation (IWPEC 2009)*, volume 5917 of *Lecture Notes in Computer Science*, pages 324–335. Springer, 2009. doi: 10.1007/978-3-642-11269-0_27.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. doi: 10.1145/3372885.3373824.

Jonathan S. Turner. On the probable performance of heuristics for bandwidth minimization. *SIAM Journal on Computing*, 15(2):561–580, 1986. doi: 10.1137/0215041.

J.-W. van den Broek and Hans L. Bodlaender. TreewidthLIB. http://www.cs.uu.nl/research/projects/treewidthlib/, n.d. Graph files no longer served; colouring subset obtained from https://mat.tepper.cmu.edu/COLOR/instances.html.

Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429, 2014. doi: 10.1007/978-3-319-09284-3_31.

Omar Wing, S. Huang, and R. Wang. Gate matrix layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 4(3):220–231, 1985. doi: 10.1109/TCAD.1985.1270118.

Horacio Hideki Yanasse. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional*, 17(1):57–70, 1997a.

Horacio Hideki Yanasse. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research*, 100(3):454–463, 1997b. doi: 10.1016/S0377-2217(97)84107-0.

Horacio Hideki Yanasse and Edson Luiz França Senne. The minimization of open stacks problem: A review of some properties and their use in pre-processing operations. *European Journal of Operational Research*, 203(3):559–567, 2010. doi: 10.1016/j.ejor.2009.09.017.

Appendix A Formal names

Every theorem, lemma and counterexample of this paper is checked in the Lean 4 proof assistant with the Mathlib library, unless the text says otherwise. The reader need not read Lean to follow the paper. This appendix is for a reader who wants to find the formal statement of a result: Table A.1 maps each result, in the order in which it appears, to the names of the Lean declarations that state it. Where a row cites a definition rather than a theorem, it is the definition the text uses. Lean names use `repaired` for what the text calls *corrected*.

All the theorems named depend only on the three standard axioms of Lean's classical logic, `propext`, `Classical.choice` and `Quot.sound`; an axiom check in the repository prints this for every theorem named in the paper. The development has no `sorry`, Lean's marker for a statement left unproved.

Two Lean proofs take a route that the text does not describe. The proof of Theorem 4.2 goes through narrowness (Kornai and Tuza, 1992), which counts the closing order forwards rather than reversing it. The proof of Theorem 4.7 reduces the deficiency form of Hall's theorem to Mathlib's Hall theorem by adding δ dummy stacks. One Lean proof takes a different route from the published one: the proof that node search is monotone (Theorem 2.4(5)) follows the crusade argument of Bienstock and Seymour (1991), not the reduction to edge search of LaPaugh (1993) on which Kirousis and Papadimitriou (1986) rely. Three kinds of fact are computed rather than proved. The values of K_2, $K_{1,3}$ and $K_{3,3}$ after Theorem 2.5, which show that every end of the two bands occurs, are computed by brute force from each quantity's own definition. The search for smaller counterexamples (Appendix B.5) and the results of Section 5 are computations.

Table A.1: Every result of the paper has a formal statement in Lean, listed here in the order in which the results appear.

Entry	Result in the paper	Lean names
Section 2, the equivalences		
A.1	Lemma 2.1, cliques sit in a bag	`PathDecomposition.exists_bag_of_isClique`
A.2	Theorem 2.2, vertex separation is pathwidth	`vertexSeparation_eq_pathwidth`
A.3	Theorem 2.3, MOSP is pathwidth plus one	`MOSPInstance.mospValue_eq_pathwidth_add_one`
A.4	The pattern graph gives no bound: the star $K_{1,4}$ (after Theorem 2.3)	`star_mospValue`, `star_pathwidth_agreementGraph`, `star_refutes_pattern_graph_bound`
A.5	Theorem 2.4(1), gate matrix layout	`NetGateMatrix.tracks_eq_mospValue`, `NetGateMatrix.tracks_eq_pathwidth_add_one`
A.6	Theorem 2.4(2), interval thickness	`intervalThickness_eq_pathwidth_add_one`
A.7	Theorem 2.4(3), one-dimensional logic	`LogicArray.tracks_eq_intervalThickness`, `LogicArray.tracks_eq_pathwidth_add_one`
A.8	Theorem 2.4(4), narrowness	`narrowness_eq_pathwidth_add_one`

Entry	Result in the paper	Lean names
A.9	Theorem 2.4(5), node search	`nodeSearchMonotonicity`, `nodeSearch_eq_vertexSeparation_add_one`, `nodeSearch_chain`
A.10	Theorem 2.4(6), multiple folding	`NetGateMatrix.foldTracks_eq_pathwidth_add_one`
A.11	Theorem 2.4(7), Lengauer's vertex separator game	`vsg_eq_max`
A.12	Theorem 2.5, the bands	`pathwidth_le_splitBandwidth_le_pathwidth_add_one`, `vertexSeparation_le_edgeSearch_le_add_two`, `vertexSeparation_le_progressiveEdgeSearch_le_add_two`
A.13	Theorem 2.6(1), PLA folding	`plaTracks_idMatrix_five`, `plaTracks_idMatrix_unbounded`, `plaTracks_pathMatrix_unbounded`
A.14	Theorem 2.6(2), edge separation read as cutwidth	`cutwidth_unbounded`, `modCutwidth_unbounded`
Section 4, the search		
A.15	Step cost, Ext_k and Sol_k (Section 4.1)	`stepCost`, `Solvable`, `SearchSol`
A.16	Lemma 4.1, free moves never hurt	`solvable_iff_searchSol_cl`
A.17	Theorem 4.2, the search decides pathwidth	`searchSol_empty_iff_pathwidth_add_one_le`, `searchSol_mospGraph_iff_mospValue_le`
A.18	Table 4.1: closing order from $\emptyset$	`orderOfLayout`
A.19	Table 4.1: open stacks $b(T)$	`openStacks_eq_card_boundary`
A.20	Table 4.1: cost(T, c)	`stepCost_eq_card_boundary_add_one`
A.21	Table 4.1: $\mathrm{Sol}_k(\emptyset)$	`searchSol_empty_iff_pathwidth_add_one_le`
A.22	Table 4.1: free closure cl(T)	`mem_cl_iff`
A.23	Table 4.1: definite move, corrected	`isHereditarilyDefinite_iff_isCommittable`
A.24	Lemma 4.3, what the premise says	`closeCount_eq`, `isDefinite_iff`
A.25	Counterexample 4.4	`definiteMove_counterexample`, `cexFamily_closed`
A.26	Chu and Stuckey's Theorem 1 (the definite move), under both readings, and Chu's Theorem 6.3.6 are false	`chuStuckey_theorem1_false`, `chuStuckey_theorem1_false_literal`, `chuThesis_theorem636_false`
A.27	Fink's restatement of the definite move is false	`fink_theorem1_false`
A.28	The hereditarily definite premise, and its failure in Counterexample 4.4	`IsHereditarilyDefinite`, `not_isHereditarilyDefinite_cex`
A.29	Lemma 4.5, open stacks are submodular	`openStacks_submodular`
A.30	Theorem 4.6, the corrected definite move is sound	`solvable_cl_insert_of_hereditarilyDefinite`, `searchSol_cl_insert_of_hereditarilyDefinite`
A.31	Theorem 4.7, the corrected premise is a matching condition	`isHereditarilyDefinite_iff_hasDefiniteMatching`

Entry	Result in the paper	Lean names
A.32	Lemma 1 of Kitsunai et al. (2016), transcribed for digraphs in their terms and proved by their argument; on the symmetric digraph with the extension fixed to the child, its commitment condition is the hereditary premise, our theorem follows from it under its two feasibility hypotheses, it follows from ours, and a five-vertex graph shows those hypotheses are not redundant	`Tamaki.lemma1`, `Tamaki.inDeg_submodular`, `Tamaki.isCommittableExt_symm_iff_hereditarilyDefinite`, `Tamaki.solvable_cl_insert_of_lemma1`, `Tamaki.lemma1_symm_of_isCommittable`, `Tamaki.premise_gap`
A.33	The published definite move is right when open$(q, S) \leq 1$	`isHereditarilyDefinite_of_openCount_le_one`
A.34	The subset rule is sound	`searchSol_cl_insert_of_newlyOpened_subset`, `subsetFilter_sound`
A.35	The subset rule needs its tie-break (7-customer graph)	`noTieBreak_counterexample`
A.36	The better move's premise, restated	`isBetter_iff`
A.37	Chu and Stuckey's Theorem 2 (the better move) is false	`betterMove_counterexample`, `chuStuckey_theorem2_false`
A.38	Chu's Theorem 6.3.8 is false	`chuThesis_theorem638_false`, `chuThesis_theorem638_false_literal`
A.39	The corrected better move is sound	`IsRepairedBetter`, `searchSol_cl_insert_of_repairedBetter`
A.40	The better move's two requirements: the close count and the rule order	`bugA_counterexample`, `bugB_counterexample`
A.41	The old move is sound	`searchSol_reinsert`, `not_searchSol_of_oldMove`
A.42	The old move needs its reinsertion test (path on five vertices)	`reinsert_needs_test`
A.43	Solvability of a state does not depend on the path to it (the memo)	`solvable_iff_of_cl_eq`
A.44	A node with an unrefuted old move can record a solvable state	`exec_fake_oldMove`
A.45	Node-soundness of a filter (Section 4.5)	`NodeSoundAt`
A.46	Theorem 4.8, our corrected algorithm is sound	`exec_repairedFullFilter_sound`, `exec_repairedFullFilter_pathwidth`, `exec_repairedFullFilter_mospValue`
A.47	Under Chu and Stuckey's algorithm the filter is not node-sound	`not_codeFilterSound_cexGraph`
A.48	Figure 4.2: Chu and Stuckey's algorithm refutes 6 and finds 7 on the 34-customer instance, whose optimum is 6 and pathwidth 5; its run is a run of the search with the published filter, which is therefore not sound there	`previousAlgorithm_wrong_optimum`, `not_filterSound_codeFilter`

Entry	Result in the paper	Lean names
A.49	Theorem 4.11: contracting an edge (merging v into u, as the solvers do) or deleting a vertex or an edge never increases pathwidth; along a legal contraction sequence, a minor of pathwidth above w forces pathwidth above w, and on the MOSP graph more than $w+1$ open stacks	`pathwidth_contractEdge_le`, `pathwidth_induce_le`, `pathwidth_mono`, `pathwidth_minorOf_le`, `pathwidth_gt_of_minorOf`, `MOSPInstance.lt_mospValue_of_minorOf`
A.50	The (h, w)-grid has pathwidth $\min(h, w)$, except the 1×1 grid (pathwidth 0); Ellis and Warren (2008, Theorem 4.1). Upper bound by the row-major layout's bandwidth; lower bound directly on vertex separation, at the first prefix that completes a row or a column	`pathwidth_grid`, `pathwidth_grid_le`, `min_le_pathwidth_grid`, `pathwidth_grid_one_one`, `pathwidth_grid_square`
A.51	The d-dimensional hypercube has pathwidth $\sum_{i<d} \binom{i}{\lfloor i/2 \rfloor}$ (Chandran and Kavitha, 2006, Theorem 2), with Harper's vertex-isoperimetric theorem (Harper, 1966) proved by compression. Upper bound by the layout along Harper's simplicial order, lower bound at a prefix of every size, both by induction on d and on vertex separation	`pathwidth_hypercube`, `pathwidth_cubeGraph`, `Hypercube.harper`, `Hypercube.card_nbhd_init_le`, `Hypercube.exists_init_profile`
A.52	For every graph G, its Mycielskian $\mu(G)$ (originals, a shadow of each vertex adjacent to its neighbours, and an apex adjacent to the shadows) has $\mathrm{pw}(G)+1 \le \mathrm{pw}(\mu(G)) \le 2\,\mathrm{pw}(G)+1$. The upper bound interleaves each vertex with its shadow and places the apex last; we found no published statement of it. The lower bound keeps the bags meeting the apex and the shadows	`pathwidth_mycielskian_le`, `pathwidth_add_one_le_mycielskian`
A.53	Step 1 of Appendix E: at every position of every layout the prefix and the inactive suffix are an empty pair, so for every $j \le n$ some j-set has closed neighbourhood of size at most $j + \mathrm{pw}$ (Harper's bound in layout form), and $n - 2s^* - 2 \le \mathrm{pw}$ for the empty-pair index s^*	`exists_empty_pair_of_layout`, `exists_closedNbhd_card_le`, `exists_empty_pair`, `card_sub_two_mul_emptyPairIndex_sub_two_le_pathwidth`
A.54	Step 2 of Appendix E: the number of graphs on n labelled vertices of pathwidth at most k is at most $\binom{n}{t}\binom{n-t}{t-1}2^{\binom{n}{2}-t(t-1)}$, $t = \lceil (n-k)/2 \rceil$, and the $G(n,p)$-weighted sum of those graphs is at most $\binom{n}{t}\binom{n-t}{t-1}(1-p)^{t(t-1)}$; there are $2^{\binom{n}{2}}$ graphs on n labelled vertices	`card_pathwidth_le_le`, `weightedSum_graphsOfPathwidthLe_le`, `card_emptyPairGraphs`, `card_simpleGraph`

Entry	Result in the paper	Lean names
A.55	Theorem E.1 and its consequence: the weighted sum is at most $\exp(-\delta n + 3\ln n + ca)$ for $n \geq \max(3, c)$, at most $e^{c/2} n^3 e^{-\delta n}$, and tends to 0 when $\delta > 0$; the trinomial entropy bound from the binomial theorem	`firstMoment_pathwidth_le_fin`, `firstMoment_pathwidth_le_fin'`, `tendsto_firstMoment_pathwidth`, `choose_mul_choose_le_exp_entropy`
A.56	Appendix E.4: for independent uniform patterns of given sizes, the probability that the instance's MOSP value is at most $k+1$ (its pathwidth at most k) is at most $\binom{n}{t}\binom{n-t}{t-1}\prod_j N(s_j)/\binom{n}{s_j}$, $N(s)$ the number of s-sets missing A or missing R	`prob_coversOfMospValueLe_le`, `prob_coversOfPathwidthLe_le`, `card_coversOfPathwidthLe_le`, `card_filter_avoid`
A.57	Appendix E.4, the coupling: if every pattern of one cover is contained in the corresponding pattern of another, every empty pair of the larger cover's graph is an empty pair of the smaller's, and the pathwidth is monotone	`isEmptyPair_coverGraph_of_subset`, `pathwidth_coverGraph_mono`
A.58	Theorem E.3, the blow-up bound: for every instance, MOSP $\leq \omega\,(\mathrm{pw}(B)+1)$ with ω the largest pattern and B the customer–pattern incidence graph (as a `SimpleGraph` on the sum type); a path decomposition of B blown up to one of the MOSP graph	`MOSPInstance.mospValue_le_maxPatternSize_mul`, `MOSPInstance.pathwidth_mospGraph_add_one_le`, `MOSPInstance.blowUp_width_add_one_le`
A.59	Theorem E.3, the forest form: if B is acyclic, MOSP $\leq \omega(\lfloor \log_3((6(n+m)+3)/5)\rfloor + 1)$; a forest on N vertices has pathwidth at most $\lfloor \log_3((6N+3)/5)\rfloor$, the least k with $N \leq (5\cdot 3^k - 3)/2$; a tree on at most $(5\cdot 3^k-3)/2$ vertices has pathwidth at most k	`MOSPInstance.mospValue_le_of_incidence_isAcyclic`, `pathwidth_le_treeBoundInv_of_isAcyclic`, `le_treeBound_treeBoundInv`, `treeBound_lt_of_lt_treeBoundInv`, `tree_pathwidth_le_treeBound`
A.60	Appendix E.5: the MOSP value is monotone under adding requirements (Yanasse's subinstance monotonicity)	`MOSPInstance.mospValue_mono`
A.61	Lemma E.5, the layer lemma: for any level function with adjacent vertices at most one level apart, $\mathrm{pw} \leq \max_i(\lvert L_i\rvert + \lvert L_{i+1}\rvert) - 1$, in pathwidth and vertex-separation form, and for the distance layers from any vertex of a connected graph	`pathwidth_le_layerWidth`, `vertexSeparation_le_layerWidth`, `pathwidth_le_layerWidth_dist`
A.62	Lemma E.5, the independent-set bound $\mathrm{pw} \leq n - \alpha$, by the bags $(V \setminus I) \cup \{v\}$	`pathwidth_le_card_sub_indepNum`

Entry	Result in the paper	Lean names
A.63	Lemma E.4, the two-sided pendant bound: a clique R with $q \geq 1$ cliques of size at least t meeting R in one vertex each, at distinct vertices, forces pw $\geq t + \lceil q/2 \rceil - 2$; for any graph, for one decomposition, in the depth-one case $t = q = \lvert R \rvert$, and in MOSP terms (a pattern with q patterns of size at least t hanging off distinct customers forces MOSP $\geq t + \lceil q/2 \rceil - 1$)	`pathwidth_ge_of_pendant_cliques`, `PathDecomposition.pendant_bound`, `pathwidth_ge_of_clique_pendant_cliques`, `MOSPInstance.mospValue_ge_of_pendant_patterns`
A.64	Theorem E.6, the deterministic step: a layout of width $k < n$ forces pairwise disjoint A, R, S covering V with $\lvert A \rvert = \lceil (n-k)/2 \rceil$, $\lvert R \rvert = \lfloor (n-k)/2 \rfloor$, $\lvert S \rvert = k$ and no A–R edge (on the MOSP graph: no pattern meets both A and R); a walk avoiding S that starts in A stays in A, so every component of $G - S$ has at most $\lceil (n-k)/2 \rceil$ vertices; $3K_2$ at $k = 0$ satisfies the component condition and not the bipartition one	`exists_balanced_separator`, `exists_balanced_separator_cover`, `exists_isBalancedBip`, `componentOff_subset_side`, `card_componentOff_le_of_isBalancedBip`, `not_isBalancedBip_threeK2`, `card_componentOff_threeK2_le`
A.65	The closed pair (Appendix E.5): $R \subseteq V \setminus N[A]$ iff (A, R) is disjoint and empty; one round of the alternating closure $R^* = V \setminus N[A]$, $A^* = V \setminus N[R^*]$ is monotone on empty pairs, lands on a closed pair from every A, is idempotent, has separator $N(A^*) \cap N(R^*)$ and keeps the sizes; a pair is closed iff it is empty and every outside vertex sees both sides; a layout of width $k < n$ forces a closed empty pair with sides of at least the forced sizes	`subset_compl_closedNbhd_iff`, `subset_closeStep`, `closeStep_isClosedPair`, `closeStep_closeStep`, `sdiff_eq_inter_boundary_of_isClosedPair`, `card_closeStep`, `isClosedPair_iff_sees_both`, `exists_closedPair_of_pathwidth_le`
A.66	The weighted count (Appendix E.5): for any weight $1 \leq Z(P) \leq \#\{\text{empty pairs}\}$ on the empty pairs, $\sum_P \mathbf{1}[P \text{ empty}]/Z(P) \geq \mathbf{1}[\text{some pair is empty}]$, so the $G(n,p)$-weighted sum of the graphs of pathwidth at most k is at most $E[\sum_P \mathbf{1}[P \text{ empty}]/Z(G,P)]$ over the exact-size pairs; with Z the number of flips of P (exchanging equally many vertices isolated in $G[A]$ and $G[R]$ gives distinct empty pairs of the same sizes), the flip-weighted count	`weightedCount_ge_indicator`, `weightedSum_graphsOfPathwidthLe_le_weighted`, `flipPair_isEmptyPair`, `flipPair_injOn`, `card_flipPair`, `card_flips_le`, `weightedSum_graphsOfPathwidthLe_le_flipWeighted`

Entry	Result in the paper	Lean names
A.67	Theorem E.6: for $0 \le p \le 1$ and $k < n$, the $G(n,p)$-weighted sum of the graphs of pathwidth at most k is at most $\binom{n}{k}$ times the $G(N,p)$-weighted count, $N = n-k$, of the graphs on $V \setminus S$ whose vertices split into an empty pair of sizes $\lceil N/2 \rceil$, $\lfloor N/2 \rfloor$; the union bound over the $\binom{n}{k}$ separators is the only inequality: the graphs on V with a given restriction to $V \setminus S$ have total weight equal to the restriction's weight (the pairs meeting S carry weight $(p+(1-p))^{\lvert\text{pairs meeting } S\rvert} = 1$), and the inner count is the same for every S by a relabelling of V	`weightedSum_graphsOfPathwidthLe_le_choose_mul_innerBip`, `weightedSum_graphsOfPathwidthLe_le_separator`, `weightedSum_graphsOfPathwidthLe_le_sum_separators`, `sum_gnpWeight_fiber`, `sum_gnpWeight_bipGraphs`, `card_pairsOff`, `innerBip_eq_of_card_eq`
A.68	The closed-pair count (Appendix E.5), the union bound: the weighted sum of the graphs of pathwidth at most k is at most $\sum_{(A,R)} \Pr[(A,R) \text{ is closed}]$ over the ordered disjoint pairs with $\lvert A\rvert \ge \lceil (n-k)/2 \rceil$, $\lvert R\rvert \ge \lfloor (n-k)/2 \rfloor$; each term evaluated in the next row	`weightedSum_graphsOfPathwidthLe_le_sum_closedPairs`
A.69	The closed-pair count, the independence step: the p-weight of the subsets of a set of pairs that satisfy one predicate on each of pairwise disjoint blocks is the product of the per-block weights (the pairs in no block are free), so the $G(n,p)$ probability that (A,R) is closed is exactly $(1-p)^{\lvert A\rvert\lvert R\rvert}((1-(1-p)^{\lvert A\rvert})(1-(1-p)^{\lvert R\rvert}))^{\lvert S\rvert}$ (one block of the forbidden A–R pairs, two star blocks per $v \in S$); hence $\Pr[\text{pw} \le k] \le \sum_{a \ge t,\, r' \ge r} \binom{n}{a}\binom{n-a}{r'}(1-p)^{ar'}((1-(1-p)^{a})(1-(1-p)^{r'}))^{n-a-r'}$, the expected number of closed pairs of at least the forced sizes	`sum_subsetWeight_split`, `sum_subsetWeight_blocks`, `sum_gnpWeight_closedPairGraphs`, `weightedSum_graphsOfPathwidthLe_le_closedPairCount`, `weightedSum_graphsOfPathwidthLe_le_closedPairFormula`
A.70	Theorem 3.2 of Edwards and McDermid (2015), the reduction theorem (Appendix E): every graph with every degree 0 or at least 3 has a vertex set X such that $G - X$ reduces to a non-empty basic graph B with $\lvert X\rvert + f^-(B) \le f^-(G)$; unconditional, with the step inequality in all four cases and a local commutation lemma in place of the appeal to the uniqueness of the reduced graph	`exists_reduction_set`, `one_add_potentialMinus_le`, `SPReduces.isolateSet_comm`, `potential_isolate`

Entry	Result in the paper	Lean names
A.71	Lemma 5.1 and Corollary 5.2 of Edwards and McDermid (2015) (Appendix E): $\mathrm{pw}(G) \le \sum_v g(d_G(v)) + \tau(n) + 3\rho(n) + 1$ for every G, conditional on two cited results stated as named propositions and never asserted — the cubic base case `CubicPathwidthBound` with budget τ (Fomin and Høie, 2006, Theorem 5) and the reduction lemma `ReductionPathwidthBound` with budget ρ (Kneis et al., 2009) — and on τ monotone and non-negative; the paper gives no constants	`pathwidth_le_histogram_of_hypotheses`, `pathwidth_le_degreeSum_of_hypotheses`, `pathwidth_le_potentialMinus_of_hypotheses`, `pathwidth_basic_le`, `CubicPathwidthBound`, `ReductionPathwidthBound`
A.72	Lemma 4.10, the premises as commitment conditions	`isDefinite_iff_endpoint`, `isHereditarilyDefinite_iff_isCommittable`
A.73	In Counterexample 4.4 the endpoint test passes and the commitment fails	`cex_isDefinite_not_isCommittable`
A.74	The published premise gives a commitment to the least-border set	`exists_isCommittable_of_isDefinite`
A.75	In Counterexample 4.4 that set is $\{2, 3, 4\}$	`cex_isCommittable_234`
A.76	A commitment of depth one holds iff $\mathrm{open}(q, S) \le 1$	`isCommittable_insert_iff`
A.77	The greedy step of Coudert, Mazauric and Nisse is that condition	`isGreedyStep_iff`
Section 5, computational results		
A.78	The split of the root into independent tasks is sound	`execSplit_repairedFullFilter_sound`, `execSplit_repairedFullFilter_mospValue`
Appendix B, supporting material		
A.79	Node search equalities need an edge (Appendix B.2)	`nodeSearch_ne_intervalThickness_of_edgeless`, `monotoneNodeSearch_ne_vertexSeparation_add_one_of_edgeless`
A.80	Lengauer's Theorem 4 fails at $k = 0$ (Appendix B.2)	`isPositiveVSG_triangleGraph_counterexample`
A.81	$t = \mathrm{ns}$ fails on the matrix $[1]$ (Appendix B.2)	`tracks_ne_monotoneNodeSearch_one`
A.82	The Kirousis–Papadimitriou claim fails on $K_{1,3}$ (Appendix B.2)	`kirousisPapadimitriou_claim2_false`
A.83	Progressive pebbling is in the exact core (Appendix B.3)	`pbw_lengauerD_eq_pathwidth`, `pbw_eq_mospValue_pebbleMatrix`, `mpb_eq_pathwidth_add_one`
A.84	The vertex search is in the exact core (Appendix B.3)	`vertexSearchNumber_eq_pathwidth_add_one`, `monotoneVertexSearchNumber_eq_pathwidth_add_one`, `isNodeSearch_of_isSuccessfulVertexSearch`, `vertexSearchNumber_eq_nodeSearch`, `vertexSearchNumber_of_isEmpty`

Entry	Result in the paper	Lean names
A.85	The maximum wavefront is in the exact core (Appendix B.3)	`card_wavefront`, `maxWavefront_eq_vertexSepOfLayout_add_one`, `minMaxWavefront_eq_pathwidth_add_one`, `minMaxWavefront_eq_narrowness`, `minMaxWavefront_of_isEmpty`
A.86	Proposition B.1, the corrected filter is node-sound	`repairedFullFilter_sound`
A.87	Theorem B.2, runs are sound	`Exec.sound`
A.88	Appendix D, the SAT formulation is faithful (clauses 1–4)	`encodes_iff_mospValue_le`
A.89	Theorem F.3, a blockage of order n forces pathwidth $\geq n$ (Appendix F); the definitions and Lemma (2.2) of Bienstock et al. (1991)	`Blockage.IsBlockage`, `Blockage.IsBlockage.mem_of_mem`, `Blockage.IsBlockage.empty_mem_and_univ_not_mem`, `Blockage.att_prefixUnion_subset`, `Blockage.prefixUnion_succ_sdiff_subset`, `Blockage.pathwidth_ge_of_isBlockage`
A.90	Lemma F.2 and the three frame operators of Appendix F: a blockage is an avoidance family; a clique of $n+1$ vertices, the pullback along a minor model, and three linked branches of order k give blockages of order n, n and $k+1$; a frame on V forces pw $\geq n$	`Blockage.avoid`, `Blockage.Frame`, `Blockage.isBlockageIn_clique`, `Blockage.isBlockageIn_minor`, `Blockage.isBlockageIn_branch`, `Blockage.Frame.pathwidth_ge`, `Blockage.pathwidth_ge_of_clique_frame`, `Blockage.pathwidth_ge_of_branch_frames`
A.91	Theorems G.1, G.2 and G.3, the tabu search's certificates (Appendix G)	`MinorTabu.contractEdge_quotient`, `MinorTabu.isContractionSeq_ufSeq_id`, `MinorTabu.applySeq_ufSeq_id`, `MinorTabu.tabuMinor_eq`, `MinorTabu.pathwidth_tabuMinor_le`, `MinorTabu.floor_sound_of_refutation`, `MinorTabu.floor_sound_of_degeneracy`, `MinorTabu.degeneracy_tabuMinor_le`, `MinorTabu.Justified.le_pathwidth`, `MinorTabu.tabu_output_le_pathwidth`, `MinorTabu.tabu_output_ge_of_visited_refutation`, `MinorTabu.exists_visited_of_le_output`, `MinorTabu.output_eq_pathwidth_iff`, `MOSPInstance.floor_sound_of_refutation_mosp`, `MOSPInstance.tabu_output_lt_mospValue`

Entry	Result in the paper	Lean names
A.92	A checked lower-bound certificate implies the bound (Section 5.5): an UNSAT expansion level (t, b) with $t \leq n$ gives $b + 1 \leq t +$ pw (Harper's bound, contrapositive; the hypothesis $t \leq n$ is necessary, one-vertex counterexample); a legal contraction sequence whose minor has minimum degree d gives $d \leq$ pw, in the `minDegreeIn` form and in the degree form on a nonempty minor (necessary, likewise); the `Bool` replay `checkSeq` is sound and complete for legal sequences; both in MOSP units	`pathwidth_ge_of_forall_closedNbhd`, `pathwidth_ge_of_unsat`, `pathwidth_ge_of_forall_closedNbhd_needs_le`, `mospValue_ge_of_unsat`, `checkSeq_sound`, `checkSeq_complete`, `checkSeq_iff`, `pathwidth_ge_of_minor_minDegree`, `pathwidth_ge_of_checkSeq`, `degree_minorOf_eq`, `pathwidth_ge_of_minor_forall_degree`, `pathwidth_ge_of_minor_forall_degree_needs_nonempty`, `mospValue_ge_of_minor_minDegree`
A.93	Three certificates closed by `decide`: the 8-cycle contracted to a triangle ($2 \leq$ pw), $K_{3,4}$ at level one over all 128 subsets ($3 \leq$ pw), and the record instance *Warwick 1729* (30 customers, 90 requirements), whose 22-step contraction sequence to K_8 is replayed on the MOSP graph Lean builds from the matrix: every pattern order opens at least 8 stacks (optimum 9)	`two_le_pathwidth_octGraph`, `three_le_pathwidth_k34`, `seven_le_pathwidth_warwick`, `eight_le_mospValue_warwick`

Appendix B Supporting material

This appendix holds the material that supports Sections 2, 4 and 5 without being needed to follow them.

B.1 Formal definitions of eight problems

These are the formal definitions of the eight problems that Table 2.2 states in one line. As in Section 2.1, each is a decision problem with an integer k, and its value is the least k for which the answer is yes.

One-dimensional logic. Given gates, nets and the nets each gate connects, is there an order of the gates whose net intervals have an interval graph of clique number at most k? Its *net graph* H joins two nets that share a gate. A *boundary* variant pins two gates to the ends (Ohtsuki et al., 1979, §§II–IV).

PLA folding $\mathrm{pla}(M)$**.** As gate matrix layout, with at most two nets per track. With no such bound (*multiple folding*) it is gate matrix layout itself (Möhring, 1990, pp. 25, 36).

Interval thickness $\theta(G)$**.** Is there an interval supergraph of G on the same vertices with clique number at most k? (Kirousis and Papadimitriou, 1985, p. 182); (Möhring, 1990, p. 31).

Node search $\mathrm{ns}(G)$**.** Can searchers, placed on and removed from vertices, clear every edge with at most k on the graph at once? An edge is cleared when both ends are guarded, and a clear edge is recontaminated by a searcher-free path to a contaminated one. $\mathrm{mns}(G)$ is the same over strategies that never recontaminate (Kirousis and Papadimitriou, 1985, p. 181); (Kirousis and Papadimitriou, 1986, §2).

Edge search $\mathrm{es}(G)$**.** The same game with a third move, sliding a searcher along an edge, which clears that edge; $\mathrm{pes}(G)$ for progressive strategies, which never recontaminate an edge (Kirousis and Papadimitriou, 1986, p. 208); (Ellis et al., 1994, p. 53).

Narrowness $\nu(G)$**.** Is there an order $v_1, \ldots, v_n$ such that the following process never holds more than k vertices: at step i add v_i, then remove every v_j, $j \leq i$, with no neighbour v_ℓ, $\ell > i$? (Kornai and Tuza, 1992, §2).

Split bandwidth $\mathrm{sb}(G)$**.** Is there a graph obtained from G by repeatedly splitting a vertex into an edge, its neighbours partitioned between the two ends, with bandwidth at most k? (Fomin, 1998, §3.2).

Edge separation. The Linhares–Yanasse table cites Lengauer (1981), which supports three readings: cutwidth $\mathrm{cw}(G)$, the least over layouts of the most edges crossing a gap (p. 468); modified cutwidth $\mathrm{mcw}(G)$, counting only edges that pass strictly over a vertex (Definition 6); and Lengauer's vertex separator game $\mathrm{VSG}(G)$, the least k such that the vertices can be pebbled in an order that leaves at most k unpebbled vertices adjacent to pebbled ones (p. 467).

B.2 Edge cases and a gap in a published proof

Three of the published equalities fail on degenerate inputs, and each failure is proved in Lean. On a nonempty graph with no edges, $\mathrm{ns} = 0$ while $\mathrm{vs} + 1 = \theta = 1$, so both equalities of Kirousis and Papadimitriou (1985, 1986) need an edge (Appendix A, A.79). For a matrix with no 1s, $Z = 0$ while $\mathrm{pw} + 1 = 1$. Lengauer's Theorem 4 fails at $k = 0$ on an edgeless graph (Appendix A, A.80). Möhring (1990, Thm 3.9), $t = \mathrm{ns}$, fails literally on the 1×1 matrix $[1]$ (Appendix A, A.81).

The published proof of $\mathrm{vs} \le \mathrm{ns} - 1$ (Kirousis and Papadimitriou, 1986, Thm 4.1) has a gap. It orders the vertices by when they first receive a searcher and claims that every vertex counted at the worst cut still holds a searcher, "since the vertices in D_{i_0} are joined to vertices not yet cleared" (p. 217). A strategy may place a searcher and remove it again before clearing anything, which recontaminates nothing. On $K_{1,3}$, placing and removing a searcher on each leaf before guarding the centre is an optimal recontamination-free strategy for which the claim fails (Appendix A, A.82). The theorem is true. The correction orders the vertices by clearing time, as our Lean proof does, or uses the paper's own normal form, its Corollary 2.4.

B.3 Members outside the table

Progressive black-white pebbling, which is not in the Linhares–Yanasse table, belongs to the exact core. Write pbw for its value and mpb for the minimum progressive pebbling number of Kirousis and Papadimitriou (1986). Lengauer (1981, Thm 3) gives $\mathrm{vs}(G) \le k \iff \mathrm{pbw}(G_d) \le k + 2$ on the dag G_d built from G. His Theorem 2 gives $\mathrm{pbw}(D) = \mathrm{vs}(D_u) + 1$ on a dag D, where D_u is the MOSP graph of the matrix whose column v is v with its predecessors. And $\mathrm{mpb} = \mathrm{pw} + 1$ (Kirousis and Papadimitriou, 1986, Thm 3.1). All three are proved in Lean, each with its hypothesis: Lengauer's Theorem 2 in its instance form fails at $k = 1$ on a dag with no arcs, and $\mathrm{mpb} = \mathrm{ns}$ fails on edgeless graphs (Appendix A, A.83).

The vertex search. Bienstock et al. (1991, §5) define a search as a sequence of vertex sets $X_1 = \emptyset, \ldots, X_m$, each a subset or a superset of the one before; the fugitive may be at any vertex joined to its previous region by a path avoiding the current set, and the search succeeds when no such vertex remains. Their (5.1) states that a successful search with every $|X_i| \le n$ exists if and only if $\mathrm{pw}(G) \le n - 1$, so the least n is $\mathrm{pw}(G) + 1$ on every graph with a vertex; on an edgeless graph one searcher is needed, where node search needs none. We prove it in Lean (Appendix A, A.84), close to their proof. The upper bound is their construction: from a path decomposition $W_0, \ldots, W_L$ of least width, the search $\emptyset, W_0, W_0 \cap W_1, W_1, \ldots, W_L$ is monotone and successful, because after W_j, and again after $W_j \cap W_{j+1}$, the fugitive's region is exactly the set of vertices in no bag up to W_j. The lower bound reduces to node search: playing each change of position as single removals and then single placements turns a successful vertex search with at most n searchers into a successful node search with at most n searchers, since every contaminated edge keeps an endpoint in the fugitive's region. With $\mathrm{ns} = \mathrm{pw} + 1$ on graphs with an edge (Theorem 2.4), and $\mathrm{pw} = 0$ on edgeless graphs, this gives $\mathrm{pw}(G) + 1$. A graph with no vertices needs no searcher. A brute force over all 1,252 graphs with at most seven vertices agrees. Where our proof departs from theirs is the lower bound: they derive it from blockages, their (2.1), a min-max theorem for pathwidth, while the formal proof replays the search as a node search and uses the monotonicity of node search proved for Table 1.1's node-search row.

The maximum wavefront. For a symmetric $n \times n$ matrix A with nonzero diagonal, Kumfert and Pothen (1997, §2.1) call row k active at step i if $k \geq i$ and $a_{kl} \neq 0$ for some $l \leq i$; the ith wavefront is the set of active rows, and $\max_i |\mathrm{wf}_i(A)|$ is the storage the frontal method needs. In the graph G of A, $\mathrm{wf}_i = \{v_i\} \cup (N(V_i) \setminus V_i)$ with $V_i = \{v_1, \ldots, v_i\}$, so $|\mathrm{wf}_i|$ is one more than the number of later vertices with a neighbour among the first i. Hence for each order the maximum wavefront is the vertex separation of that order plus one, and the least over symmetric orderings is $\mathrm{vs}(G) + 1 = \mathrm{pw}(G) + 1$ when $n \geq 1$; an empty matrix gives 0 (Appendix A, A.85). The set wf_i is the shack of narrowness in its out-sequence form, so the least maximum wavefront is also $\nu(G)$. The source defines the wavefront but does not state the equality. A brute force over all 1,252 graphs with at most seven vertices and every ordering, 5.4 million pairs, agrees.

B.4 Why the published proof of the definite move fails

The proof moves q to the front of a solution and claims that after each prefix "we have at most open(q, S) extra stacks open, but at least close(q, S) extra stacks closed" (p. 6). At a prefix $T \supseteq S$ that does not yet contain q, moving q forward changes the open stacks by $b(T \cup X) - b(T) = |N[q] \setminus O(T)| - |X \setminus T|$. At $T = S$ this is open $-$ close ≤ 0. But customers of X that the solution has *already* closed are in T and no longer count as extra stacks closed, while the stacks q opens may still be new. In the example, a solution may begin with 3, after which 4 is free: at $T = \{2, 3, 4\}$, moving 0 forward costs one more stack. A second, independent slip: even where the conclusion holds, the literal sequence $S +\!\!+ [q] +\!\!+ R'$ need not be a solution, because a customer that q frees stays open until its turn; the argument needs the free moves.

B.5 The search for smaller counterexamples

The smallest graph on which the definite move of Chu and Stuckey's algorithm, as published (Chu and Stuckey, 2009, Theorem 1), loses the last solution has 14 customers: there is none on any graph to 9 vertices, by exhaustion, and none on the 88,802 connected graphs on 10 vertices with at most 14 edges. The counterexample was found by a search over a family of gadgets built from the failed proof, and minimised by deletion.

B.6 The search for prior reports of the error

We looked for any earlier report of the error in the definite move. We listed the works citing Chu and Stuckey (2009), Chu's thesis and its IJCAI 2013 abstract in OpenAlex, Semantic Scholar and OpenCitations: 141 works, 66 of which we read in full. No work read gives a counterexample, an erratum or a correction. One, Fink (2012), restates the definite move as true with a stronger premise, refuted in Section 4.3. Six use an implementation of the rules and report no failure: De Giovanni et al. (2010, 2013a) ran the original code, with its authors' help, on all 200 of their random instances, on 5,841 instances in all and on 150 larger ones of their own, De Giovanni et al. (2013b) used it to certify the optimum number of open stacks on every instance of their connection-cost benchmark, and Frinhani et al. (2018) ran it to obtain the optima that Martin et al. (2022) reuse.

Algorithm B.1 The customer search (Chu and Stuckey, 2009) is a plain depth-first search; all its pruning sits in the memo (lines 4 and 12), the filter (line 6) and the old moves (line 9). It decides $\mathrm{Sol}_k(S)$ for a graph $G = (C, E)$ and a stack bound k, and is called as $\textsc{Search}(\emptyset, \emptyset)$. Q is the set of old moves the node inherits.

```
1  SEARCH(S, Q):
2    S ← cl(S)                                          free moves (Lemma 4.1)
3    if S = C then return true
4    if S is in the memo then return false              memo
5    A ← the c ∉ S ∪ Q with cost(S, c) ≤ k, in index order   cost cut
6    L ← filter of A: definite move, then subset rule, then better move   Sections 4.3–4.4
7    seen ← Q \ S
8    for each c ∈ L:
9      Q' ← {q ∈ seen : cost(S ∪ {q}, c) ≤ k}           old moves
10     if SEARCH(S ∪ {c}, Q') then return true
11     seen ← seen ∪ {c}
12   add S to the memo
13   return false
```

B.7 The customer search, as published

Algorithm B.1 is Chu and Stuckey's customer search as Section 4.2 describes it, with the ingredients named as that section names them.

B.8 Implementation checks

The theorem is about the search as specified; that our C and Python implementations run it is checked, not proved. Three checks link them:

- a transcription of the Lean filter agreed with the implemented filter on 16,244,090 node checks over graphs on 1 to 17 vertices;
- the matching agreed with the hereditary premise on 36,939,226 candidates;
- a search written from the theorems agreed with both implementations and an exact oracle on 17,431,232 runs over 52,592 graphs on 1 to 17 vertices.

Section 5.5 adds a certificate checker for individual refutations.

B.9 The filter and the runs are sound

Proposition B.1. *At every state with the invariant, for every set Q of refuted old moves, the filter definite → subset → better with both corrections is node-sound (Appendix A, A.86).*

Proof. If the definite move fires, Theorem 4.6 applies. Otherwise the subset rule keeps a survivor with a solution; let u be the earliest. If the better move dropped u, it cited an earlier survivor that the corrected argument proves solvable, contradicting the choice of u. □

Theorem B.2 (runs are sound)**.** *Let the filter be node-sound and return only playable customers. Suppose that at the start of a run the invariant holds, every memo entry is a genuine refutation, and every old move is refuted. Then the run leaves only genuine memo entries, and if it answers* false *at S, then $\neg\mathrm{Sol}_k(S)$. This holds for any child order and any memo policy, with the old move on or off (Appendix A, A.87).*

The proof is an induction over the run, in the order in which subtrees complete. Every old move a node holds is a sibling subtree that completed with *false* earlier in the same run, or an inherited one that passed the reinsertion test.

B.10 The split re-certification

Table B.1 gives the seven hardest re-certifications of Section 5.2 and the two values never certified before, both now certified by the same split. Each instance's tree was expanded at the top into independent subtrees (tasks), and each task was refuted by one worker. The seven re-certifications ended with every task refuted and the value unchanged. No task of any instance has found an order of the smaller value.

Table B.1: All seven hardest values re-certify unchanged, at 18 to 369 core-hours each. The two values never certified before, 21 for `Random-125-125-2-3_0` and 25 for `Random-125-125-2-2_0`, are now certified. *Value*: the stored optimum; k: the value minus one, which must be refuted.

Instance	Value	k	State	Tasks refuted	Nodes	Core-hours
`Random-125-125-4-1_0`	57	56	refuted	669	4.97×10^{10}	18.1
`Random-125-125-2-4_0`	24	23	refuted	1,686	4.98×10^{11}	154.1
`Random-125-125-4-5_0`	46	45	refuted	616	7.96×10^{10}	26.1
`Random-125-125-4-2_0`	57	56	refuted	842	7.64×10^{10}	30.2
`Random-125-125-2-1_0`	24	23	refuted	4,096	1.30×10^{12}	366.5
`Random-125-125-2-5_0`	20	19	refuted	4,168	1.28×10^{12}	368.8
`Random-125-125-4-4_0`	51	50	refuted	2,504	4.12×10^{11}	148.8
`Random-125-125-2-3_0`	21	20	refuted	9,665	4.25×10^{12}	1127.8
`Random-125-125-2-2_0`	25	24	refuted	17,001	7.38×10^{12}	2230.6

Appendix C The bibliometric study

This appendix gives the method behind Section 3 in full, the counts per name and when each name was in use.

Search. We measured how much each name is used with OpenAlex (Priem et al., 2022), on 29 September 2026. For each of the twelve problems we searched titles and abstracts for its name and its common variants, and restricted the search to the four fields in which the problems are studied: Computer Science, Mathematics, Engineering and Decision Sciences. Without that restriction several names are ordinary words: "narrowness" returns over a million works, and "edge separation" returns aerodynamics.

Relevance. Inside those fields a search hit is still not a work that uses the name. OpenAlex's search stems words and ignores punctuation, so a quoted phrase matches more than the phrase. "Narrowness" matches "narrowing" in term rewriting; "node searching" matches look-ups in peer-to-peer networks; "split bandwidth" matches radar interferometry and 5G spectrum assignment; "path-width" matches the widths of roads, tool paths and datapaths. We therefore fetched all 2,271 hits with their titles, abstracts, topics and venues, and kept a work only if it is about the problem in the sense of the Linhares–Yanasse table. For the eleven smaller names the 700 hits were judged one by one. For pathwidth, the 1,571 hits were filtered by a rule: keep a work that uses the one-word spelling, that OpenAlex places in a theory subfield, or that mentions graphs, treewidth or minors and not roads, vehicles, trajectories or datapaths. We checked the rule by reading samples of what it kept and what it dropped. Of the 2,271 hits, 1,593 are relevant: 1,213 for pathwidth and 380 for the other eleven names. The two search-game names keep 27% and 40% of their hits.

Robustness. The counts are lower bounds. A work can study a problem without naming it in its abstract, OpenAlex has few abstracts before about 1990, and its citation counts run below Google Scholar's. The judgements were made by one annotator with model assistance, record a yes or no per work and not the reason, and have not been checked by a second annotator. As a second reading we applied a mechanical rule: the literal phrase with a context term and no exclusion term. It agrees with the judgements on 543 of the 700 judged works (78%). Under both readings pathwidth has more than three times as many works as the other eleven names together (3.2 and 3.6 times), the three empty names stay empty apart from one 5G false positive, and the search games lose most of their hits. MOSP's rank over all years is the one conclusion that depends on the reading: fourth under the judgements, fifth under the rule.

Counts and citations. Table C.1 gives the counts per name. Citation counts do not track name usage. Kirousis and Papadimitriou (1986) is the most-cited paper in the table (293 citations), while only 38 works use "edge search" in its sense. The paper is cited as a foundational result on graph searching, not because people work on the edge search game under that name. The split is starker for Ohtsuki et al. (1979): 108 citations against 11 works using "one-dimensional logic". Split bandwidth is Fomin's own term in the paper the table cites (Fomin, 1998), and no one adopted it.

Table C.1: Citations do not track usage: the most-cited papers are cited far more often than their problem's name is used. Works that use each problem's name in the sense of the Linhares–Yanasse table, and citations of the paper the table cites for it. *Relevant*: hits kept by the relevance check. *Citations*: OpenAlex `cited_by_count`, 29 September 2026. Kashiwabara and Fujisawa (1979) has no OpenAlex record. [a]OpenAlex returned one pathwidth record twice: the counts are of records, as Figure 3.1 draws them, and there are 1,212 distinct relevant pathwidth works.

Problem	Discipline	Relevant	Hits	Reference in the 2002 table	Citations
Pathwidth	graph theory	1,213[a]	1,571	Kinnersley (1992)	214
Gate matrix layout	VLSI	110	125	Möhring (1990); Wing et al. (1985)	134 / 89
PLA folding	VLSI	68	74	Möhring (1990)	134
MOSP	operations research	58	60	Yanasse (1997b); Fink and Voß (1999)	75 / 71
Vertex separation	graph theory	55	100	Kinnersley (1992)	214
Edge search	graph theory	38	95	Kirousis and Papadimitriou (1986)	293
Node search	graph theory	34	127	Kirousis and Papadimitriou (1985)	124
One-dimensional logic	VLSI	11	18	Ohtsuki et al. (1979)	108
Interval thickness	graph theory	6	10	Kashiwabara and Fujisawa (1979)	–
Narrowness	graph theory	0	35	Kornai and Tuza (1992)	44
Split bandwidth	graph theory	0	35	Fomin (1998)	19
Edge separation	graph theory	0	21	Lengauer (1981)	77
All twelve		1,593	2,271		

When each name was in use. Figure C.1 shows the counts over time. PLA folding and gate matrix layout peak in 1980–1989 and fall to almost nothing after 2010, and one-dimensional logic has no relevant work after 2009. MOSP first appears in the 1990s, peaks in 2005–2014 and is still active. Pathwidth's rate has roughly doubled since the 1990s, from about 15 to 27 works per million, and its 311 relevant works in 2020–2024 are more than the whole history of any other name. Rows with a few works per period move by whole cells on one paper, and should be read as anecdote rather than trend.

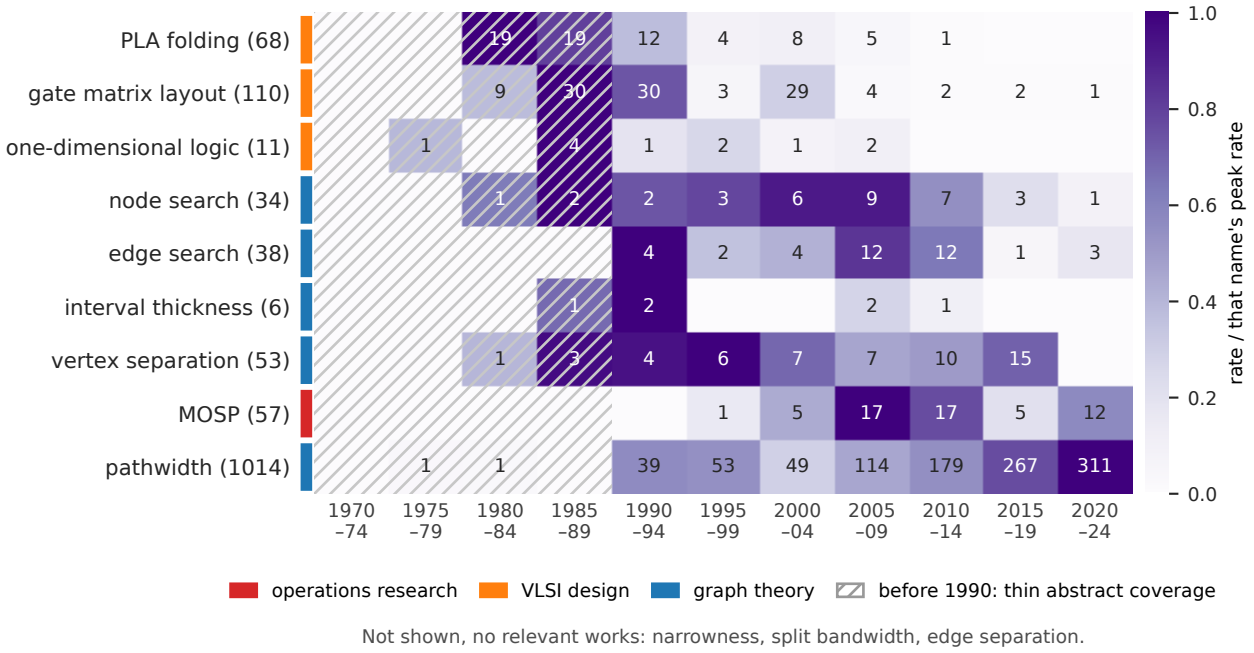


Figure C.1: The VLSI names belong to the 1980s, the search games peak later and fade, MOSP peaks in 2005–2014, and only pathwidth is still rising. The shade is the name's rate, relevant works per million works in the same four fields in each five-year period, as a fraction of that name's own peak rate; the numbers are raw counts. Rows are ordered by the period of their peak. Counts cover 1970–2024, since 2025–2026 is not fully indexed, so row totals are below those of Table C.1. Before 1990 (hatched) OpenAlex has few abstracts.

Appendix D A SAT formulation of MOSP

Our SAT formulation encodes the question "is $Z(M) \le k$?", for a matrix M with customers C (rows) and patterns P (columns), $m = |P|$, as a formula in conjunctive normal form, and hands it to a SAT solver; Knuth (2015) covers the solvers and the encodings used here. It shares no search code and no pruning rule with the customer search of Section 4, so it is an independent check on that search's answers. A satisfying assignment gives a production order that can be verified by simulation. A refutation can be logged as a DRAT proof and checked by an independent checker (drat-trim). Write P_c for the patterns of customer c, and positions $t = 0, \ldots, m-1$.

Variables. $x_{p,t}$: pattern p is cut at position t. $y_{p,t}$: pattern p is cut at or before position t. $o_{c,t}$: the stack of customer c is open at position t. Two auxiliaries per customer and position, $\alpha_{c,t}$ (some pattern of c has been cut) and $\omega_{c,t}$ (all patterns of c have been cut).

Clauses.

1. *Permutation.* Every pattern takes at least one position and every position at least one pattern, $\bigvee_t x_{p,t}$ and $\bigvee_p x_{p,t}$; and at most one of each, by a ladder encoding, shown below.
2. *Prefixes.* $x_{p,t} \Rightarrow y_{p,t}$; $y_{p,t} \Rightarrow y_{p,t+1}$; $y_{p,t} \Rightarrow y_{p,t-1} \vee x_{p,t}$ for $t > 0$; and $y_{p,0} \Rightarrow x_{p,0}$.
3. *Open stacks.* A stack is open at t if one of its patterns is cut at t, $x_{p,t} \Rightarrow o_{c,t}$ for $p \in P_c$, or if some of its patterns are cut by t and some are not: $y_{p,t} \Rightarrow \alpha_{c,t}$ and $\omega_{c,t} \Rightarrow y_{p,t}$ for $p \in P_c$, and $\alpha_{c,t} \wedge \neg\omega_{c,t} \Rightarrow o_{c,t}$. These are $2|P_c| + 1$ clauses per customer and position instead of the $|P_c|(|P_c| - 1)$ of the direct pairwise form. Only these polarities are needed: no solution gains by setting $o_{c,t}$ true without cause.
4. *Width.* At most k of the $o_{c,t}$ are true at each position t, by a totalizer.
5. *Symmetry.* Reversing a production order leaves the number of open stacks at each step unchanged (Yanasse, 1997a, Proposition 1), so the pattern with the most customers may be placed in the first $\lfloor m/2 \rfloor + 1$ positions.

The ladder. To say that at most one of $z_1, \ldots, z_n$ is true, the ladder introduces auxiliaries $r_1, \ldots, r_n$, where r_i reads "the true literal, if any, comes after position i", and one slack literal s for "none is true". Its clauses are

$$r_{i+1} \Rightarrow r_i \ (1 \le i < n), \qquad z_1 \Leftrightarrow \neg r_1, \qquad z_i \Leftrightarrow r_{i-1} \wedge \neg r_i \ (1 < i \le n), \qquad s \Leftrightarrow r_n.$$

The first family makes $r_1, \ldots, r_n$ a ladder, true up to some point and false after it, so it has exactly one step; the channelling clauses make each z_i true exactly when the step is at i, and s true when there is no step. So at most one z_i is true, with $n + 1$ auxiliaries and about $4n$ clauses, linear in n, where the pairwise form needs $n(n-1)/2$ clauses. Our encoder emits this form through PySAT's `EncType.ladder`.

What is proved. The formula with clauses 1–4 is satisfiable exactly when some production order keeps at most k stacks open (Appendix A, A.88). The proof models the two cardinality constraints by their meaning, not by the clauses the ladder and totalizer encodings emit; both are standard and independent of MOSP. The symmetry clause 5 is justified by the reversal argument above but is not in the Lean model. That the encoder emits exactly these clauses is a reading of its source, not a proof.

On the instance of Figure 4.2. The formula has 31,598 variables and 109,923 clauses. Kissat 4.0.4 answers "satisfiable" at $k = 7$ in 9 s and at $k = 6$ in 56 s, with an order whose peak simulates to 6, and "unsatisfiable" at $k = 5$ in 158 s. CaDiCaL 1.9.5 refutes $k = 5$ in 53 s with a DRAT proof of 1.4 million lemmas (112 MB compressed), which drat-trim verifies in 44 s. So the optimum is 6, independently of both versions of the customer search, with a refutation that a third party can check. The repository holds the proof trimmed by drat-trim to the lemmas the refutation uses (27 MB compressed) and a script that rebuilds the formula from the instance and checks the proof in about 35 s (`paper1/data/false_refutation/drat/`).

Appendix E The pathwidth of sparse random graphs

The equivalences of Section 2 make every statement about the pathwidth of a graph a statement about the open stacks of an instance. This appendix proves one such statement for random graphs, in plain language, and records what of it is checked in Lean. The result is that a sparse random graph has pathwidth proportional to its number of vertices, with an explicit constant. The argument is a first moment, an old tool; what is new here is the exact form of its deterministic step, the constant it yields, and the machine check. We found no prior statement of the constant.

E.1 The picture

Everything in this appendix is one picture, so we draw it first, in the words of the factory and then in the words of Section 2.

Take any instance and any production sequence, and freeze it at some moment. Every customer is then in one of three states. *Done*: the stack has closed, every pattern the customer needs has been made. *Open*: not done, but at least one of the customer's patterns has been made, so the stack is open. *Untouched*: none of the customer's patterns has been made. The number of open stacks at that moment is the size of the open group, and a sequence of value $k+1$ never has more than k customers open at once. A done customer and an untouched customer can share no pattern: a shared pattern would already have been made for the done customer and would have opened the untouched one. So at every moment the done group and the untouched group are *strangers*, with no pattern in common, and only the open customers connect them.

In the MOSP graph, where two customers are joined when they share a pattern, the same picture reads as follows. A production sequence is a line of customers in the order their stacks close. At a cut of the line, the done customers are the prefix and the open customers are the suffix vertices with a neighbour in the prefix; their number is what *vertex separation* counts at that cut (Theorem 2.2: the least over all lines of the largest such count is the pathwidth, which is the optimum minus one). Two things follow at every cut of a line of vertex separation k, and they are the two objects the proofs count.

- An *empty pair*: two disjoint sets of vertices with no edge between them, here the prefix and the untouched suffix. Since at most k suffix vertices are open, the two sets together hold at least $n-k$ vertices, and at the cut nearest the middle each holds about $(n-k)/2$.
- A *balanced separator*: a set S of at most k vertices, here the open customers, whose removal leaves the rest in two sides of about $(n-k)/2$ vertices each with no edge between the sides.

Both are *necessary* for small pathwidth, not sufficient: a graph that has them may still need many stacks, because a line needs a small separator at every cut, not at one. That direction is exactly what a lower bound needs. If a random instance almost never contains an empty pair of the required sizes, it almost never has pathwidth k or less. The appendix counts empty pairs; the balanced separator is the object a sharper count would use, and Appendix E.5 says why.

E.2 The statement

$G(n,p)$ is the random graph on n labelled vertices in which each of the $\binom{n}{2}$ possible edges is present with probability p, independently. We take $p = c/n$, so that a vertex has about c neigh-

bours. Write pw for pathwidth, ln for the natural logarithm, and

$$\phi(x) = -x \ln x \quad (\phi(0) = 0), \qquad H(x,y,z) = \phi(x) + \phi(y) + \phi(z)$$

for the entropy of a probability vector with three entries.

Theorem E.1 (first moment, finite form)**.** *Let $c > 0$ and $0 < \beta < 1$, and put $a = (1-\beta)/2$ and*

$$\delta(\beta, c) \;=\; c\,a^2 \;-\; H(a, a, 1-2a).$$

For every integer $n \geq \max(3, c)$,

$$\Pr\big[\mathrm{pw}(G(n, c/n)) \leq \beta n\big] \;\leq\; \exp\big(-\delta(\beta,c)\,n + 3\ln n + c\,a\big) \;\leq\; e^{c/2}\, n^3\, e^{-\delta(\beta,c)\,n}.$$

So whenever $\delta(\beta, c) > 0$, the probability that the pathwidth is as small as βn is exponentially small, and the pathwidth is at least βn with high probability. The next statement says for which β that happens.

Theorem E.2 (the admissible region)**.** *If $c \leq 4\ln 2$ there is no $\beta \in (0,1)$ with $\delta(\beta, c) > 0$. If $c > 4\ln 2$ there is a number $\beta^*(c) \in (0,1)$ such that $\delta(\beta, c) > 0$ exactly when $\beta < \beta^*(c)$; it is the root in $(0,1)$ of*

$$-(1-\beta)\ln\tfrac{1-\beta}{2} - \beta\ln\beta \;=\; \tfrac{c}{4}(1-\beta)^2.$$

Hence $\mathrm{pw}(G(n, c/n)) \geq \beta n$ with probability $1 - O(n^3 e^{-\delta n})$ for every $\beta < \beta^*(c)$, and as $c \to \infty$, $1 - \beta^*(c) = \big(4\ln c - 4\ln\ln c + O(1)\big)/c$.

Table E.1 gives $\beta^*(c)$ at a few values of c, beside the only published explicit lower bound we know for a width parameter of $G(n, c/n)$, the treewidth bound $1 - \sqrt{9\ln 3/(2c)}$ of Perarnau and Serra (2014), which is a lower bound on pathwidth too since treewidth never exceeds pathwidth.

Table E.1: The constant $\beta^*(c)$ of Theorem E.2, to four decimals, and the treewidth bound of Perarnau and Serra (2014) at the same c (blank where that bound is not positive). Two independent programs agree on every digit of β^*.

c	3	4	6	8	10	20	50	100
$\beta^*(c)$	0.0087	0.0613	0.1706	0.2628	0.3371	0.5535	0.7606	0.8574
$1 - \sqrt{9\ln 3/(2c)}$			0.0918	0.2136	0.2966	0.5027	0.6855	0.7776

The threshold $4\ln 2 \approx 2.77$ in Theorem E.2 belongs to the method, not to the graphs: it is the point at which an edge-free split of the vertices into two equal halves stops being likely, and below it the first moment simply says nothing. The pathwidth of $G(n, c/n)$ is in fact linear in n for every $c > 1$ (Lee et al., 2012), with a constant that no one has made explicit.

E.3 The proof, in four steps

Step 1: a narrow layout leaves an empty pair. This is the picture of Appendix E.1, made exact. Put the vertices in a line and look at any position i. The first $i+1$ vertices are the prefix; the rest are the suffix. A suffix vertex is *active* if it has a neighbour in the prefix. The vertex separation of the line is the largest number of active vertices at any position, and by Theorem 2.2 the smallest

vertex separation over all lines is the pathwidth. Now take a line of vertex separation k and any position i. The suffix has $n-i-1$ vertices, at most k of them active, so at least $n-i-1-k$ suffix vertices have no neighbour at all in the prefix. Call that set R and the prefix A. Then A and R are disjoint, $|A| = i+1$, $|R| \geq n-i-1-k$, and there is no edge between them: an *empty pair*. This holds at every position of every line, so a graph of pathwidth k has an empty pair with $|A| = t$ and $|R| \geq n-t-k$ for every $t \leq n$. With $t = \lceil (n-k)/2 \rceil$ both parts have about $(n-k)/2$ vertices (Appendix A, A.53).

Step 2: count the graphs that have one. Fix a pair of disjoint sets A and R with $|A| = t$ and $|R| = t-1$. In $G(n,p)$ the $t(t-1)$ possible edges between them are all absent with probability $(1-p)^{t(t-1)}$. There are $\binom{n}{t}\binom{n-t}{t-1}$ such pairs. A graph of pathwidth at most k has at least one of them empty, so by the union bound

$$\Pr[\mathrm{pw} \leq k] \;\leq\; \binom{n}{t}\binom{n-t}{t-1}(1-p)^{t(t-1)}, \qquad t = \lceil (n-k)/2 \rceil.$$

This is the whole probabilistic content of the proof, and it is a Lean theorem over the finite sum of the weights $p^{|E|}(1-p)^{\binom{n}{2}-|E|}$ of all graphs on n vertices, with no probability model needed; the count of graphs with a given empty pair is exact there, $2^{\binom{n}{2}-t(t-1)}$ of them (Appendix A, A.54).

Step 3: estimate the count. Two standard inequalities turn the right-hand side into an exponential in n. The number of ways to choose the pair is the trinomial coefficient $n!/\big(t!\,(t-1)!\,(n-2t+1)!\big)$, and a trinomial coefficient is at most $\exp(nH)$ with H the entropy of the three proportions t/n, $(t-1)/n$ and $(n-2t+1)/n$ (Cover and Thomas, 2006, Theorem 11.1.3): expand $(x+y+z)^n = 1$ at $x = t/n$, $y = (t-1)/n$, $z = (n-2t+1)/n$ and keep one term of the sum. And $(1-p)^m \leq e^{-pm}$ because $1-p \leq e^{-p}$. With $p = c/n$ the second factor is at most $\exp(-c\,t(t-1)/n)$.

Step 4: the bookkeeping. The three proportions are not exactly a, a and $1-2a$: with $k = \lfloor \beta n \rfloor$, each is within $1/n$ of its target. A shift of d in one argument of ϕ changes ϕ by at most $\phi(d)$ when $d \leq 1/e$, and $\phi(1/n) = (\ln n)/n$; three shifts cost $3\ln n$ in the exponent, which is the $3\ln n$ of the theorem and the reason for $n \geq 3$. Likewise $t(t-1) \geq a^2n^2 - an$, and the $-an$ costs the term $c\,a$. Multiplying the two estimates of Step 3 into the bound of Step 2 gives $\exp(nH(a,a,1-2a) - ca^2n + 3\ln n + ca)$, which is Theorem E.1; $a \leq 1/2$ gives its second form. Nothing is hidden in an error term: the inequalities of Step 3 are exact bounds on the two factors, and the two corrections are the ones just named (Appendix A, A.55).

The admissible region. On $(0, \frac{1}{2}]$ the function $g(a) = H(a,a,1-2a) - ca^2$ is strictly concave, tends to 0 as $a \to 0$, and equals $\ln 2 - c/4$ at $a = \frac{1}{2}$. If $c \leq 4\ln 2$ it is nonnegative throughout, so $\delta = -g$ is never positive. If $c > 4\ln 2$ it has exactly one zero a^* in $(0, \frac{1}{2})$, is positive before it and negative after it, so $\delta(\beta, c) > 0$ exactly when $a = (1-\beta)/2 > a^*$, that is when $\beta < 1 - 2a^* =: \beta^*(c)$. Substituting $a = (1-\beta)/2$ into $g(a) = 0$ gives the equation of Theorem E.2, and the asymptotic form follows from the two-sided estimate $2\ln(1/a) - ca \leq g(a)/a \leq 2\ln(1/a) + 2 - ca$. This part is proved on paper only.

E.4 The benchmark generator

The random instances of Section 5 are not $G(n,p)$. The generator of Chu and Stuckey (2009) fills a customer-by-pattern table with independent entries at a density chosen to give d customers per pattern on average, and discards any instance whose MOSP graph is disconnected; the files show two further rules the paper does not state, every customer given at least one pattern and every pattern at least two customers, and with those rules a simulated generator reproduces the size statistics of all forty classes. The MOSP graph of such an instance is the union of one clique per pattern. Steps 1 and 2 of the proof survive this change unaltered: a graph of pathwidth k still has an empty pair, and a pair is empty exactly when no pattern contains a customer of each side. If the patterns are independent random customer sets of sizes $s_1,\dots,s_m$, the probability that a given pair (A,R) with $|A|=t$, $|R|=t-1$ is empty factorises over the patterns, and the union bound gives

$$\Pr[\mathrm{pw}\le k] \;\le\; \binom{n}{t}\binom{n-t}{t-1}\prod_{j=1}^{m}\frac{\binom{n-t}{s_j}+\binom{n-t+1}{s_j}-\binom{n-2t+1}{s_j}}{\binom{n}{s_j}},$$

the fraction being the share of s_j-sets that miss A or miss R. This is a Lean theorem in the paper's own vocabulary, with the instance built from the patterns and the event stated as MOSP $\le k+1$ (Appendix A, A.56). It is computable from the pattern sizes alone, so it predicts the benchmark from the generator's parameters. Table E.2 gives, for each of the forty classes, the mean certified optimum of its five instances beside the floor the inequality gives at confidence 0.01, that is, the largest value it does not rule out at that level plus one. None of the 200 certified optima lies below its floor. At $d=10$ the floor is within 5% of the optimum, at $d=4$ within 20%; at $d=2$ it is a third short, and asymptotically it says nothing there: with $m=\mu n$ patterns of d customers, the exponent of the expected number of balanced empty pairs is $\ln 2\,\big(1+\mu(1-d)\big)$, so the method gives a positive constant exactly when $\mu(d-1)>1$. That quantity is the mean number of patterns per customer minus one, the number of ones in the table minus the number of patterns, per customer; call it the excess. The classes of Section 5 that took thousands of core-hours to certify, the sparsest ones ($d=2$), have excess between 1.66 and 1.78, just inside the region where this argument proves linear pathwidth; on generated random covers the cost of the exact search peaks at excess about 2 to 2.4, between the $d=2$ classes and the $d=4$ classes (excess 2.9 to 3.4 at 125 customers). Section E.5 places every instance on that axis and tests, and rejects, the hypothesis that the peak is where this bound is loosest.

Table E.2: The Chu & Stuckey classes: mean certified optimum of the five instances, and the floor from the first-moment inequality at confidence 0.01, computed from the pattern sizes of each file. No certified optimum lies below its floor.

customers	patterns	$d=2$	4	6	8	10
30	30	8.6 / 4.6	16.0 / 11.8	21.2 / 17.8	25.2 / 21.8	27.0 / 24.6
40	40	11.0 / 5.4	20.2 / 15.0	28.2 / 23.4	32.6 / 29.0	35.4 / 32.6
50	50	13.2 / 7.4	23.8 / 19.0	35.2 / 30.2	40.4 / 36.6	44.4 / 41.4
50	100	21.2 / 15.8	34.6 / 30.2	42.4 / 38.6	46.2 / 43.0	47.8 / 45.0
75	75	15.8 / 9.6	34.2 / 26.4	50.0 / 43.2	59.4 / 54.4	64.8 / 59.6
100	50	11.6 / 4.6	26.4 / 15.4	44.0 / 31.8	56.2 / 45.4	67.2 / 57.4
100	100	20.2 / 11.8	45.0 / 35.4	64.6 / 57.0	76.6 / 70.2	84.6 / 79.4
125	125	22.8 / 13.6	53.0 / 42.4	78.4 / 69.6	93.6 / 86.8	103.2 / 98.0

The inequality assumes the patterns are uniform given their sizes, which the padding and the rejection break. What is proved is the inequality with the sizes the table has *before* the repairs. On the Bernoulli table a given pair is empty with probability exactly $[(1-p)^t + (1-p)^{t-1} - (1-p)^{2t-1}]^m$, the binomial average of the fraction above; the repairs only add ones, so an empty pair of the padded table is an empty pair of the original (Appendix A, A.57) and the bound with Bernoulli sizes holds for the padded instance; and if the patterns are padded before the customers, the rows after that pad are independent uniform sets of size $\max(s, 2)$ with s binomial, and the bound holds with those sizes. The rejection is not an increasing event of the table entries: adding an entry can make a pad vanish, disconnect the instance and lower its pathwidth, so Harris's inequality does not apply to it, and conditioning on acceptance costs at most the factor $1/\Pr[\text{accepted}]$, which is measured: within 1% of 1 at $d \geq 6$ and on the square classes at $d = 4$, 0.77 at the 100×50, $d = 4$ class, 0.53 to 0.87 at $d = 2$, and 0.005 at the 100×50, $d = 2$ class. A Chernoff bound on the pattern sizes gives a quenched form of the same floor, two to four stacks higher on the largest classes. These proved floors lie below the floors of Table E.2, by zero to five stacks at $d \geq 8$, by two to twelve at $d = 4$ and 6 on the square classes and by more on the 100×50 classes; at $d = 2$ they are trivial, because 40% of the Bernoulli patterns have at most one customer and the pads that make them count are exactly what the proof cannot use. What remains between the proved floors and the table is, at $d \geq 4$, the difference between a bound on the average over size vectors and the typical size vector, and at $d = 2$ the pads. With the sizes the files actually have the statement is measured, not proved: under the fitted generator the repairs and the rejection make empty pairs rarer than the uniform model says, by a factor that falls from 1.0 to 0.6 as the sets grow, and on 7,200 fresh instances at 30 and 40 customers solved exactly none has an optimum below its 0.01 floor.

The same first moment appears in Gao (2012) for random intersection graphs, with treewidth in place of pathwidth. His proof drops the factor corresponding to the product above between two lines, which is harmless when the number of patterns is sublinear in n and changes the constant when it is linear; his threshold of two patterns per customer is the dropped factor's artefact, and with it restored the condition at $m = n$ is about 2.46.

E.5 The excess, and a phase diagram

The quantity that ends the previous subsection deserves a name and an axis. For an instance with n customers, m patterns and N ones in the table, the *excess* is $e = (N - m)/n$, the mean number of patterns per customer minus one. With $r = m/n$ and $c = N/m$ the mean customers per pattern, $e = r(c - 1)$; and if the customer–pattern incidence graph B, with a node per customer and per pattern and an edge per one in the table, has κ components, it has $n(e-1) + \kappa$ independent cycles, so B can be a forest only when $e < 1$. We read e from the table, never from a generator's nominal parameters, whose repairs inflate it. Three values of e carry a meaning. The constant $\beta^*(d, \mu)$ is positive exactly when $\mu(d-1) > 1$, that is when $e > 1$ (on paper, from the exponent of (Appendix A, A.56)). On random covers of 15 to 75 customers at every ratio m/n from $1/8$ to 2, the nodes our search spends to refute optimum $- 1$ peak at $e = 2$ to 2.4, falling by two orders of magnitude on either side (measured). And at $e_g(r) = (\sqrt{r^2 + 4r} - r)/2$, 0.62 for $m = n$, the branching factor of the random incidence graph is one: on generated covers of 10 to 75 customers the share of instances whose B is a forest falls from 0.74–0.92 below $e_g(r)$ to 0.23–0.40 between $e_g(r)$ and 1, and is zero above (measured). Figure E.1 places every instance of Section 5 against the three. Below $e_g(r)$ the open stacks are governed by the largest pattern (Theorem E.3);

between $e_g(r)$ and 1 the first moment says nothing and the deterministic bounds are all there is; between 1 and the ridge the certified optimum per customer sits 0.02–0.10 above β^* and the cost of certification climbs, and this is where the $d = 2$ classes of Section 5 ($e = 1.66$–1.78) and 22 of the 24 industrial instances sit; above the ridge the cost falls and β^* comes within 0.04–0.06 of the certified slope at $e \geq 6$ ($m = n$, 30–100 customers), the $d = 4$ classes ($e = 2.9$–3.7) being the dense shoulder. No benchmark instance we hold has $e < e_g(r)$, and 12 of 6,376 have $e < 1$.

The forest regime. Write ω for the size of the largest pattern.

Theorem E.3 (blow-up). *For every instance,* MOSP $\leq \omega\,(\mathrm{pw}(B) + 1)$. *If B is a forest,* MOSP $\leq \omega\big(\lfloor \log_3 \frac{6(n+m)+3}{5} \rfloor + 1\big)$.

Replace each pattern node in each bag of a path decomposition of B by the pattern's customers: a customer's nodes in B form a star, so its bags stay an interval, and no bag grows by more than a factor ω. Nothing about B is used; the forest enters only through the pathwidth of a forest on N vertices, at most $\lfloor \log_3((6N+3)/5) \rfloor$, which is sharp. Both are Lean theorems (Appendix A, A.58) (Appendix A, A.59). The bound is loose: on 7,004 graphs with exact pathwidth (clique trees to 936 vertices, random block trees, 3,448 certified random covers whose B is a forest, and 2,000 small random instances) it is never violated and never exact, 2 to 5.5 times the truth on the block-graph families; on the 3,448 random forests the pathwidth is $\omega - 1$ on 99.8%, so below $e_g(r)$ the optimum is, in practice, the largest pattern (measured). The exact theory on block graphs is polynomial (Chou et al., 2008); on clique trees the exact values grow by about $(\omega - 2)/2$ per level of depth, no formula linear in ω and the depth fits them, and we conjecture no closed form.

Lemma E.4 (pendant cliques). *If a clique R has $q \geq 1$ cliques of size at least t pendant at distinct vertices of R, each meeting R in its attachment vertex only, then* $\mathrm{pw} \geq t + \lceil q/2 \rceil - 2$.

Every clique lies inside one bag; if a pendant's bag is the bag of R the two share a bag of size $|R| + t - 1$, and otherwise at least $\lceil q/2 \rceil$ pendants have their bags on one side of R's, and the one nearest R holds its own clique together with the attachment vertices of the others, by the interval property. It is a Lean theorem (Appendix A, A.63), and with $t = q = |R|$ it gives the exact value $\lceil 3s/2 \rceil - 2$ of a clique K_s with a pendant K_s at every vertex, the depth-one clique tree; with the matching decomposition (on paper), K_s with a pendant K_t at q of its vertices has pathwidth exactly $\max(s - 1,\ t + \lceil q/2 \rceil - 2)$. It is a pathwidth argument that is not degree-based, and on the certified instances it is worth little: evaluated over the maximal cliques of the MOSP graph it is above the clique floor $\omega - 1$ on 554 of the 6,376 and tight on 44%, but above the solver's bound on 5, and above $\max(\text{solver bound},\ \mathrm{tw} + 1)$ on 3 of the 338 instances where that bound misses by two or more, by one stack each (measured, 9–134 customers).

Constructions from above.

Lemma E.5 (layers). *If $L_0, L_1, \ldots$ are the distance layers from any vertex of a connected graph,* $\mathrm{pw} \leq \max_i(|L_i| + |L_{i+1}|) - 1$; *and* $\mathrm{pw} \leq n - \alpha$ *for any independent set of α vertices.*

The bags are $L_i \cup L_{i+1}$, and $(V \setminus I) \cup \{v\}$ for $v \in I$ (Appendix A, A.61) (Appendix A, A.62). On the uniform cover model with $m = \mu n$ patterns of size d the layer sizes follow a two-type

exploration recursion and the greedy independent set that takes the customer with fewest patterns follows a differential equation; the smaller limit is a constant $\beta^+(d,\mu)$, derived on paper up to the concentration step, which we cite and do not re-prove. Table E.3 gives it beside β^* and the slope b of $\text{opt} = a + bn$ on the certified instances at 30–100 customers. The sandwich $\beta^* \le \liminf \text{opt}/n \le \limsup \text{opt}/n \le \beta^+$ has width below 0.12 at $e \ge 14$ and 0.42–0.49 for $1 \le e \le 3.5$: it says least where the search works hardest. The best published upper bound we know, the degree-sequence bound of Edwards and McDermid (2015, Corollary 5.2), is wider still on the cover degree law, 0.66, 0.86 and 0.96 at $d = 4$, 6, 10 with $\mu = 1$, because it sees the degrees of a union of cliques and not the cliques. The greedy closing rule we use for upper bounds cannot be analysed this way; at 5,000 customers its width per customer is 0.06–0.36 below β^+ (measured, no exact solve).

Table E.3: Random covers with $m = n$ patterns of d customers: the excess, the first-moment constant (a proved lower bound on $\liminf \text{opt}/n$), the slope b of $\text{opt} = a + bn$ on 600–625 certified instances at 30–100 customers (measured; one standard error in parentheses), and the constant of the analysed layouts (an upper bound on $\limsup \text{opt}/n$, on paper up to concentration).

d	e	$\beta^*(d,1)$	b ($\pm$ se)	$\beta^+(d,1)$
2	1.13	0.000	0.038 (0.001)	0.296
3	2.04	0.140	0.200 (0.003)	0.596
4	3.02	0.286	0.348 (0.003)	0.709
6	5.00	0.513	0.577 (0.003)	0.830
8	7.00	0.661	0.706 (0.002)	0.889
10	9.00	0.757	0.795 (0.002)	0.922

Why the first moment is loose. A layout of width k forces ℓ nested empty pairs, one per prefix size, but counting them gives a constant *below* β^* on every cell (by 0.051 at $\ell = 2$, $d = 6$, $\mu = 1$, and more for larger ℓ, at every spacing): naming εn more vertices costs $\Theta(\varepsilon \log(1/\varepsilon))$ of entropy and buys $\Theta(\varepsilon)$ of constraint. The slack is in the union bound. The other direction is a count that names *fewer* vertices, and we tried three: the event itself, counted once per separator; the closed pair; and a weighted count. A layout of width $k < n$ forces a set S of k vertices whose removal leaves the other $N = n - k$ vertices split into two sides of sizes $\lceil N/2 \rceil$ and $\lfloor N/2 \rfloor$ with no edge between them, and for a fixed S the probability of that event depends only on the component structure of $G - S$, which is $G(N,p)$.

Theorem E.6 (separators, finite form)**.** *For $0 \le p \le 1$ and $k < n$, $\Pr[\text{pw}(G(n,p)) \le k] \le \binom{n}{k} P_N(E_{\text{bip}})$, where $P_N(E_{\text{bip}})$ is the probability that the components of $G(N,p)$ can be grouped into two classes of sizes $\lceil N/2 \rceil$ and $\lfloor N/2 \rfloor$.*

The union bound over the $\binom{n}{k}$ sets S is the only inequality: the graphs on n vertices with a given restriction to $V \setminus S$ have total weight equal to the restriction's $G(N,p)$ weight, and the inner probability is the same for every S. It is a Lean theorem with the inner probability as a finite weighted count (Appendix A, A.67); on paper that count is a sum over the integer partitions of N with Gilbert's connectedness recursion, which we evaluate exactly to $N = 120$. Step 2 bounds the same event by its number of witnesses, $\binom{N}{t}(1-p)^{t(N-t)}$; the theorem merges every balanced empty bipartition of $G - S$ into one event. Writing $c' = c(1-\beta)$ for the mean degree of $G - S$ and

$I(c')$ for the rate at which the probability that no component of $G(N, c'/N)$ exceeds $N/2$ decays, the constant becomes the root $\beta_{\mathrm{sep}}(c)$ of $H(\beta) = (1-\beta)\, I(c(1-\beta))$. That rate is a corollary of the large-deviation principle for the largest component (O'Connell, 1998; Puhalskii, 2005; Andreis et al., 2021), whose rate function at one half is positive exactly when $c' > 2\ln 2$; our exact inner probabilities reproduce the quoted rate to 0.002 at seven values of c' (fitted over $N = 100$–400), and the bipartition event has the same rate as its relaxation to $N = 120$. With it, $\beta_{\mathrm{sep}} = 0.037$ at $c = 3$ against $\beta^* = 0.009$, and 0.341 at $c = 10$ against 0.337: a third of the gap to the slope fitted to exact pathwidths (0.099) at $c = 3$, a twentieth at $c = 10$ (0.415). Conditional on the cited large-deviation upper bound, the threshold of the method moves from $4\ln 2$ to $2\ln 2$, the density at which the giant component passes half the vertices and the bisection width becomes linear (Luczak and McDiarmid, 2001); the true threshold is 1. The separator as the object of a width first moment is in Do et al. (2024), with a deterministic bound on the bipartitions per separator in place of their probability, and the exact inner count of a single vertex cut of a random regular graph in Kolesnik and Wormald (2014); we found no prior statement of the inner probability of the bipartition event, nor of its use for a width constant.

The closed pair, in which every separator vertex is adjacent to both sides ($S = N(A) \cap N(R)$; a layout forces one with sides at least the forced sizes), is the dense-side counterpart. Its expected number is exactly $\sum n!/(a!\, r!\, s!)\, (1-p)^{ar} \big((1-(1-p)^a)(1-(1-p)^r)\big)^s$ over the admissible size triples (the union bound and the characterisation in Lean (Appendix A, A.68), the independence step by a product over disjoint blocks of pairs, in Lean (Appendix A, A.69)), and its constant exceeds β^* by 0.007–0.010 at $c = 6$–12, and by 0.011 on random covers with $d = 4$ and $d = 6$, $\mu = 1$, where the separator count has no large-deviation rate to cite. A weighted count, each empty pair divided by the number of pairs reachable from it by moving low-degree separator vertices across, is valid too (Appendix A, A.66) and adds nothing to the closed pair on this record, its lower tail being the event that the pair is closed. None of the three moves the headline constants by the 0.02 we had set as worth a page; at $c = 6$ and 10 the best of them closes 12% and 11% of the gap to the fitted slope.

What the cluster is we measured directly, by planting an empty pair and enumerating the others ($n \le 32$ on $G(n, c/n)$ at $c = 6$ and 10, $n \le 28$ on covers at $d = 6$, $\mu = 1$, 200 instances per cell). The other empty pairs form two lobes, perturbations of the planted pair and of its reverse through the separator, with nothing between; on the covers the median planted instance has no pair but the reverse, so the $e^{0.23n}$ is carried by rare instances, and on $G(n, c/n)$ the median cluster grows at about half the rate of the mean. The whole slack of the union bound at a cell is the mean of $1/X$ under the planted law, a bounded statistic: 0.065, 0.039 and 0.043 nats per vertex at the largest sizes, 0.019, 0.012 and 0.015 in β, sub-linear in n over the range (measured). In stacks, on certified instances, the counts of this paragraph move the floor by one to two at 60 to 75 vertices and the pathwidth sits four to five higher; Harper's vertex-isoperimetric bound, a deterministic consequence of the empty pair, is within a stack of the pathwidth wherever our branch and bound reaches the forced prefix, which puts at least three quarters of the leftover in the union bound as read on instances, where the planted measurement at $n \le 32$ reads 0.012–0.019 in β; the two instruments see different sizes, and we do not reconcile them.

One parameter, or two. The excess sets the threshold, the ridge and the scale, not the value. At $e = 2$ the certified slope is 0.282, 0.232, 0.200, 0.167 for $\mu = 1/4, 1/2, 1, 2$ (standard errors 0.002–0.006), and 13 of 16 pairs of series matched in e to ± 0.05 differ by more than twice their error; β^*

has the same dependence (0.180, 0.169, 0.147, 0.123), and over 41 series $b = 0.057 + 0.984\,\beta^*$ with residual 0.024. Nor is the ridge where the first moment is loosest: on 25 series at 50–75 customers the gap between the certified optimum and the floor peaks at $e = 2.2$–5.3, on the dense side, with the ridge on a plateau within one stack of the maximum. What peaks on the ridge is the number of closed sets the search can reach, not the slack of the bound.

Twenty-four industrial instances. The 24 SCOOP instances (Frinhani et al., 2018) have excess 0.97 to 2.50, median 1.35; 19 lie in $[1, 2)$, none has a forest B, ten decompose. Every refutation on record for them took at most 22,433 nodes, and the greedy closing rule is exact on 12 and at most $0.061n$ above the optimum. One, with 60 customers, 49 patterns and optimum 7, sits two stacks below the floor a random cover with its pattern sizes has at confidence 0.01: no contradiction, the floor being a random-model statement, but a measure of distance from random, on which the designed Challenge instances sit up to 39 stacks below. These are 24 instances from one source, read with pieces as customers (the orientation of the files' format line, the opposite of their description's prose); we say nothing about industry beyond them.

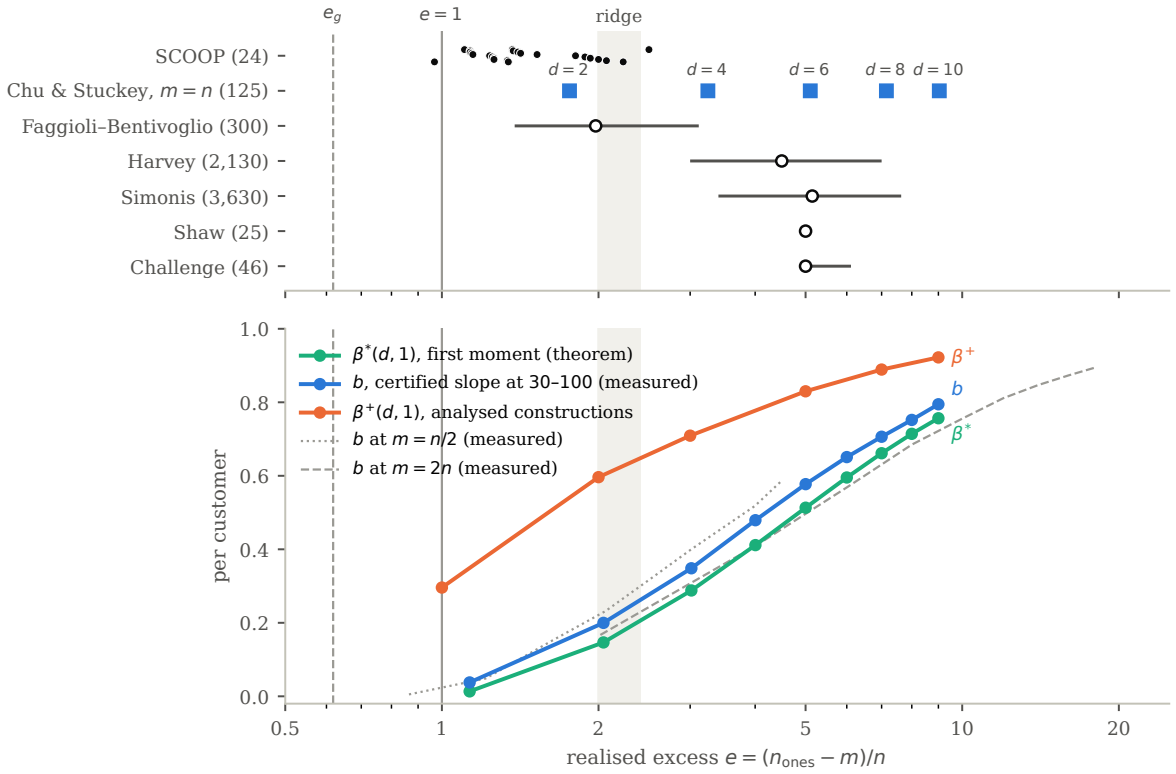


Figure E.1: The excess phase diagram. Above, where the instances of Section 5 sit: the 24 SCOOP instances; the Chu & Stuckey classes with $m = n$ as the median of each density over all sizes; the median and quartiles of the other collections. Dashed, $e_g(1) = 0.62$; solid, $e = 1$; the band, the ridge $e = 2$–2.4. Below, random covers with $m = n$ patterns of $d = 2$ to 10 customers: β^* (a theorem), the certified slope b at 30–100 customers with its two-standard-error bar (measured), and β^+ of the analysed layouts; in grey, b at $m = n/2$ and $m = 2n$, which differ from it at equal excess. Regenerate: `paper1/latex/figures/excess_phase_diagram.py`.

E.6 What is checked, and what is known

Steps 1, 2, 3 and 4 and the two inequalities of Theorem E.1 are Lean theorems with the statement exactly as printed, including the constant 3; so is the consequence that the probability tends to zero whenever $\delta(\beta, c) > 0$. The entropy bound on trinomial coefficients is not in Mathlib and is proved there from the binomial theorem. Theorem E.2 is proved on paper, and the values in Table E.1 were computed by two programs written independently. Theorem E.6 is a Lean theorem with its inner probability as a finite weighted count; the closed-pair count is a Lean theorem in both its pair-summed and its size-summed form, with the independence step a product over disjoint blocks of pairs; the constants β_{sep} rest on a cited large-deviation rate, and every rate we fitted is called fitted.

The argument is published twice in cruder form. Turner (1986) used the same empty pair, with the count $(en/k)^{2k}$ in place of the entropy, to prove that almost every $G(n, p)$ with p fixed has bandwidth at least $n - 4\log_{1/(1-p)} n$; since bandwidth is at least pathwidth, our dense statement implies his and not conversely. Díaz et al. (2001b) used it for vertex separation on mixing graphs, with a 3^n count and Chernoff slack at a single position, which gives no positive constant below $c \approx 227$. Neither states the entropy form, a value of $\beta^*(c)$, the threshold $4\ln 2$ or the asymptotics, and we found no later work that does. Three more things are known. The best published upper bound on the pathwidth of $G(n, c/n)$ is the degree-sequence bound of Edwards and McDermid (2015), which on Poisson degrees gives 0.183, 0.458 and 0.656 at $c = 3, 6, 10$, so the measured slopes sit near the middle of the sandwich between $\beta^*(c)$ and it. We did not take that bound on trust. Their Corollary 5.2 states $\mathrm{pw}(G) \le \sum_v g(d_{r(G)}(v)) + \tau(n) + O(\log n)$ for every graph on n vertices, with g a weight function ($g(d) = 0$ for $d \le 2$, $g(3) = 1/6$, $d\,g(d) = (d-2)g(d-1) + g(d-2) + 1$), $r(G)$ the series-parallel reduction of G, and $\tau(n) = o(n)$ the error term of the cubic base case (Fomin and Høie, 2006, Theorem 5); the paper gives no constant for either lower-order term. On every graph of the record with a proved pathwidth, 63,330 graphs on 1 to 2,916 vertices, the main term alone is at least the pathwidth on 61.8%, and the pathwidth exceeds it by at most 5 (the 202-vertex extremal trees), by at most 4.34 where the reduction is non-empty (the Coxeter graph). Fomin and Høie's explicit budget exceeds these deficits by a factor of 2.2 at ten vertices and 100 at 2,916: the corollary holds everywhere on record and, at these sizes, says nothing a graph could violate. Lemma 5.1 and Corollary 5.2 are also Lean theorems (Appendix A, A.70) (Appendix A, A.71), conditional on the two cited results the proof rests on, each a named proposition that no theorem asserts: the cubic base case $\mathrm{pw} \le N/6 - 1/6 + \tau(N)$ for connected cubic graphs with a budget τ (Fomin and Høie, 2006, Theorem 5), whose bisection step is Monien and Preis (2006), and the reduction lemma $\mathrm{pw}(H) \le \mathrm{pw}(r(H)) + \rho(|V(H)|)$ with a budget ρ (Kneis et al., 2009); τ is taken monotone and non-negative. With the budgets explicit the bound reads $\mathrm{pw}(G) \le \sum_v g(d_G(v)) + \tau(n) + 3\rho(n) + 1$, ρ once per use of the reduction lemma and 1 for the deleted vertex of a basic component of the second kind; the paper's $\tau(n) + O(\log n)$ is this with $\rho = O(\log n)$. This is a formalisation of a published theorem with its lower-order term made explicit, not a new result. Theorem 5.3, the average-degree form, rests on an averaging lemma of Edwards and Farr (2012) that we checked on the record and did not formalise. The prefix-at-$(1-\varepsilon)n$ variant of the empty-pair argument is published by Cheung and Girardet (2020), who reach pathwidth $(1 - o(1))n$ for every $p \ge n^{-1+\varepsilon}$, still nothing at $p = c/n$. And a sharp constant for vertex separation is known for one random model only, dense random geometric graphs (Díaz et al., 2001a), and for none of $G(n, c/n)$, random regular graphs or random clique covers.

One computation says how far the finite inequality is from the truth. On 133 random graphs

with 60 vertices and $c = 10$ whose pathwidth we computed exactly, the smallest pathwidth is 25, while Theorem E.1's first inequality, evaluated exactly rather than through Step 3, puts the probability of pathwidth at most 23 below 0.35 and of at most 21 below 10^{-3}. The gap between $\beta^*(c)$ and the slopes fitted to such data is of the order of 0.1 and does not close with n. Counting the event a narrow layout forces instead of its witnesses, once per separator or once per closed pair (the paragraph *Why the first moment is loose* in Appendix E.5), closes a third of it at $c = 3$ and a tenth at $c = 6$ and 10; the best published upper bound, the degree-sequence bound of Edwards and McDermid (2015, Corollary 5.2), verified on every proved graph of the record and formalised conditional on its two cited inputs as described above, gives 0.183, 0.458 and 0.656 on Poisson degrees at $c = 3$, 6 and 10, so the provable sandwich on pw/n is $[0.037, 0.183]$, $[0.182, 0.458]$ and $[0.345, 0.656]$ there, against fitted slopes of 0.099, 0.268 and 0.415.

Appendix F The dual of pathwidth

Every lower bound this paper uses reads one object at a time: a clique, a vertex of large minimum degree in a minor, a prefix of a layout in Harper's bound, a contracted copy of the graph in Theorem 4.11, a minor found by the search of Appendix G. Each is a certificate that the pathwidth is at least some value, and each family has a ceiling on what it can certify. This appendix asks the question behind all of them: is there a single family with no ceiling, a *dual* of pathwidth in the sense that linear programming has a dual, and if so, why can it not simply be maximised? The answer to the first question is yes, the blockages of Bienstock et al. (1991); the answer to the second is three theorems and one conjecture. Throughout, n is the order of a blockage, never the number of vertices, which is written $|V|$. The definitions and the weak half of the duality are formalised in `Blockage.lean` and `BlockageClass.lean`, with no `sorry` (Appendix A, A.89) (Appendix A, A.90).

F.1 The problem of the dual

Pathwidth is a minimum: over all orderings of the vertices, the largest number of vertices that a prefix has in common with the neighbourhood of its complement, minimised. An ordering is therefore a short proof of an upper bound: $|V|$ numbers, checked in linear time, and the search of Section 4 finds one in milliseconds on most instances. A lower bound has no such object. The statement $\mathrm{pw}(G) \geq w$ is proved, in the body of this paper, by exhausting a search at width $w-1$, and the proof is the search tree, 10^{11} nodes on the hardest instances of Section 5.2.

Linear programming resolves the same asymmetry by a second optimisation problem: a maximisation whose every feasible point is a lower bound on the minimum (weak duality) and whose optimum equals it (strong duality). A feasible dual point is then a certificate of the lower bound, and the best certificate is tight. For treewidth the dual object is the bramble of Seymour and Thomas (1993): a family of connected, pairwise touching vertex sets, whose order is the size of the smallest set meeting all of them, and treewidth plus one is the largest order of a bramble. For pathwidth it is the blockage. This appendix states the object, proves that it is the dual, and then proves why maximising it is not an algorithm: not in polynomial time unless P = NP, not with a short certificate unless NP = coNP, and not by the two concrete dual searches one would write first, whose cost is provably at least a binomial where the primal's is small.

F.2 Blockages

Let $G = (V, E)$ be a finite simple graph. For $X \subseteq V$ the *attachments* of X are its vertices with a neighbour outside X,

$$\mathrm{att}(X) \;=\; \{\, v \in X : \exists u \notin X,\ uv \in E \,\},$$

and $\alpha(X) = |\mathrm{att}(X)|$. Two sets X_1, X_2 are *complementary* if $X_1 \cup X_2 = V$ and $\mathrm{att}(X_1) \subseteq X_2$; equivalently no edge joins $X_1 \setminus X_2$ to $X_2 \setminus X_1$, so a complementary pair is a separation of G with separator $X_1 \cap X_2$, and every X has the canonical partner $X^c = (V \setminus X) \cup \mathrm{att}(X)$.

Definition F.1 (blockage, Bienstock et al., 1991)**.** A *blockage of order n* is a family $\mathcal{B}$ of subsets of V such that

(i) every $X \in \mathcal{B}$ has $\alpha(X) \leq n$;

(ii) if $X \in \mathcal{B}$, $Y \subseteq X$ and $\alpha(Y) \leq n$, then $Y \in \mathcal{B}$;

(iii) if X_1, X_2 are complementary and $|X_1 \cap X_2| \leq n$, then exactly one of X_1, X_2 lies in $\mathcal{B}$.

Read as a game, $\mathcal{B}$ is the strategy of an invisible fugitive against $n+1$ searchers: for every way the searchers can cut the graph with at most n of them, (iii) says which side the fugitive is on, and (i), (ii) make those choices consistent; this is Theorem (5.1) of the paper. The bounds are $\leq n$ throughout, as in the paper's proof of its Theorem (2.1) and in the Lean statement; with strict bounds the family $\{\emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}\}$ on the path a–b–c would be a "blockage of order 2" on a graph of pathwidth 1.

A blockage is a large object, but it is determined by a small part of itself. Call X *big* if $\alpha(X) \leq n$ and $X \notin \mathcal{B}$.

Lemma F.2 (a blockage is an avoidance family)**.** *Let $\mathcal{B}$ be a blockage of order n. Bigness is upward closed among the sets with $\alpha \leq n$, and*

$$\mathcal{B} \;=\; \{\, X \subseteq V : \alpha(X) \leq n \text{ and no big set is contained in } X \,\}.$$

Hence $\mathcal{B}$ is determined by its frame*, the $\subseteq$-minimal big sets $F_1, \ldots, F_r$. For such a frame, axioms (i) and (ii) hold automatically, and (iii) reads:*

(a) every separation of order at most n has a side containing some F_j;

(b) no separation of order at most n has F_i inside one side and F_j inside the other.

Proof. If X is big and $X \subseteq Y$ with $\alpha(Y) \leq n$, then $Y \notin \mathcal{B}$, for otherwise (ii) would put X in $\mathcal{B}$; so bigness is upward closed. A set with $\alpha \leq n$ is in $\mathcal{B}$ or big, and it is big exactly when it contains a minimal big set, which gives the display. For the reading of (iii): a complementary pair (X_1, X_2) with $|X_1 \cap X_2| \leq n$ has $\mathrm{att}(X_i) \subseteq X_1 \cap X_2$, so both sides satisfy (i); "exactly one in $\mathcal{B}$" says exactly one side contains no frame set, which is (a) and (b) together. Condition (b) is a Menger condition and is checked pair by pair in polynomial time; condition (a) is the whole difficulty. □

Three frames are proved to give blockages in `BlockageClass.lean`: a clique K of $n+1$ vertices, with the single frame set K (`isBlockageIn_clique`); the pullback of a blockage of H along a minor model of H in G, each frame set pulled back along its branch sets (`isBlockageIn_minor`); and three disjoint connected branches each carrying a frame of order k, pairwise joined by connected connectors missing the third, whose union carries a frame of order $k+1$ (`isBlockageIn_branch`), the blockage form of the corrected branch lemma of Section 4.7. Every statement was checked by exhaustive enumeration on every graph with at most six vertices before it was stated (Appendix A, A.90).

F.3 Why it is the dual

Theorem F.3 (weak duality: a blockage is a lower bound)**.** *If G has a blockage of order n, then* $\mathrm{pw}(G) \geq n$ *(Appendix A, A.89).*

Proof. Suppose $\mathcal{B}$ is a blockage of order n and, for a contradiction, that $W_0, \ldots, W_r$ is a path decomposition of G of width less than n, so every bag has at most n vertices. Let $X_i = W_0 \cup \cdots \cup W_{i-1}$ be the prefix unions, with $X_0 = \emptyset$ and $X_{r+1} = V$. Two facts follow from the interval

property of a path decomposition: $\mathrm{att}(X_i) \subseteq W_i$, since a vertex of X_i with a neighbour outside X_i shares a bag with that neighbour, which lies at or after W_i, so the vertex's interval of bags reaches W_i; and $X_{i+1} \setminus X_i \subseteq W_i$ by definition. Hence $\alpha(X_i) \le n$ for every i.

Now $\emptyset \in \mathcal{B}$ and $V \notin \mathcal{B}$, because $(\emptyset, V)$ is a complementary pair with empty intersection, and (iii) puts exactly one of the two in $\mathcal{B}$, while (ii) with $Y = \emptyset$ rules out the second choice. Lemma (2.2) of the paper, proved in Lean as `IsBlockage.mem_of_mem`, says: if $X \in \mathcal{B}$, $\alpha(Y) \le n$ and $|(Y \setminus X) \cup \mathrm{att}(X)| \le n$, then $Y \in \mathcal{B}$. Apply it to $X = X_i \in \mathcal{B}$ and $Y = X_{i+1}$: $\alpha(X_{i+1}) \le n$, and $(X_{i+1} \setminus X_i) \cup \mathrm{att}(X_i) \subseteq W_i$ has at most n vertices, so $X_{i+1} \in \mathcal{B}$. By induction $V = X_{r+1} \in \mathcal{B}$, a contradiction. □

Theorem F.4 (strong duality, Bienstock et al., 1991, Theorem (2.1))**.** *G has a blockage of order n if and only if* $\mathrm{pw}(G) \ge n$*. Consequently*

$$\mathrm{pw}(G) \;=\; \max\{\, n : G \text{ has a blockage of order } n \,\},$$

and the feasible orders form an interval: a blockage of order n restricted to its members with $\alpha \le n-1$ is a blockage of order $n-1$.

The "if" direction is the paper's Sections 3 and 4, through partial blockages and fractures, and it is not formalised here. A shorter route, which we record because it is the one a formalisation would take, goes through orientations: Erde (2017), Theorem 12, proves by induction on the set of separations that a graph has either a path decomposition of small width or a consistent orientation of its small separations with no two chosen sides covering the graph, never both; and Diestel and Oum (2019), Theorem 7.2, shows that such an orientation is exactly a blockage, the "two sides" clause being the paper's stoppage condition. Strengthening "two" to "three" gives the tangle axioms, which is why blockages are stronger than tangles and $\mathrm{pw} \ge \mathrm{tw}$. The restriction claim is immediate from the axioms: the separations of order at most $n-1$ are among those of order at most n, and both sides of such a separation have $\alpha \le n-1$.

Weak duality is what makes a bottom-up search for lower bounds possible, and strong duality is what says such a search has no ceiling. Every lower-bound family in this paper is a restricted class of blockages. A clique of size $n+1$ gives one, with the clique as its only frame set. Harper's vertex-isoperimetric bound at one prefix size is a blockage built from the separations at that size alone. A minor H with $\mathrm{pw}(H) \ge n$ gives a blockage of G by pulling H's back along the branch sets, which is Theorem 4.11 read from the dual side. The duality gaps measured in Section 5.5, family by family, are the gaps of these restricted classes, and the finding there that the residual's obstruction is a band of consecutive prefix sizes says, in this vocabulary, that the blockages of those instances need separations at many consecutive orders at once, which no single-prefix class supplies.

F.4 Three theorems on maximising the dual

Strong duality invites an algorithm: maximise the order over blockages, from below, and stop when the order meets the width of the best layout. The following three results say, in increasing concreteness, why that is not an algorithm. The first two are standard complexity theory applied to one hardness theorem; we state them because they are exactly what the dual is being asked to do.

Theorem F.5 (no efficient maximisation)**.** *If there is a polynomial-time algorithm that, given G, returns the largest order of a blockage of G, then* P $=$ NP*. The same holds for an algorithm that returns, for given G and n, whether a blockage of order n exists.*

Proof. By Theorem F.4 the largest order of a blockage is $\mathrm{pw}(G)$, and a blockage of order n exists if and only if $\mathrm{pw}(G) \geq n$. So either algorithm decides, in polynomial time, whether $\mathrm{pw}(G) \leq k$ for a given k. That problem is NP-complete: Kashiwabara and Fujisawa (1979) prove it in the form of interval graph completion, and Lengauer (1981), Theorem 7, in the form of the vertex separation game, which equals pathwidth by Kinnersley (1992). A polynomial-time algorithm for an NP-complete problem gives P $=$ NP. □

Theorem F.6 (no short certificate)**.** *Suppose there is a polynomial p and a polynomial-time verifier $\mathcal{V}$ such that for every graph G and every $n \leq \mathrm{pw}(G)$ there is a string c of length at most $p(|V|)$ with $\mathcal{V}(G, n, c) = 1$, while $\mathcal{V}(G, n, c) = 0$ for every c whenever $n > \mathrm{pw}(G)$. Then* NP $=$ coNP*. In particular, no representation of blockages can be both polynomial in size and polynomial to check on every graph unless* NP $=$ coNP*.*

Proof. Let $L = \{(G, n) : \mathrm{pw}(G) \geq n\}$. Its complement, $\{(G, k) : \mathrm{pw}(G) \leq k\}$ with $k = n - 1$, is NP-complete by the theorems cited above, so L is coNP-complete: every language in coNP reduces to L in polynomial time. The verifier $\mathcal{V}$ with its short strings c is by definition a nondeterministic polynomial-time acceptor of L, so $L \in$ NP. Then every coNP language reduces to a language in NP, and NP is closed under polynomial-time reductions, so coNP $\subseteq$ NP; taking complements, NP $\subseteq$ coNP. The last sentence is the case in which c encodes a blockage of order n in any fixed representation and $\mathcal{V}$ checks the axioms (Garey and Johnson, 1979, Chapter 7). □

The theorem is conditional and cannot be otherwise: an unconditional proof that blockages have no short representation would separate NP from coNP. What it rules out is representation-independent. A frame of r sets (Lemma F.2) is a certificate whose verifier runs in time $|V|^{O(r)}$, polynomial for bounded r, and the theorem says that no scheme makes r bounded on every graph. For treewidth the same statement is in print: Grohe and Marx (2009) write that a bramble "is not a good characterization of tree width in the coNP sense" for exactly this reason, and prove the unconditional form in a fixed representation, to which we return in Conjecture F.9. We found no statement of either kind for pathwidth and blockages.

The third theorem is about the two dual searches one would actually write, and it is unconditional. Fix a width w and let

$$\mathcal{F}_w \;=\; \{\, S \subseteq V : |N(S) \setminus S| \leq w \,\},$$

the sets whose outer boundary has at most w vertices. A layout of width at most w is a chain $\emptyset = S_0 \subset S_1 \subset \cdots \subset S_{|V|} = V$ of sets in $\mathcal{F}_w$, each obtained from the last by adding one vertex, so $\mathrm{pw}(G) \leq w$ exactly when $\emptyset$ and V lie in the same connected component of the graph on $\mathcal{F}_w$ whose edges are single additions. Our corrected algorithm explores the component of $\emptyset$; its states are the sets of that component, within a factor of about two (Section 5.5). The natural dual search explores the component of V instead, removing vertices one at a time; a refutation by either search is the exhaustion of its component.

Theorem F.7 (the lattice floor of a top-down search)**.** *Let $w < |V|$. Every set S with $|V \setminus S| \leq w$ lies in $\mathcal{F}_w$ and in the component of V. Hence a refutation of width w that exhausts the component of V visits*

at least $\sum_{j=0}^{w} \binom{|V|}{j}$ *states, on every graph. Moreover, complementation* $S \mapsto V \setminus S$ *is an order-reversing bijection from* $\mathcal{F}_w$ *onto* $\mathcal{F}'_w = \{T : \alpha(T) \leq w\}$*, the sets whose inner boundary has at most* w *vertices, carrying single additions to single removals; so the top-down search on* $\mathcal{F}'_w$ *is the bottom-up search on* $\mathcal{F}_w$*, and the two directions that avoid the binomial region are the same algorithm.*

Proof. If $|V \setminus S| \leq w$ then $N(S) \setminus S \subseteq V \setminus S$ has at most w vertices, so $S \in \mathcal{F}_w$; and removing the vertices of $V \setminus S$ from V one at a time passes only through sets with still smaller complements, each in $\mathcal{F}_w$ by the same argument, so S is in the component of V. There are $\sum_{j \leq w} \binom{|V|}{j}$ such sets. For the second claim, $\mathrm{att}(V \setminus S)$ is the set of vertices outside S with a neighbour in S, which is $N(S) \setminus S$; so $S \in \mathcal{F}_w$ if and only if $V \setminus S \in \mathcal{F}'_w$, and adding a vertex to S removes it from $V \setminus S$. □

The theorem places the free region, where every set is feasible before any constraint bites, at the top of the lattice under the outer boundary and at the bottom under the inner one, and says that a search avoiding it is the primal search under the other name. What it does not say is how the two components compare; that is a measurement, and it was made (Section 5.5, the two-ended enumeration): on 64 bound-gap instances at 30 to 50 customers the component of $\emptyset$ has between 2,000 and 281,000 states, while the component of V exceeded 3,000,000 on 61 of them and was 300 to 1,000 times the other on the three where it was enumerated to the end. A dual search in this lattice would need a rule that discards most of that component, of a kind no primal rule is: the rules of Section 4 bring the visited states to within a factor of about two of the component, not orders of magnitude below it.

A second concrete dual search maximises over a compact class of frames. The class we built, cliques, minor pullbacks and linked branches, has its ceiling measured in Section 5.5; here is the one statement about such classes that is a theorem rather than a measurement.

Proposition F.8 (connected frames are blind to pw $>$ tw)**.** *Let* $F_1, \ldots, F_r$ *be connected vertex sets with* $|F_i| \geq n+1$*, pairwise inseparable by fewer than* $n+1$ *vertices, and such that no set of at most* n *vertices meets every* F_i*. Then the frame* $F_1, \ldots, F_r$ *generates a blockage of order* n*, and* $\mathrm{tw}(G) \geq n$*. In particular such a frame certifies* $\mathrm{pw}(G) \geq n$ *only on graphs with* $\mathrm{tw}(G) \geq n$*.*

Proof. For the blockage, check (a) and (b) of Lemma F.2. A separation of order at most n that splits every F_j has, since each F_j is connected, a separator meeting every F_j, which is a set of at most n vertices meeting all of them; so some F_j lies inside a side, which is (a). Two frame sets on opposite sides of a separation of order at most n would be separated by at most n vertices, against the hypothesis; this is (b). For the treewidth, take a tree decomposition of width $\mathrm{tw}(G)$. Each connected F_i induces a subtree of the decomposition tree. If two of these subtrees are disjoint, some bag separates F_i from F_j, so $\mathrm{tw}(G)+1 \geq n+1$. Otherwise the subtrees pairwise intersect, and subtrees of a tree have the Helly property, so some bag meets every F_i; that bag has at most $\mathrm{tw}(G)+1$ vertices, so $\mathrm{tw}(G)+1 \geq n+1$. □

This is the pathwidth copy of a bramble, and the proposition says it is only a bramble: it certifies what treewidth certifies. The ingredient that sees pw $>$ tw is branching, and the census of Section 5.5 measures how rarely branching is the obstruction.

F.5 What is open: the size of a blockage

Theorem F.6 is conditional. For treewidth, Grohe and Marx (2009) proved the unconditional version in a fixed representation: on bounded-degree expanders every bramble of order at least

$k^{1/2+\varepsilon}$ has $2^{\Omega(k^{\varepsilon})}$ sets, while every graph of treewidth k has a bramble of order $\Omega(k^{1/2}/\log^2 k)$ with polynomially many sets; see also Hatzel et al. (2022). The analogue for blockages, in the frame representation of Lemma F.2, is the natural open question, and we state it as a conjecture with the data beside it.

Conjecture F.9 (blockage size)**.** *There is a family of bounded-degree graphs on $|V|$ vertices with* $\mathrm{pw} = \Theta(|V|)$ *on which every blockage of order at least $|V|^{1/2+\varepsilon}$ has a frame of $2^{\Omega(|V|^{\varepsilon})}$ sets; and every graph has a blockage of order* $\mathrm{pw}^{\Omega(1)}$ *whose frame has polynomially many sets.*

The second half follows, with a small exponent, from the certificate of Groenland et al. (2023), which is the iterated branch operator above. For the first we have measurements and no proof. The minimum frame size over all blockages of each order was computed exactly, by a SAT model of the axioms, on every connected graph with at most nine vertices and on structured and expander-like families to 22 vertices. At order equal to the pathwidth the minimum is one on 28% of the graphs on nine vertices and at most three on 96%, never above 13; on every graph with $\mathrm{pw} > \mathrm{tw}$ it is at least three, on 13,548 of 13,548 such graphs; on random cubic graphs it is flat in $|V|$ at fixed pathwidth and grows by a factor of 1.5 to 2 per unit of pathwidth, from 2 at pathwidth 3 to 9 at pathwidth 6; the largest values are 15 on the Petersen graph, 22 on the Shrikhande graph, 26 on the 4-cube, at least 25 on the Heawood graph, whose search was cut off between 25 and 32, and 32 on the Clebsch graph, which is twice the number of vertices and the largest ratio seen. Two laws fit every exact number, a constant times 2^{pw} and $2|V|$, and with pathwidth at most nine they cannot be told apart. The measure is well defined but not canonical: the shortest list of arbitrary generators is a constant factor shorter than the frame on every graph measured, up to four times, and the minor-pullback language of Section 5.5 is shorter still for clique minors, so a theorem has to name its language. If the first is the law, the conjecture's hard half follows on expanders, where pathwidth is linear in $|V|$; the data show the factor per unit of order and cannot see the exponent. A proof would need the expansion argument of Grohe and Marx (2009) redone for the invisible-fugitive game, since strategies transfer only from the visible robber to the invisible fugitive and not back. We found no prior statement about the size of a blockage.

Appendix G A tabu search over minors, and its certificates

The minor step of Section 4.7 builds one contracted copy of the graph per size by a greedy rule and tests it. On the classes of the dataset that it leaves open, the greedy minor typically certifies one less than the recorded width: the contraction has cost one unit of pathwidth, and the obstruction of the full width, if the recorded width is the truth, was crushed by one of its merges. The ladder never varies the minor. This appendix describes a search that does, states what it certifies, and proves the certificates sound in Lean (Appendix A, A.91). The program is `pathwidth_solver/cli/minor_tabu.c`, built on the same engine as the solver.

The search. The state is a set F of edges of G forming a forest; its components are the branch sets of a minor $H = G/F$, the graph on the components with an edge between two of them whenever G has one. A move adds an edge of G joining two different components, one more contraction, or removes an edge of F, one contraction undone, after which the component splits into two connected halves. The search is a tabu search in the plain sense: at every step every neighbour is evaluated, or a random sample of a fixed size when there are too many, the best non-tabu neighbour is taken, the reverse of the last seven moves is tabu, and a move that certifies the target width is always allowed. The walk is kept inside a band of minor sizes around its starting point, since a minor that keeps too few vertices has lost the obstruction and one that keeps too many cannot be decided within the budget. Two starting points are used: the greedy rung of the ladder, or, on an open class, the best minor certificate on record, from which the walk explores alternatives to the greedy contractions at that size.

A neighbour is scored by what it certifies. First, whether our corrected algorithm refutes the target width $w - 1$ on it within a small node budget, which is the event that closes the class; then, whether it still refutes the width the floor rests on, which says the known obstruction survived the move; then its contraction degeneracy; ties are broken at random within the band, with no preference for a size, and the program writes every minor on which a neighbour refuted either level to a log, as a contraction sequence in the certificate format below with the level and the step, so each such minor can be recovered and re-checked afterwards and where certificates occur in the band is measured rather than assumed. The chosen minor is then climbed by the engine under a larger budget, each refutation of a width v raising the floor to $v + 1$. The floor only rises, and the best certificate found is what the program reports.

The certificate. The output is the floor, how it was obtained, the minor's size, and a contraction sequence: the forest's edges are walked with a union-find, and for every edge whose ends are still in different components the pair (representative of one end, representative of the other) is emitted, meaning "contract the first into the second". Every pair is an edge of the contracted graph at the moment it is emitted, so the sequence is a contraction sequence in the sense of Theorem 4.11 and the checker of Section 5.5 replays it. The floor is of one of two kinds: *refutation*, the engine refuted width w on the minor, so the floor is $w + 1$; or *contraction degeneracy*, the minor step's bound on the minor.

What is proved. The Lean development models the walk and the certificate exactly (`Search/MinorTabu.lean`, 45 theorems, no `sorry`, no axiom beyond the three standard ones). Write r for the representative map the union-find maintains, idempotent on V; merging an edge pq

replaces r by r' with $r'(x) = r(q)$ when $r(x) = r(p)$ and $r(x)$ otherwise; the quotient G/r is the graph on the representatives with an edge between $r(p)$ and $r(q)$ whenever $pq \in E$ and $r(p) \neq r(q)$.

Theorem G.1 (the emitted sequence is a legal contraction sequence, and replays to the quotient)**.** *For any list of edges of G in any order, the sequence emitted by the walk is a contraction sequence of G in the sense of Theorem 4.11, and applying it to G yields G/r for the final representative map r, restricted to the surviving representatives. In particular* $\mathrm{pw}(G/F) \leq \mathrm{pw}(G)$ *(Appendix A, A.91).*

The proof rests on one identity that needs no edge at all: contracting $r(p)$ into $r(q)$ in G/r is the same graph as G/r' for the merged map r', so one step of the walk is one contraction; the edge pq only makes the step legal. Acyclicity of F is defined and never used: it is the program's bookkeeping, by which an addition shrinks the minor by exactly one vertex, and the theorems hold for any list of edges, cyclic or not.

Theorem G.2 (both floors are sound)**.** *Let F be a set of edges of G and $H = G/F$. If no layout of H has width at most w, then* $\mathrm{pw}(G) \geq w + 1$*. For every further contraction sequence on H and every vertex set S of the result, the minimum degree inside S is at most* $\mathrm{pw}(G)$*; in particular the contraction degeneracy of H, which is the largest minimum degree the minor step reads along its own contractions, is at most* $\mathrm{pw}(G)$ *(Appendix A, A.91).*

The second statement is quantified over every sequence because the minor step does not emit the contractions behind its degeneracy value (Section 5.5); each value it reads is a minimum degree of some contracted copy, and the theorem bounds every such value, so soundness needs no emitted sequence.

Theorem G.3 (a run reports a floor, and reaches w exactly by a certified minor)**.** *Model a run as a starting floor g (the program's `-L` input, trusted by hypothesis) and a list of visited forests each with a claimed floor justified by one of the two kinds; the output is the largest claim, or g. Then the output is at most* $\mathrm{pw}(G)$*; and for $w > g$, the output is at least w if and only if some visited minor carries a claim of at least w, in which case that minor has pathwidth at least w. In particular, if the output equals the recorded width w, some visited minor refuted $w - 1$ (Appendix A, A.91).*

The theorems are stated for an arbitrary list of visited forests: the tabu rule, the band, the sampling, the aspiration and the seeding decide which forests are visited and are irrelevant to soundness. One direction is deliberately not claimed: a visited minor whose pathwidth is w but on which the engine's budget ran out is not reported, so the hypothesis is the certified claim and not the bare property; the check below confirmed that the stronger statement is false. On the MOSP side, a refutation of width w on a minor of the customer graph proves that every pattern order opens more than $w + 1$ stacks, with no hypothesis on the instance.

Checked before stated. Every statement was checked by brute force before it was written: on every graph of the atlas with at most seven vertices (1,252 graphs) and 15,024 edge subsets, forests and cyclic sets alike, a quarter in shuffled order, the walk was replayed exactly as the program prints it, every emitted pair verified to be an edge of the current graph, the replay verified to equal the quotient, the quotient's pathwidth computed over all orderings and compared with the graph's (tight on 1,614), every degeneracy value re-derived from the program's own rule and bounded, and 3,756 simulated runs compared with the run theorems; no violation (the script `reports/scratch/minor_tabu_lean/check.py`). The axiom audit lists `propext`, `Classical.choice` and `Quot.sound` for every theorem.

Status. The walk does what it is meant to do where a minor is the only route. On 100 Rome graphs of 50 to 80 vertices with certified pathwidth 4 to 9, each seeded from a deliberately bad minor (random contractions down to 60% of the vertices, the minor's exact pathwidth as the floor, below the width on 97) and with every exact call capped at 3,000 nodes, so that the whole graph cannot be refuted, 20 seconds of walking raised the floor on 59 graphs, by a proper minor on 56, of 56% of the vertices at the median, and reached the width on 39, against 4 for the same program with no steps; the floor was never above the certified width. On 120 smaller certified graphs, unseeded, it reaches the width on all 120, there by the exact step on the whole graph. The scripts are `reports/scratch/minor_tabu/sanity/`. On the 91 open classes of the dataset, seeded from each class's best minor certificate on record, with 40, 80 or 120 CPU minutes per class by the gap between floor and width, eight walks at a time, it closed no class and raised no floor. What its log says is more useful than that count. Every neighbour refutation was written out with its minor: 243,310 refutations of the floor's level on 26 classes, on minors of 0.77 to 1.02 times the seed's size by class median, so the known obstruction survives nearly everywhere in the band and is cheap to re-prove there; and not one refutation of the target level, on any class, at any size. On 23 further classes, among them every `Random-150` and `Random-175` class and `myciel7`, no neighbour refuted even the floor's level within the per-neighbour budget of 400,000 nodes, and the walk ran on contraction degeneracy alone. The 42 classes whose seed minor has more than 120 vertices, the cap on the exact step, were not reached by the first run, whose size band was empty for them; a second run with the cap lifted is in progress. The reading is the one the duality gaps of Section 5.5 gave: on these classes, a minor small enough to decide within the budget does not have the full width, and the certificate of the last unit is the whole graph or a minor too large to decide cheaply, as on `Random-125-125-2-2_0`, where no minor of any size tried refutes $w - 1$. The run's record, with every claim replayable, is `reports/scratch/minor_tabu/`.